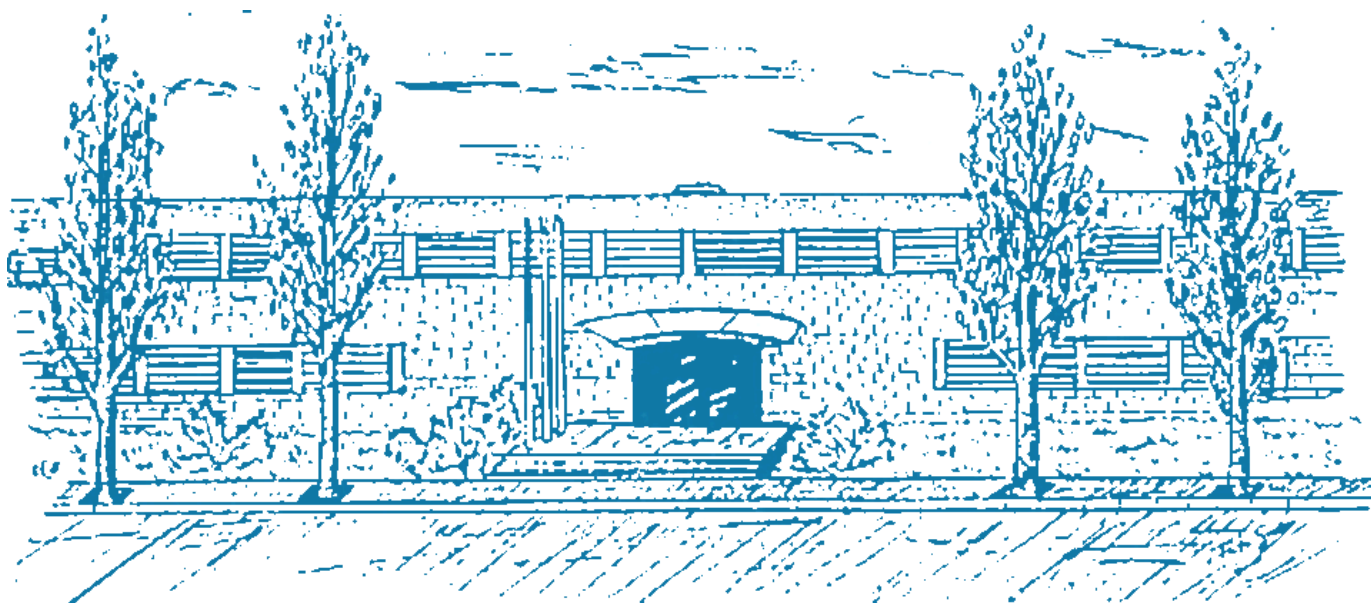


UNIVERSITAT POLITÈCNICA DE CATALUNYA - BarcelonaTech (UPC)
FME - FACULTAT DE MATEMÀTIQUES I ESTADÍSTICA

MASTER IN ADVANCED MATHEMATICS AND MATHEMATICAL ENGINEERING
(MAMME)
MASTER'S THESIS

Amortized Multi-Verifier Proofs: Implementation and Benchmarks



Author: Antonio Peso Vilella

Director: Nikitas Paspis and Javier Herranz

Department: Mathematics

Barcelona, 21/09/2026



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat de Matemàtiques i Estadística



Universitat Politècnica de Catalunya
Facultat de Matemàtiques i Estadística

Master in Advanced Mathematics and Mathematical Engineering
Master's thesis

Amortized Multi-Verifier Proofs: Implementation and Benchmarks

Antonio Peso Vilella

Supervised by Nikitas Paspis and Javier Herranz

September, 2026

Acknowledgments

I would like to express my gratitude to my tutor Javier and my good friend and tutor Nikitas. Thank you for your patience and your willingness to help me understand your work. It was fun.

I couldn't have done it without my family's support and my wife's encouragement. They are the pillars upon which I stand and I wouldn't have completed this project without them.

Abstract

Succinct non-interactive arguments of knowledge (SNARKs) allow a prover to convince a verifier that a computation was done correctly. SNARKs have been studied and developed so that the computational overhead of the prover and the verifier is reduced, though in general different SNARKs give different tradeoffs that should be considered in every context.

We study the recent amortization protocol of [Paslis, Ràfols and Zacharakis, ASIACRYPT 2026] in the multi-verifier context, where a single prover receives independent statements from K verifiers and outputs a proof for each, such that the prover amortizes work (as opposed to running independent proofs for each verifier). The statements are bivariate polynomial evaluations encoding matrices for an R1CS relation that is common to all the verifiers. In this setup, the bivariate polynomials are evaluated over random points and contain only *public* information. The cost of the prover is $\mathcal{O}(K \log K + s)$ group operations, where s is the joint *sparsity* of the R1CS matrices. The verifier’s cryptographic cost is $\mathcal{O}(\log K)$ and does not depend on the size of the circuit.

We analyze the amortization protocol step-by-step, providing all its building blocks and a computational analysis of its costs. We develop an implementation of the entire protocol instantiated over the Marlin SNARK [Chiesa et al., EUROCRYPT 2020] and run benchmarks to validate its expected computational savings. The benchmarks for $K = 128$ verifiers and matrix size $n = 2^{20}$ show: (i) a prover’s time drop from $\sim 1.7 \cdot 10^6$ ms to $\sim 8 \cdot 10^4$ ms (a $\sim 95\%$ reduction); and (ii) a verifier’s time rise from ~ 4 ms to $\sim 3.5 \cdot 10^1$ ms (an increase of roughly $\times 9$), while still presenting a speedup of up to $\times 325$ (compared to a verifier avoiding delegation and running Marlin locally). We help justify the practicality of the protocol by running additional experiments that quantify the split of the Marlin SNARK between its private and public parts. The results show a computational cost for the witness-independent part of Marlin ranging from $\sim 40\%$ to $\sim 60\%$ when varying the sparsity of the underlying R1CS between $2n$ and $8n$.

Keywords: cryptography; succinct arguments; reductions of knowledge; folding; implementation; benchmarks

Contents

Abstract	i
Contents	ii
1 Introduction	1
1.1 Motivation	1
1.2 Related Work	2
1.3 Contributions	2
1.4 Overview	3
2 Preliminaries	4
2.1 Groups and Fields – Lagrange Basis and Schwartz-Zippel Lemma	4
2.2 Relations, Arithmetic Circuits and R1CS	6
2.3 Hardness Assumptions	9
2.4 Interactive Proofs and Arguments of Knowledge	10
2.5 Polynomial Interactive Oracle Proofs and Polynomial Commitment Schemes	11
2.6 The KZG Polynomial Commitment Scheme	13
2.7 Merkle Trees and the Fiat-Shamir Transform	14
2.8 Succinct Non-Interactive Arguments of Knowledge	15
2.9 Reductions of Knowledge	18
2.10 Local Folding	20
3 Amortization Protocol	22
3.1 Relations	22
3.2 Reductions	24
3.3 Local Folding Scheme Compiler	28
3.4 Protocol	30
4 SNARK Instantiation: Marlin	35
4.1 Witness Dependence	35
4.2 The Lincheck Problem	38
5 Results	42
5.1 Implementation Details	42
5.2 Benchmarks	46
6 Conclusion and Future Work	52
6.1 Conclusion	52
6.2 Future Work	52

Chapter 1

Introduction

In this chapter we will provide an introduction to the context of the thesis and its motivation, as well as previous related work, contributions and an overview of its structure.

1.1 Motivation

Zero-knowledge proof systems [GMR89] have become central to cryptographic research in the last decades. They provide a means to prove the correctness of a statement without revealing any further information, which fits many real-world scenarios in digital services. The original interactive nature of zero-knowledge proofs was a limiting factor to applications where interactivity was restricted, so a new line of research developed *non-interactive proofs* [BFM88], where the prover would single-handedly compute a proof and a verifier would only check its validity. Subsequent efforts tried refining non-interactive proofs to be succinct (i.e. short proofs and fast verification time), which resulted in the definition of *succinct non-interactive arguments of knowledge* (SNARKs) [Kil92, Mic94].

SNARKs are a great tool to provide proofs of correct computation in settings where the prover has more computational resources than the verifier and the proofs need to be relatively small (say a few KB). Applications in *Blockchain* have evolved to favor a faster prover and account for *scalability*. Many of the underlying computations to be proven are very large in size and require specialized hardware. Therefore, a new line of research focuses on *SNARK delegation*, where a local prover wants to delegate proving part of a proof to a specialized provider or *server*. In this setup, the local prover acts as a verifier and the server as a prover.

This thesis works under the setup of SNARK delegation and, more precisely, an independent multi-verifier context, where a server wants to provide proof of a bivariate polynomial evaluation of the form $P(\alpha, \beta) = y$ to K independent verifiers. The polynomial P encodes a common matrix $\mathbf{M} \in \mathbb{F}^{n \times n}$ that represents an arithmetic circuit over a finite field $\mathbb{F}$. In many SNARK proof systems, the prover has work that can be classified as witness-dependent and witness-independent. The witness-dependent part can be delegated using *collaborative SNARKs* [SVdV16, CLMZ23, LZW⁺25, GGJ⁺25], which focus on designing protocols that preserve privacy even when the witness is needed in the protocol. An alternative to collaborative SNARKs is the usage of *Fully-Homomorphic Encryption* (FHE) [RAD⁺78, Gen09], which is based on encrypting the witness and running the underlying arithmetic circuit on the encrypted data. In this setup, it might not be necessary to provide a proof of computation for the witness-dependent part, since the party running it cannot decrypt the data. Of course, FHE has a different set of tradeoffs

and might still incur impractical overhead for big computations.

More precisely, this work implements the delegation protocol of [PRZ26a], which provides a *local folding scheme* to amortize proving all K independent statements in the multi-verifier setting mentioned above. In particular, the protocol allows for a server to perform less computational work than proving all K statements individually (by running a SNARK with each verifier separately), while only adding a small cost to the verifiers, which is independent of the size of the underlying circuit. This thesis analyzes the work of [PRZ26a] by implementing a Rust program¹, by running benchmarks on it and by studying the results of such experiments.

1.2 Related Work

We summarize several works in this section that develop intersecting topics with this thesis.

Multi-Verifier Setting. This setting was considered originally for vector and polynomial commitments in [SCP⁺22, CNR⁺22, ZXH⁺22, TCZ⁺20, YLF⁺22]. It was generalized in subsequent work by [RZ23] by means of a compiler capturing many-to-one reductions, which culminated in this thesis’ reference work of [PRZ26a]. Previous works [YLF⁺22, ZXH⁺22] had introduced the use of the Fiat–Shamir Merkle Tree paradigm, but it was the work of [PRZ26a] which provided a proof in the general reduction-of-knowledge framework of [KP23].

Reductions of Knowledge. [BGH19, BCMS20, BCL⁺21] develop the literature on accumulation schemes and [KST22] introduces the concept of folding schemes. They were generalized in [KP23] by introducing reductions of knowledge and providing a simpler framework to express complex protocols. The work of [BMNW25] extends these notions as Interactive Oracle Reductions.

Private Proof Delegation. Parallel to this work’s public proof delegation setting, a line of work focuses on its private counterpart by constructing collaborative zkSNARKs [OB22]. In these, the witness is split into pieces and sent separately to different parties, which jointly output a SNARK proof. Further work has developed on collaborative SNARKs [SVdV16, CLMZ23, LZW⁺25, GGJ⁺25] and can be seen as complementary to this thesis, which follows the work of [PRZ26a] on the witness-independent part of some SNARKs.

1.3 Contributions

This thesis is complementary to the work of [PRZ26a], where the authors introduce a folding scheme to amortize witness-independent statements from K independent verifiers. The contributions of this thesis are:

1. provide an implementation of the amortization protocol, bringing the folding scheme closer to industrial applications,
2. testing theoretical costs for the prover and verifier, giving concrete numbers on simulations of real-world scenarios,

¹<https://github.com/apesvil/amortized-proofs-masters-thesis>

3. running experiments that help justify the existence of a delegation service to prove witness-independent statements, and
4. give a complementary analysis of [PRZ26a] to facilitate outreach of its contents.

1.4 Overview

The thesis is structured as follows:

- Chapter 2 introduces the necessary background and notation for the rest of the thesis, including relations, reductions of knowledge and folding schemes.
- Chapter 3 presents the amortization protocol as a local folding scheme by naming its building blocks and the theorems that glue them together.
- Chapter 4 gives a detailed description of the SNARK Marlin [CHM⁺20], which is used in the implementation to instantiate the amortization protocol.
- Chapter 5 exposes the main implementation details and the experimental results.
- Chapter 6 summarizes the findings in this thesis and names some open questions as future work.

Chapter 2

Preliminaries

In this chapter, we introduce the mathematical and cryptographic preliminaries used throughout this thesis. We begin with basic notation, followed by the algebraic structures we work over, the Lagrange basis and the Schwartz-Zippel lemma; then the vocabulary of languages and relations together with circuit satisfiability and rank-1 constraint systems; the hardness assumptions; polynomial interactive oracle proofs and polynomial commitment schemes, and the KZG scheme that instantiates them; Merkle trees and the Fiat-Shamir transform; SNARKs; reductions of knowledge; and finally local folding. The goal of this chapter is to provide the reader with the necessary background to understand the subsequent chapters, which will build upon these concepts to present our main results.

Notation ([PRZ26a]). We write $[n] = \{1, \dots, n\}$. Collections are often indexed over the hypercube $\{0, 1\}^\kappa$: for a set $S = \{s_1, \dots, s_{2^\kappa}\}$ we equivalently write $S = \{s_{\mathbf{b}}\}_{\mathbf{b} \in \{0, 1\}^\kappa}$, where $s_{\mathbf{b}} = s_i$ for $i = 1 + \sum_j b_j 2^{j-1}$, and we use the shorthand $s_{\mathbf{b}\mathbf{b}'}$, $s_{\mathbf{b}'\mathbf{b}}$ and $s_{\mathbf{b}_1\mathbf{b}_2}$ for the corresponding concatenated bitstrings. The empty string is denoted ϵ . We write $x \leftarrow S$ for sampling a uniform element from S , and $y \leftarrow \mathcal{A}^\mathcal{O}(x)$ for executing $\mathcal{A}$ on input x with fresh randomness and oracle access to $\mathcal{O} = (\mathcal{O}_1, \dots, \mathcal{O}_\ell)$. Vectors are denoted $\mathbf{v} = (v_1, \dots, v_n) \in \mathbb{F}^n$ and matrices $\mathbf{M} = (m_{ij})_{i,j=1}^n$. The *sparsity* $\|\mathbf{M}\|$ of a matrix is its number of non-zero entries, and the sparsity $\|p\|$ of a polynomial is its number of non-zero coefficients with respect to a fixed basis. The subspace of polynomials of degree less than d in $\mathbb{F}[X]$ is denoted $\mathbb{F}^{<d}[X]$. We denote the security parameter by λ and assume all inputs are of length polynomial in λ .

2.1 Groups and Fields – Lagrange Basis and Schwartz-Zippel Lemma

The vast majority of cryptographic applications rely on the algebraic structure of groups and fields. In this section, we introduce the basic definitions and properties of these structures, specifically focusing on finite fields and multiplicative subgroups. Throughout, $\mathbb{F}$ is a finite field and $\mathbb{H} = \{h_1, \dots, h_n\}$ is a multiplicative subgroup of $\mathbb{F}^*$ of order n .

A specific type of group that is particularly useful in cryptography is a bilinear group, which consists of three cyclic groups of prime order and a bilinear map between them. Bilinear groups are used in many cryptographic protocols, including pairing-based cryptography. Pairings are essential in constructing efficient zero-knowledge proofs and other cryptographic primitives, such as identity-based encryption and short signatures.

Definition 2.1 (Bilinear group). A *bilinear group* is a tuple $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2)$ where

$\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ are cyclic groups of prime order p , g_1 generates $\mathbb{G}_1$, g_2 generates $\mathbb{G}_2$, and $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is an efficiently computable map such that

- **Bilinearity.** $e(g_1^a, g_2^b) = e(g_1, g_2)^{ab}$ for all $a, b \in \mathbb{Z}_p$.
- **Non-degeneracy.** $e(g_1, g_2) \neq 1_{\mathbb{G}_T}$.

Throughout this work, we will use the concept of negligible functions to describe the security of cryptographic protocols. A function is negligible if it decreases faster than the inverse of any polynomial, which means that, as the security parameter increases, the probability of an adversary successfully breaking the protocol becomes vanishingly small.

Definition 2.2 (Negligible function). A function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is *negligible* if for every $c \in \mathbb{N}$ there exists $\lambda_0 \in \mathbb{N}$ such that $\text{negl}(\lambda) < \lambda^{-c}$ for all $\lambda > \lambda_0$.

Polynomials over finite fields can be represented in various ways, and one common representation is the Lagrange basis. The Lagrange basis is a set of polynomials that can be used to interpolate a polynomial given its values at specific points. This representation is particularly useful in cryptography, as it allows for efficient evaluation and manipulation of polynomials. The structure of the zeros of polynomials over large fields is particularly useful for constructing efficient zero-knowledge proofs and other cryptographic protocols.

Definition 2.3 (Lagrange basis [PRZ26a]). Let $\mathbb{H} = \{h_1, \dots, h_n\}$ be a multiplicative subgroup of $\mathbb{F}^*$. We denote by $\lambda_{\mathbb{H}}(X) = (\lambda_{h_1}(X), \dots, \lambda_{h_n}(X))$ the vector of Lagrange polynomials associated to $\mathbb{H}$, that is, the unique polynomials of degree less than n satisfying $\lambda_{h_i}(h_j) = 1$ if $i = j$ and 0 otherwise. We omit the subscript and write $\lambda(X) = (\lambda_1(X), \dots, \lambda_n(X))$ when the context allows.

The following proposition provides a sparse representation of the inner product of Lagrange polynomials, which is essential to the work of [PRZ26a] and it will appear frequently throughout this thesis. Observe that the resulting numerator and denominator polynomials can be sparsely represented with 3 coefficients and 2 coefficients, respectively.

Proposition 2.4 ([CFF⁺21], as stated in [PRZ26a]). *For every $\alpha \in \mathbb{F}$,*

$$\lambda(\alpha)^\top \lambda(X) = \frac{(X^n - 1)\alpha - (\alpha^n - 1)X}{n(X - \alpha)}.$$

A commonly used matrix representation in the literature consists of encoding matrices as bivariate polynomials over a finite domain $\mathbb{H}$. It is quite straightforward to picture a bijection between an element in the i -th row and j -th column and the value of a polynomial evaluated at the points h_i and h_j of $\mathbb{H}$.

Proposition 2.5 (Matrix encoding of bivariate polynomials [PRZ26a]). *Bivariate polynomials $P(X, Y) \in \mathbb{F}[X, Y]$ of individual degree at most $n - 1$ are in bijection with their evaluation matrices $\mathbf{M} = (P(h_i, h_j))_{i, j \in [n]}$ over the domain $\mathbb{H}$, and*

$$P(\alpha, \beta) = \lambda(\alpha)^\top \mathbf{M} \lambda(\beta).$$

If P is sparse then so is $\mathbf{M}$, and conversely every matrix $\mathbf{M}$ over $\mathbb{H} \times \mathbb{H}$ defines a corresponding bivariate polynomial of individual degree at most $n - 1$.

A fundamental result in cryptography is the Schwartz-Zippel lemma, which provides a probabilistic bound on the number of roots of a non-zero polynomial over a finite field. This lemma is particularly useful in the analysis of cryptographic protocols, as it allows us to reason about the probability of certain events occurring when working with polynomials. The lemma states that if a non-zero polynomial is evaluated at random points from a finite field, the probability that it evaluates to zero is bounded by the degree of the polynomial divided by the size of the field. It can be stated as in Corollary 2.7, which is used to check polynomial identities at random points.

Lemma 2.6 (Schwartz-Zippel [Sch80, Zip79]). *Let $P \in \mathbb{F}[X_1, \dots, X_m]$ be a non-zero polynomial of total degree d over a field $\mathbb{F}$, and let $S \subseteq \mathbb{F}$ be finite. Then*

$$\Pr[P(r_1, \dots, r_m) = 0 \mid r_1, \dots, r_m \leftarrow S] \leq \frac{d}{|S|}.$$

Corollary 2.7. *Let $P, Q \in \mathbb{F}[X_1, \dots, X_m]$ be distinct polynomials of total degree at most d and let $S \subseteq \mathbb{F}$ be finite. Then $\Pr[P(\mathbf{r}) = Q(\mathbf{r})] \leq d/|S|$ over $\mathbf{r} \leftarrow S^m$.*

The main takeaways from this section are that (i) the Lagrange basis provides a sparse representation of polynomials, which also encodes elements of a matrix as a bivariate polynomial; and (ii) that the Schwartz-Zippel lemma provides a probabilistic bound that allows a verifier to check polynomial identities at random points.

2.2 Relations, Arithmetic Circuits and R1CS

Every proof system in this thesis proves membership in some relation, so we first fix the vocabulary of languages and relations, and in particular the indexed and ternary variants that the later definitions are stated over. We then specialize it to the two computational models the protocol is concerned with: arithmetic circuits, which are how a computation is written down, and rank-1 constraint systems, which are how it is arithmetized before being proven.

Definition 2.8 (Language). A *language* $\mathcal{L}$ over an alphabet Σ is a subset $\mathcal{L} \subseteq \Sigma^*$. We say that $\mathcal{L}$ is decidable in time t if there is a deterministic algorithm that on input $\mathbf{x} \in \Sigma^*$ decides whether $\mathbf{x} \in \mathcal{L}$ in time $t(|\mathbf{x}|)$. In general, we will consider $\Sigma = \{0, 1\}$.

Definition 2.9 (Relation and its language). A *relation* $\mathcal{R}$ is a set of pairs $(\mathbf{x}, \mathbf{w}) \in \Sigma^* \times \Sigma^*$, where $\mathbf{x}$ is called the *statement* (or instance) and $\mathbf{w}$ the *witness*. We write $\mathcal{R}(\mathbf{x}) = \{\mathbf{w} : (\mathbf{x}, \mathbf{w}) \in \mathcal{R}\}$ for the set of valid witnesses of $\mathbf{x}$. The *language of* $\mathcal{R}$ is

$$\mathcal{L}(\mathcal{R}) = \{\mathbf{x} \in \Sigma^* \mid \exists \mathbf{w} \text{ such that } (\mathbf{x}, \mathbf{w}) \in \mathcal{R}\}.$$

Relations as defined in Definition 2.9 are common in complexity theory and describe well-known languages such as **NP**. In this thesis though, relations require an extra element in the definition set that works as an *indexer*. This indexer can be viewed as complementary information for both the Prover and the Verifier that is computed in a *setup* stage in some cryptographic protocols. This motivates the following definitions.

Definition 2.10 (Indexed and ternary relations [PRZ26b, PRZ26a]). An *indexed relation* $\mathcal{R}$ is a set of triples $\{(i, \mathbf{x}, \mathbf{w})\}$, where i is the index, $\mathbf{x}$ is the instance and $\mathbf{w}$ is the witness. A *ternary relation* over a parameter space $\mathbb{P}$ is the set $\{(\mathbf{pp}, \mathbf{x}, \mathbf{w}) : \mathbf{pp} \in \mathbb{P}\}$.

In this thesis, we will work exclusively over ternary relations with a set of *public parameters* $\mathbf{pp}$, though many fundamental definitions and theorems in the literature are described more generally over indexed relations.

A very natural operation that arises when working with relations is their Cartesian product. In the case of ternary relations, the Cartesian product is defined over ternary relations with the same set of public parameters $\mathbf{pp}$:

Definition 2.11 (Product of relations, [PRZ26a, Def. 4.4.1]). Let $\mathcal{R}_1, \mathcal{R}_2$ be ternary relations. We define

$$\mathcal{R}_1 \times \mathcal{R}_2 = \{\mathbf{pp}, (\mathbf{x}_1, \mathbf{w}_1), (\mathbf{x}_2, \mathbf{w}_2) \mid (\mathbf{pp}, \mathbf{x}_1, \mathbf{w}_1) \in \mathcal{R}_1 \wedge (\mathbf{pp}, \mathbf{x}_2, \mathbf{w}_2) \in \mathcal{R}_2\}.$$

Furthermore, for $n \in \mathbb{N}$ we define $\mathcal{R}^n = \underbrace{\mathcal{R} \times \cdots \times \mathcal{R}}_{n \text{ times}}.$

The very core of zero-knowledge proofs and verifiable computation consists of the following definitions and theorem, where we establish an equivalence between a *computational model* and a set of *arithmetic constraints*. This connection allows for algebraic manipulation over statements that constitute the inputs and outputs of any computation, creating a bridge between computation and algebra.

We will use *arithmetic circuits* over a finite field $\mathbb{F}$ as a computational model, which are expressive enough to encode any computation of a fixed-size input and output over $\mathbb{F}$.

Definition 2.12 (Arithmetic circuit [Tha22, CFF⁺21]). An *arithmetic circuit* C over a field $\mathbb{F}$ is a directed acyclic graph whose vertices are called *gates* and whose edges are called *wires*, such that

- every gate of in-degree 0 is an *input gate*, labeled either with a variable or with a constant of $\mathbb{F}$,
- every other gate has in-degree 2 and is labeled with an operation in $\{+, \times\}$, computing respectively the sum or the product in $\mathbb{F}$ of the values carried by its two incoming wires,
- a designated ordered tuple of ℓ_{out} gates are the *output gates*.

The *size* $|C|$ of C is its number of gates and the *depth* of C is the length of the longest directed path from an input gate to an output gate. For an assignment of field elements to the variables, $C(\cdot)$ denotes the tuple of values carried by the output gates.

Let C be a fixed circuit, $\mathbf{x}$ an input and $\mathbf{y}$ an output that satisfy $C(\mathbf{x}) = \mathbf{y}$. The tuple $(C, (\mathbf{x}, \mathbf{y}), \perp)$ can be interpreted as an element of an indexed relation with index C , instance $(\mathbf{x}, \mathbf{y})$ and no witness. In cryptography, it is common to split the input into two parts: a public input $\mathbf{x}$ and a private input (or witness) $\mathbf{w}$, so that we can model actual computations where part of the input should be kept as private. This setup motivates the following definition:

Definition 2.13 (Arithmetic circuit satisfiability, [CFF⁺21, Def. 24]). Let $\mathbb{F}$ be a finite field. The *universal arithmetic circuit satisfiability relation* $\mathcal{R}_{\text{CSAT}}$ is the set of triples $(i, \mathbf{x}, \mathbf{w})$ where

- the index i consists of an arithmetic circuit $C : \mathbb{F}^{\ell_{\text{in}}} \times \mathbb{F}^{\ell_{\text{wit}}} \rightarrow \mathbb{F}^{\ell_{\text{out}}}$ with ℓ_{in} public inputs, ℓ_{wit} private inputs and ℓ_{out} public outputs,

- the instance $\mathbf{x}$ consists of a pair of vectors $(\mathbf{x}, \mathbf{y}) \in \mathbb{F}^{\ell_{\text{in}}} \times \mathbb{F}^{\ell_{\text{out}}}$,
- the witness $\mathbf{w}$ consists of a vector $\mathbf{w} \in \mathbb{F}^{\ell_{\text{wit}}}$,

such that $C(\mathbf{x}, \mathbf{w}) = \mathbf{y}$.

Figure 2.1 shows an example of an arithmetic circuit and its corresponding instance of $\mathcal{R}_{\text{CSAT}}$.

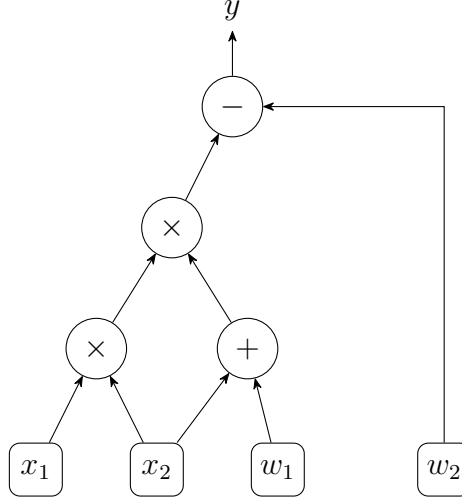


Figure 2.1: An arithmetic circuit over $\mathbb{F}$ with $\ell_{\text{in}} = \ell_{\text{wit}} = 2$ and $\ell_{\text{out}} = 1$, computing $C(\mathbf{x}, \mathbf{w}) = (x_1 \cdot x_2)(x_2 + w_1) - w_2$ on public input $\mathbf{x} = (x_1, x_2)$ and witness $\mathbf{w} = (w_1, w_2)$. The corresponding instance of $\mathcal{R}_{\text{CSAT}}$ is the pair $(\mathbf{x}, y)$, where y is the claimed output. Subtraction is shorthand for addition with the constant -1 scaling the right operand.

As mentioned before in this section, we prefer to work over an algebraic structure that encodes arithmetic circuits, in order to derive protocols that can verify whether an instance-witness pair $(\mathbf{x}, \mathbf{w})$ belongs to $\mathcal{R}_{\text{CSAT}}$. The encoding used in this thesis is called *rank-1 constraint systems* or R1CS.

Definition 2.14 (Rank-1 constraint system, [CHM⁺20]). The *rank-1 constraint system relation* $\mathcal{R}_{\text{R1CS}}$ is the set of triples $(i, \mathbf{x}, \mathbf{w})$ where

- the index i consists of size bounds $n, \ell \in \mathbb{N}$ and matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{n \times n}$,
- the instance $\mathbf{x}$ consists of a vector $\mathbf{x} \in \mathbb{F}^{\ell}$,
- the witness $\mathbf{w}$ consists of a vector $\mathbf{w} \in \mathbb{F}^{n-\ell}$,

such that for $\mathbf{z} = (\mathbf{x}, \mathbf{w})$ it holds that $\mathbf{Az} \circ \mathbf{Bz} = \mathbf{Cz}$, where $\circ$ denotes the Hadamard product.

Linear operations on vectors and matrices offer a clear mathematical advantage over circuits. The following theorem bridges both concepts and allows us to derive information from the underlying circuit while working over the corresponding R1CS relation.

Theorem 2.15 ([GGPR13], as stated in [CFF⁺21, Thm. 19]). *Let $C : \mathbb{F}^{\ell_{\text{in}}} \times \mathbb{F}^{\ell_{\text{wit}}} \rightarrow \mathbb{F}^{\ell_{\text{out}}}$ be an arithmetic circuit with N multiplication gates. Then there exists an R1CS $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{(N+\ell_{\text{out}}) \times n}$ with $n = \ell_{\text{in}} + \ell_{\text{out}} + N + 1$ such that for any $\mathbf{x}$ and $\mathbf{y}$, there exists $\mathbf{w}$ with $C(\mathbf{x}, \mathbf{w}) = \mathbf{y}$ if and only if there exists $\mathbf{w}'$ that makes $(1, \mathbf{x}, \mathbf{y})$ accepted by $(\mathbf{A}, \mathbf{B}, \mathbf{C})$, that is, such that $\mathbf{z} = (1, \mathbf{x}, \mathbf{y}, \mathbf{w}')$ satisfies $\mathbf{Az} \circ \mathbf{Bz} = \mathbf{Cz}$.*

Remark 2.16. Definition 2.14 uses square matrices; the $(N + \ell_{\text{out}}) \times n$ system of Theorem 2.15 is brought into that form by padding with zero rows. The leading entry 1 of $\mathbf{z}$ is the constant wire, so the R1CS instance corresponding to an $\mathcal{R}_{\text{CSAT}}$ instance $(\mathbf{x}, \mathbf{y})$ is $(1, \mathbf{x}, \mathbf{y}) \in \mathbb{F}^\ell$ with $\ell = \ell_{\text{in}} + \ell_{\text{out}} + 1$.

2.3 Hardness Assumptions

The security of many cryptographic algorithms and protocols is derived from the assumption that some problems are computationally hard. These *hardness assumptions* are not mathematically proven propositions, but rather specific problems that are *believed* to have no efficient (i.e. polynomial-time) algorithm that solves them. Of course, this belief is founded on decades of research and thousands of unsuccessful attempts at breaking these problems, so the cryptographic community agrees that classical computational methods are incapable of breaking them.

The following two assumptions carry the weight of many modern-day protocols, guarding financial institutions, governments and secure internet communication. The *discrete logarithm* assumption states that the probability of computing the discrete logarithm s from a group generator g and a group element g^s is negligible. The q -DLOG assumption states that given two group generators g_1, g_2 and two vectors of $q + 1$ powers $(g_i, g_i^s, g_i^{s^2}, \dots, g_i^{s^q})$ for $i \in \{1, 2\}$ in a bilinear group, it is computationally infeasible for a polynomial-time adversary to output the discrete logarithm s . The discrete logarithm problem underlies public-key cryptography since [DH76], and q -type assumptions over bilinear groups go back to [BB04]; the formulation of q -DLOG we use is the one under which [FKL18] carries out its analysis in the algebraic group model.

Definition 2.17 (Discrete logarithm assumption [DH76]). Let $\mathbb{G}$ be a cyclic group of prime order p with generator g . The *discrete logarithm assumption* holds in $\mathbb{G}$ if for every PPT adversary $\mathcal{A}$,

$$\Pr[s = s' \mid s \leftarrow \mathbb{Z}_p, s' \leftarrow \mathcal{A}(g, g^s)] \leq \text{negl}(\lambda).$$

Definition 2.18 (q -DLOG assumption [FKL18]). Let $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2)$ be a bilinear group. The *q -DLOG assumption* holds if for every PPT adversary $\mathcal{A}$,

$$\Pr \left[s = s' \mid \begin{array}{l} s \leftarrow \mathbb{Z}_p \\ s' \leftarrow \mathcal{A}(\{g_1^{s^i}\}_{i=0}^q, \{g_2^{s^i}\}_{i=0}^q) \end{array} \right] \leq \text{negl}(\lambda).$$

A common assumption in the literature is to restrict adversaries to be *algebraic*: whenever one outputs a group element, it must also reveal how that element was built from the group elements it has already received. Much of the pairing-based SNARK literature is analyzed under this restriction. Groth’s scheme [Gro16] was first proven knowledge sound in this model by [FKL18], PLONK [GWC19] carries out its soundness analysis in it, and Lunar [CFF⁺21] proves its commitment-opening argument secure in it. The extractability of KZG that we rely on in Theorem 2.27 is established in the same model.

Definition 2.19 (Algebraic algorithm [FKL18]). An algorithm $\mathcal{A}$ executed in a group $\mathbb{G}$ is *algebraic* if whenever it outputs a group element $h \in \mathbb{G}$ it additionally outputs a vector $(a_1, \dots, a_m) \in \mathbb{Z}_p^m$ such that $h = \prod_{i=1}^m g_i^{a_i}$, where $g_1, \dots, g_m \in \mathbb{G}$ are all the group elements $\mathcal{A}$ has received as input so far. The *algebraic group model* (AGM) restricts all adversaries to be algebraic.

2.4 Interactive Proofs and Arguments of Knowledge

This thesis (and the field of zero-knowledge proofs in general) is based on the concept of *interactive proofs*. These can be seen as protocols between two parties: a *prover* $\mathcal{P}$ and a *verifier* $\mathcal{V}$. In these protocols, $\mathcal{P}$ is a (possibly probabilistic) algorithm that wants to convince a deterministic algorithm $\mathcal{V}$ of the validity of some statement by exchanging pairs of messages organized in *rounds*. Definition 2.20 formalizes the concept:

Definition 2.20 (ℓ -round interactive protocol, following [GMR89]). An ℓ -round *interactive protocol* is a pair $(\mathcal{P}, \mathcal{V})$ of interactive algorithms. On common input x and prover input w , the two parties exchange messages in ℓ rounds: in round $i \in [\ell]$, $\mathcal{P}$ sends a message a_i and $\mathcal{V}$ replies with a message c_i . When $\mathcal{V}$ moves first, the interaction additionally opens with a message c_0 sent before the first round. After the last round $\mathcal{V}$ outputs a bit, denoted $\langle \mathcal{P}(x, w), \mathcal{V}(x) \rangle \in \{0, 1\}$, and we write

$$\text{tr} = (c_0, a_1, c_1, \dots, a_\ell, c_\ell)$$

for the *transcript* of the interaction, dropping c_0 when $\mathcal{P}$ moves first. The protocol is *public coin* if every c_i is a uniformly random string sampled independently of the history, and $\mathcal{V}$'s output is a deterministic function of x and tr .

Both conventions occur below. The PIOP of Definition 2.23 has $\mathcal{P}$ move first, so its transcript starts at a_1 ; the protocols that Definition 2.31 transforms open with a verifier message, the challenge γ'_0 , and it is precisely that opening message which FS_{MT} (Definition 3.9) derives from a Merkle root so that all K verifiers share it.

In a public-coin protocol the verifier's messages are random coin tosses, so they can be produced by anyone. This is exactly what allows $\mathcal{V}$'s messages to be replaced by random oracle queries in Definition 2.31. Every protocol compiled in Chapter 3 is public coin for this reason. Public-coin protocols are also called *Arthur–Merlin* protocols, after [Bab85].

Definition 2.21 (Interactive proof and interactive argument, [GMR89, BCC88]). Let $\mathcal{R}$ be a relation. An ℓ -round interactive protocol $(\mathcal{P}, \mathcal{V})$ is an *interactive proof* for $\mathcal{L}(\mathcal{R})$ with soundness error ϵ if it satisfies

- **Completeness.** For every $(x, w) \in \mathcal{R}$, $\Pr[\langle \mathcal{P}(x, w), \mathcal{V}(x) \rangle = 1] = 1$.
- **Soundness.** For every $x \notin \mathcal{L}(\mathcal{R})$ and every, possibly computationally unbounded, prover $\mathcal{P}^*$, $\Pr[\langle \mathcal{P}^*(x), \mathcal{V}(x) \rangle = 1] \leq \epsilon$.

It is an *interactive argument* if soundness is only required to hold for every PPT prover $\mathcal{P}^*$, with $\epsilon = \text{negl}(\lambda)$.

Therefore, the distinction between proof and argument can be summarized as: a proof convinces against an unbounded cheating prover while an argument only against an efficient (probabilistic polynomial-time) one. Everything in this thesis is an argument, since the protocols of Chapter 3 derive their soundness from the binding property of the polynomial commitment scheme, which by Theorem 2.27 holds only under the q -DLOG assumption of Definition 2.18: a prover able to recover τ from the commitment key could open a commitment to any value it likes and break every reduction at once.

The property of soundness is generally not enough the setting of this thesis. A verifier delegating a computation wants more than the assurance that a witness exists: it wants the guarantee that the prover it is talking to actually possesses one. That guarantee is formalized by requiring that a witness can be *extracted* from any prover that succeeds too often.

Definition 2.22 (Argument of knowledge, [BG93]). An interactive argument $(\mathcal{P}, \mathcal{V})$ for $\mathcal{R}$ is an *argument of knowledge* with knowledge error ϵ if there exist a polynomial $\text{poly}(\cdot)$ and a probabilistic oracle machine $\mathcal{E}$, the *extractor*, such that for every PPT prover $\mathcal{P}^*$ and every $\mathbf{x}$, if

$$\Pr [\langle \mathcal{P}^*(\mathbf{x}), \mathcal{V}(\mathbf{x}) \rangle = 1] = \epsilon^* > \epsilon,$$

then $\mathcal{E}^{\mathcal{P}^*}(\mathbf{x})$, with oracle access to $\mathcal{P}^*$, outputs $\mathbf{w}$ with $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$ in expected time $\text{poly}(|\mathbf{x}|)/(\epsilon^* - \epsilon)$. If soundness and extraction hold against unbounded $\mathcal{P}^*$, it is a *proof of knowledge*.

The extractor is recurrent throughout the chapter: it reappears for PIOPs in Definition 2.23, for non-interactive arguments in Definition 2.32 and for reductions of knowledge in Definition 2.36. Those definitions, though slightly different in some parts, follow the same principle: a prover that convinces noticeably more often than the knowledge error must hand over a witness under interrogation.

2.5 Polynomial Interactive Oracle Proofs and Polynomial Commitment Schemes

This section introduces two primitives that are used to construct modern cryptographic protocols: *polynomial interactive oracle proofs* or PIOPs and *polynomial commitment schemes* or PCS. PIOPs are information-theoretic primitives that use the concept of *interactive oracle* to model polynomial queries. A PCS is a cryptographic scheme that substitutes the oracle in a PIOP, so that the protocol can be instantiated. The separation of both is particularly useful to analyze the security of the underlying PIOP in the interactive oracle model and prove properties such as completeness and knowledge soundness.

Definition 2.23 (Polynomial interactive oracle proof [CHM⁺20], with knowledge soundness as in [BFS20, Def. 7]). Let $\mathcal{R}$ be an indexed relation, $\mathbb{F}$ a field and $d \in \mathbb{N}$ a degree bound. A k -round *polynomial interactive oracle proof* (PIOP) for $\mathcal{R}$ over $\mathbb{F}$ with degree bound d is a pair of interactive algorithms $(\mathcal{P}, \mathcal{V})$ such that, on common input $(i, \mathbf{x})$ and prover input $\mathbf{w}$, in each round $i \in [k]$ the prover sends an oracle to a polynomial $p_i(X) \in \mathbb{F}^{<d}[X]$ and the verifier replies with a message $r_i \in \mathbb{F}$. After the k -th round $\mathcal{V}$ may query the oracles $p_1, \dots, p_k$ at points of its choice and outputs a bit. The system satisfies

- **Completeness.** For every $(i, \mathbf{x}, \mathbf{w}) \in \mathcal{R}$, the interaction of $\mathcal{P}(i, \mathbf{x}, \mathbf{w})$ with $\mathcal{V}(i, \mathbf{x})$ makes $\mathcal{V}$ accept with probability 1.
- **Knowledge soundness with error ϵ .** There exist a polynomial $\text{poly}(\cdot)$ and a probabilistic oracle machine $\mathcal{E}$, the *extractor*, such that the following holds for every prover $\mathcal{P}^*$ whose oracles are polynomials in $\mathbb{F}^{<d}[X]$ and every pair $(i, \mathbf{x})$ admitting a witness. If $\mathcal{P}^*$ convinces the verifier with probability

$$\Pr [\langle \mathcal{P}^*(i, \mathbf{x}), \mathcal{V}(i, \mathbf{x}) \rangle = 1] = \epsilon^*,$$

then $\mathcal{E}^{\mathcal{P}^*}(i, \mathbf{x})$, with oracle access to $\mathcal{P}^*$, runs in time $\text{poly}(|\mathbf{x}|)/\epsilon^*$ and outputs $\mathbf{w}$ such that $(i, \mathbf{x}, \mathbf{w}) \in \mathcal{R}$ with probability at least

$$1 - \frac{\epsilon}{\epsilon^*}.$$

The proof is *knowledge sound* if the knowledge error ϵ is negligible.

Extraction is therefore only meaningful when $\mathcal{P}^*$ succeeds noticeably more often than the knowledge error: the guarantee is empty for $\epsilon^* \leq \epsilon$, and approaches certainty as ϵ^* grows away from ϵ .

The completeness property assures that the verifier $\mathcal{V}$ always accepts the execution of the protocol from an honest prover $\mathcal{P}$. Knowledge soundness states that a prover $\mathcal{P}^*$ that convinces $\mathcal{V}$ with probability ϵ^* must *know* a witness, in the sense that an algorithm $\mathcal{E}^{\mathcal{P}^*}$ (called *extractor*) with oracle access to $\mathcal{P}^*$ recovers one by querying it. Both the running time and the success probability of $\mathcal{E}$ degrade as ϵ^* approaches the knowledge error ϵ : extraction takes $\text{poly}(|x|)/\epsilon^*$ steps and succeeds with probability at least $1 - \epsilon/\epsilon^*$.

Next, we will introduce the definition of PCS and the properties that we need the scheme to satisfy. Broadly, we want a PCS to be an efficient algorithm that allows commitments to polynomials, such that they can later be evaluated and a proof of correct evaluation can be produced. These algorithms should satisfy: that honest commitments and evaluations are accepted by the verifier $\mathcal{V}$ (completeness); that no efficient adversary can find two distinct polynomials with the same commitment (binding); and that, if $\mathcal{V}$ accepts a commitment and an evaluation from a prover $\mathcal{P}^*$, then there is an extractor that can output a polynomial matching the commitment and the evaluation (extractability).

Definition 2.24 (Polynomial commitment scheme [KZG10]). A *polynomial commitment scheme* consists of PPT algorithms $\text{PC} = (\text{PC.Setup}, \text{PC.Commit}, \text{PC.Open}, \text{PC.Verify})$ where

- $(\text{ck}, \text{vk}) \leftarrow \text{PC.Setup}(1^\lambda, D)$: on input the security parameter 1^λ and a degree bound $D = \text{poly}(\lambda)$ outputs a commitment key ck and a verification key vk , supporting polynomials in $\mathbb{F}^{<D}[X]$,
- $\text{c} \leftarrow \text{PC.Commit}(\text{ck}, p(X))$: on input a commitment key ck and a polynomial $p(X) \in \mathbb{F}^{<D}[X]$ outputs a commitment c ,
- $\pi \leftarrow \text{PC.Open}(\text{ck}, \text{c}, x, y, p(X))$: on input a commitment key ck , a commitment c , two field elements $x, y \in \mathbb{F}$ and a polynomial $p(X) \in \mathbb{F}^{<D}[X]$ outputs a proof π ,
- $0/1 \leftarrow \text{PC.Verify}(\text{vk}, \text{c}, x, y, \pi)$: on input a verification key vk , a commitment c , two field elements $x, y \in \mathbb{F}$ and a proof π outputs a bit.

The scheme satisfies the following properties, for all $\lambda \in \mathbb{N}$ and $D = \text{poly}(\lambda)$.

- **Completeness.** For all $p(X) \in \mathbb{F}^{<D}[X]$ and $x \in \mathbb{F}$,

$$\Pr \left[1 \neq \text{PC.Verify}(\text{vk}, \text{c}, x, p(x), \pi) \mid \begin{array}{l} (\text{ck}, \text{vk}) \leftarrow \text{PC.Setup}(1^\lambda, D) \\ \text{c} \leftarrow \text{PC.Commit}(\text{ck}, p(X)) \\ \pi \leftarrow \text{PC.Open}(\text{ck}, \text{c}, x, p(x), p(X)) \end{array} \right] \leq \text{negl}(\lambda).$$

- **Binding.** For all PPT adversaries $\mathcal{A}$,

$$\Pr \left[\begin{array}{l} \text{c} = \text{PC.Commit}(\text{ck}, p_1(X)) \\ \text{c} = \text{PC.Commit}(\text{ck}, p_2(X)) \\ p_1(X) \neq p_2(X) \end{array} \mid \begin{array}{l} (\text{ck}, \text{vk}) \leftarrow \text{PC.Setup}(1^\lambda, D) \\ (\text{c}, p_1, p_2) \leftarrow \mathcal{A}(\text{ck}, \text{vk}) \end{array} \right] \leq \text{negl}(\lambda).$$

- **Extractability.** For all PPT adversaries $\mathcal{A}$ there exists an expected polynomial time extractor $\mathcal{E}$ such that

$$\Pr \left[\begin{array}{l} \text{PC.Verify}(\text{vk}, \text{c}, x, y, \pi) = 1 \wedge \\ (p(x) \neq y \vee \deg(p) \geq D) \\ \vee \text{c} \neq \text{PC.Commit}(\text{ck}, p(X)) \end{array} \middle| \begin{array}{l} (\text{ck}, \text{vk}) \leftarrow \text{PC.Setup}(1^\lambda, D) \\ (\text{c}, x, y, \pi) \leftarrow \mathcal{A}(\text{ck}, \text{vk}) \\ p(X) \leftarrow \mathcal{E}^{\mathcal{A}(\text{ck}, \text{vk})} \end{array} \right] \leq \text{negl}(\lambda).$$

In the context of this thesis, a PCS should also be *homomorphic* with respect to the sum of commitments $\text{c}_1 + \text{c}_2$ and the product of a commitment and a scalar $\alpha \cdot \text{c}$. This property allows us to batch many commitments using random linear combinations while preserving the random linear combination of the underlying polynomials.

Definition 2.25 (Homomorphic polynomial commitment, adapted from [PRZ26a, Sec. 3.1]). A polynomial commitment scheme is *homomorphic* if there exists an efficient algorithm **Combine** that on input a commitment key ck , two commitments c_1, c_2 and two field elements $\alpha, \beta \in \mathbb{F}$ outputs a fresh commitment c^* such that for all $\lambda \in \mathbb{N}$, $D = \text{poly}(\lambda)$ and all $\alpha, \beta \in \mathbb{F}$:

$$\Pr \left[\begin{array}{l} \text{c}^* \neq \text{PC.Commit}(\text{ck}, p^*(X)) \\ \text{c}_1 \leftarrow \text{PC.Commit}(\text{ck}, p_1(X)) \\ \text{c}_2 \leftarrow \text{PC.Commit}(\text{ck}, p_2(X)) \\ \text{c}^* \leftarrow \text{Combine}(\text{ck}, \text{c}_1, \alpha, \text{c}_2, \beta) \\ p^*(X) = \alpha p_1(X) + \beta p_2(X) \end{array} \right] \leq \text{negl}(\lambda).$$

We also denote $\text{Combine}(\text{ck}, \text{c}_1, \alpha, \text{c}_2, \beta)$ with $\alpha \text{c}_1 + \beta \text{c}_2$.

2.6 The KZG Polynomial Commitment Scheme

In the previous section, we introduced the definition of *Polynomial Commitment Schemes* and their desired properties. In this section, we will give an actual scheme that satisfies Definition 2.24 and Definition 2.25, the last two properties under computational assumptions. This scheme was introduced by [KZG10] (commonly known as KZG) and it is instantiated over a bilinear group where both commitments and evaluation proofs consist of a single group element each.

Definition 2.26 (KZG [KZG10]). Let $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2)$ be a bilinear group. The KZG polynomial commitment scheme is given by

- $\text{PC.Setup}(1^\lambda, D)$: sample $\tau \leftarrow \mathbb{Z}_p$ and output $\text{ck} = (g_1, g_1^\tau, \dots, g_1^{\tau^{D-1}})$ and $\text{vk} = (g_1, g_2, g_2^\tau)$,
- $\text{PC.Commit}(\text{ck}, p(X))$: output $\text{c} = g_1^{p(\tau)}$, computed from ck and the coefficients of p ,
- $\text{PC.Open}(\text{ck}, \text{c}, x, y, p(X))$: compute $q(X) = \frac{p(X) - y}{X - x}$ and output $\pi = g_1^{q(\tau)}$,
- $\text{PC.Verify}(\text{vk}, \text{c}, x, y, \pi)$: accept iff $e(\text{c} \cdot g_1^{-y}, g_2) = e(\pi, g_2^\tau \cdot g_2^{-x})$.

The commitment key holds the D powers $\tau^0, \dots, \tau^{D-1}$, so the committable polynomials are exactly $\mathbb{F}^{<D}[X]$.

The following theorem collects the guarantees of the KZG scheme. Completeness and the homomorphic property hold unconditionally; binding relies on the q -DLOG assumption, and extractability additionally requires the algebraic group model. A reader interested in the proofs is referred to [KZG10] for binding and to [FKL18] for the analysis in the algebraic group model.

Theorem 2.27 ([KZG10, FKL18]). *The scheme of Definition 2.26 is a complete and homomorphic polynomial commitment scheme in the sense of Definition 2.24 and Definition 2.25. It is binding under the q -DLOG assumption of Definition 2.18 with $q = D - 1$, and extractable in the algebraic group model (Definition 2.19) under the same assumption.*

2.7 Merkle Trees and the Fiat-Shamir Transform

The work of [PRZ26a] introduces a new transformation to efficiently derive a common random challenge from K input statements. This new transformation is a combination of two well-known concepts in cryptography: *Merkle trees* and the *Fiat-Shamir transform*. In this section, we will introduce these two concepts and the building blocks needed to define them.

The first two building blocks are: an information-theoretic primitive (*random oracles*) and a cryptographic primitive (*collision resistant hash functions*). They go hand-in-hand because it is common to argue about the security of a protocol in the random oracle model (or ROM) and, afterwards, instantiate the protocol using a hash function (similar to the use of a PIOP and a PCS in Section 2.5).

Definition 2.28 (Random oracle [PRZ26a, PRZ26b]). Let $\mathcal{U}(\lambda)$ denote the set of functions $\rho : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$. A *random oracle* is sampled uniformly from $\mathcal{U}(\lambda)$. A *relation relative to a random oracle*, denoted $\mathcal{R}^\rho$, is the set of relations $\{\mathcal{R}^\rho : \rho \in \bigcup_{\lambda \in \mathbb{N}} \mathcal{U}(\lambda)\}$.

Definition 2.29 (Collision resistance). A family of hash functions $\{H_k : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda\}_k$ is *collision resistant* if for every PPT adversary $\mathcal{A}$,

$$\Pr \left[x \neq x' \wedge H_k(x) = H_k(x') \mid k \leftarrow \{0, 1\}^\lambda, (x, x') \leftarrow \mathcal{A}(k) \right] \leq \text{negl}(\lambda).$$

Next, we introduce the Merkle tree construction, which is depicted in Figure 2.2. As discussed earlier in this section, a Merkle tree is defined in the ROM but, in practice, the queries to the random oracle are substituted by a hash function.

Definition 2.30 (Merkle tree, [PRZ26a, Sec. 3.2]). Let $K = 2^\kappa \in \mathbb{N}$ and $\boldsymbol{\rho} = (\rho_0, \dots, \rho_\kappa)$ where $\rho_j \leftarrow \mathcal{U}(\lambda)$ is the oracle used at level j , counted from the leaves: ρ_0 hashes each leaf and, for $j \in [\kappa]$, ρ_j maps the two level- $(j - 1)$ hashes of a node to its own, so that level κ is the root. A *Merkle tree* in the random oracle model consists of three algorithms

- $\mathbf{h}, \{\pi_{\mathbf{b}}\}_{\mathbf{b} \in \{0, 1\}^\kappa} \leftarrow \text{MT.Prove}^{\rho^{(\cdot)}}(\kappa, \{\mathbf{x}_{\mathbf{b}}\}_{\mathbf{b} \in \{0, 1\}^\kappa})$,
- $\mathbf{h}' \leftarrow \text{MT.Path}^{\rho^{(\cdot)}}(\kappa, \mathbf{b}, \mathbf{x}_{\mathbf{b}}, \{\mathbf{h}_i\}_{i \in \{1, \dots, \kappa\}})$,
- $0/1 \leftarrow \text{MT.Verify}^{\rho^{(\cdot)}}(\kappa, \mathbf{b}, \mathbf{x}_{\mathbf{b}}, \mathbf{h}, \pi_{\mathbf{b}})$,

where MT.Verify parses $\pi_{\mathbf{b}} := \{\mathbf{h}_i\}_{i \in \{1, \dots, \kappa\}}$ and outputs

$$\mathbf{h} \stackrel{?}{=} \text{MT.Path}^{\rho^{(\cdot)}}(\kappa, \mathbf{b}, \mathbf{x}_{\mathbf{b}}, \{\mathbf{h}_i\}_{i \in \{1, \dots, \kappa\}}).$$

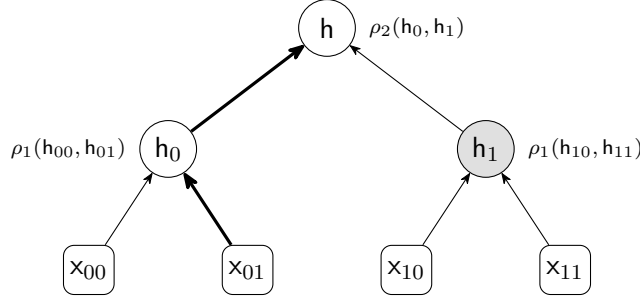


Figure 2.2: A Merkle tree over $K = 2^\kappa$ leaves, here $\kappa = 2$, writing $\mathbf{h}_b = \rho_0(\mathbf{x}_b)$ for the leaf hashes. Each internal node at level j is the image under ρ_j of its two children, and the root $\mathbf{h}$ binds all K leaves at once. The thick edges are the authentication path of leaf $\mathbf{x}_{01}$: given the shaded sibling $\mathbf{h}_1$ and the sibling leaf hash $\mathbf{h}_{00}$, a verifier holding $\mathbf{x}_{01}$ recomputes $\mathbf{h}$ with $\kappa = \log K$ oracle queries, which is what `MT.Path` and `MT.Verify` do in Definition 2.30.

The verification algorithm `MT.Verify` consists of recomputing the hashes of all the sibling nodes along the path going from the queried leaf to the root $\mathbf{h}$. This recomputation is efficient, because it only takes a logarithmic number of steps κ to verify, while the total size of the tree stays $2^{\kappa+1} - 1$.

Finally, we introduce the Fiat–Shamir transform from [FS87] (or **FS** in short). **FS** is used to transform an interactive protocol into a non-interactive one by use of a random oracle as in Definition 2.28. As in the case of Merkle trees, in practice the random oracles are instantiated using a hash function.

Definition 2.31 (Fiat–Shamir transform [FS87], as instantiated in [PRZ26a, Sec. 4.3]). Let $(\mathcal{P}, \mathcal{V})$ be an ℓ -round public-coin interactive protocol and let $\rho_0, \dots, \rho_\ell$ be random oracles. The *Fiat–Shamir transform* **FS** returns the non-interactive protocol in which the prover simulates an execution of $(\mathcal{P}, \mathcal{V})$ where the verifier’s messages $\gamma'_0, \dots, \gamma'_\ell$ are computed as

$$\gamma'_i \leftarrow \begin{cases} \rho_i(\mathbf{pp}, \mathbf{x}), & \text{if } i = 0, \\ \rho_i(\gamma'_{i-1}, \gamma_i), & \text{otherwise,} \end{cases}$$

where γ_i is the message sent by $\mathcal{P}$ on round i , and outputs the proof $\pi := (\gamma_1, \dots, \gamma_\ell)$. The non-interactive verifier recomputes $\gamma'_0, \dots, \gamma'_\ell$ from π and accepts iff $\mathcal{V}$ accepts the resulting transcript $\text{tr} := (\mathbf{pp}, \mathbf{x}, \gamma'_0, \gamma_1, \gamma'_1, \dots, \gamma_\ell, \gamma'_\ell)$.

2.8 Succinct Non-Interactive Arguments of Knowledge

Succinct non-interactive arguments of knowledge (SNARKs) are cryptographic protocols that allow a prover $\mathcal{P}$ to prove that a computation was correctly done to a verifier $\mathcal{V}$. Generally, a SNARK is any such protocol that is more efficient than the trivial proof of $\mathcal{P}$ sending the witness $\mathbf{w}$ to $\mathcal{V}$. Though this is a very general condition, many cryptographers restrict SNARKs to require the property of *succinctness* as in Definition 2.33, both for the proof size and the verifier time. In this thesis, we will work over SNARKs in the case where succinctness is required, and argue about the efficiency of the resulting protocol in contrast to running separate instances of the SNARK. See Section 3.4 for more details.

Definition 2.32 (Non-interactive argument, [PRZ26b, Def. 10]). Let $\mathcal{R}^\mathcal{U}$ be an indexed relation relative to a random oracle. A *(preprocessing) non-interactive argument* for $\mathcal{R}$ in the random oracle model is a tuple of polynomial time algorithms $\text{ARG} = (\mathcal{G}, \mathcal{I}, \mathcal{P}, \mathcal{V})$ that satisfy the following properties.

- **Completeness.** For every adversary $\mathcal{A}$,

$$\Pr \left[\begin{array}{c|c} (i, x, w) \in \mathcal{R}^\rho(\text{pp}) & \begin{array}{l} \rho \leftarrow \mathcal{U}(\lambda) \\ \text{pp} \leftarrow \mathcal{G}(1^\lambda) \\ (i, x, w) \leftarrow \mathcal{A}^\rho(\text{pp}) \\ (\text{pk}, \text{vk}) \leftarrow \mathcal{I}^\rho(\text{pp}, i) \\ \pi \leftarrow \mathcal{P}^\rho(\text{pk}, x, w) \end{array} \\ \Downarrow & \\ \mathcal{V}^\rho(\text{vk}, x, \pi) = 1 & \end{array} \right] = 1.$$

- **Knowledge soundness.** ARG is knowledge sound with respect to an auxiliary input distribution $\mathcal{D}$ if there exists a deterministic polynomial time extractor $\mathcal{E}$ such that for every (non-uniform) polynomial time adversary $\tilde{\mathcal{P}}$,

$$\Pr \left[\begin{array}{c|c} \mathcal{V}^\rho(\text{vk}, x, \pi) = 1 \wedge \\ (i, x, w) \notin \mathcal{R}^\rho(\text{pp}) & \begin{array}{l} \rho \leftarrow \mathcal{U}(\lambda), \text{pp} \leftarrow \mathcal{G}(1^\lambda) \\ \text{ai} \leftarrow \mathcal{D}(1^\lambda) \\ (i, x, \pi; \text{tr}) \leftarrow \tilde{\mathcal{P}}^\rho(\text{pp}, \text{ai}) \\ (\text{pk}, \text{vk}) \leftarrow \mathcal{I}^\rho(\text{pp}, i) \\ w \leftarrow \mathcal{E}(\text{pp}, i, x, \pi, \text{ai}, \text{tr}) \end{array} \end{array} \right] \leq \text{negl}(\lambda).$$

Here ai is the *auxiliary input*: the side information the malicious prover holds that does not come from the execution itself, such as proofs seen in earlier runs or public data, and $\mathcal{D}$ is the distribution it is drawn from. Knowledge soundness is stated relative to a fixed $\mathcal{D}$ rather than for all of them, because for arbitrary side information extraction is impossible: ai could encode an obfuscated program that produces accepting proofs from which no witness can be recovered [BCPR14, GW11]. Note also that $\mathcal{E}$ is given the same ai and the same transcript tr of oracle queries as $\tilde{\mathcal{P}}$; without them it would hold strictly less information than the prover it must extract from. Since $\mathcal{E}$ is deterministic and reads tr , extraction here is *straightline*: it inspects the prover's random oracle queries instead of rewinding it.

Definition 2.33 (Succinctness). A non-interactive argument ARG for an indexed relation $\mathcal{R}$ is *succinct* if for every $(i, x, w) \in \mathcal{R}$ the proof size satisfies $|\pi| = \text{poly}(\lambda, \log |w|)$ and the running time of $\mathcal{V}$ is $\text{poly}(\lambda, |x|, \log |w|)$. A succinct non-interactive argument that is knowledge sound in the sense of Definition 2.32 is a *SNARK*.

In Chapter 5, we will compile [PRZ26a]'s amortization protocol over *Marlin* [CHM⁺20], a SNARK that follows Definition 2.32 and Definition 2.33, though there exist several constructions that are not strictly succinct in this definition. Bulletproofs [BBB⁺18] achieve logarithmic proof size but a verifier that is linear in the circuit size, and STARKs [BBHR19] have proofs and verification polylogarithmic in the witness but with concretely large constants, so neither meets Definition 2.33 in the strict sense while both are succinct in the sense that matters in practice.

Definition 2.32 and Definition 2.33 constrain what a proof costs and what it guarantees, but say nothing about what it *reveals*. The property that captures the latter is *zero knowledge*, introduced in [GMR89]: anything the verifier learns from the proof, it could have produced on its own. It is maybe easier to understand zero knowledge as the existence

of an algorithm that can *simulate* the interaction with an honest prover from a verifier's perspective; in other words, zero knowledge is a property of the prover. This algorithm is known as the *Simulator* and it is used in the formal definition. In our definition, the simulator is additionally allowed to program the random oracle.

Definition 2.34 (Zero knowledge [CHM⁺20, Sec. 7]). A non-interactive argument $\text{ARG} = (\mathcal{G}, \mathcal{I}, \mathcal{P}, \mathcal{V})$ for an indexed relation $\mathcal{R}^{\mathcal{U}}$ has (*perfect*) *zero knowledge* if there exists a polynomial time simulator $\mathcal{S} = (\mathcal{S}.\text{Setup}, \mathcal{S}.\text{Prove})$ such that for every polynomial time adversary $\tilde{\mathcal{V}} = (\tilde{\mathcal{V}}_1, \tilde{\mathcal{V}}_2)$,

$$\begin{aligned} \Pr \left[\begin{array}{l} (i, x, w) \in \mathcal{R}^\rho(\text{pp}) \wedge \\ \tilde{\mathcal{V}}_2(\text{st}, \pi) = 1 \end{array} \middle| \begin{array}{l} \rho \leftarrow \mathcal{U}(\lambda), \text{pp} \leftarrow \mathcal{G}(1^\lambda) \\ (i, x, w, \text{st}) \leftarrow \tilde{\mathcal{V}}_1^\rho(\text{pp}) \\ (\text{pk}, \text{vk}) \leftarrow \mathcal{I}^\rho(\text{pp}, i) \\ \pi \leftarrow \mathcal{P}^\rho(\text{pk}, x, w) \end{array} \right] \\ = \Pr \left[\begin{array}{l} (i, x, w) \in \mathcal{R}^\rho(\text{pp}) \wedge \\ \tilde{\mathcal{V}}_2(\text{st}, \pi) = 1 \end{array} \middle| \begin{array}{l} (\text{pp}, \rho, \text{td}) \leftarrow \mathcal{S}.\text{Setup}(1^\lambda) \\ (i, x, w, \text{st}) \leftarrow \tilde{\mathcal{V}}_1^\rho(\text{pp}) \\ \pi \leftarrow \mathcal{S}.\text{Prove}(\text{td}, i, x) \end{array} \right], \end{aligned}$$

where td is a trapdoor and, in the second experiment, $\mathcal{S}$ answers the oracle queries of $\tilde{\mathcal{V}}$. The malicious verifier is split into two stages: $\tilde{\mathcal{V}}_1$ sees pp and chooses the index, statement and witness, and st is the *state* it hands to $\tilde{\mathcal{V}}_2$, which sees the resulting proof and outputs a bit. Replacing the equality by statistical or computational indistinguishability gives *statistical* and *computational* zero knowledge. A SNARK that is zero knowledge is a *zkSNARK*.

Note that the simulator is given the statement x but not the witness w , which is what makes the equality meaningful: a proof that can be produced without w cannot be leaking it. Carrying st across the two stages is what keeps the requirement strong, since $\tilde{\mathcal{V}}_1$ may store w in it and $\tilde{\mathcal{V}}_2$ may then use it to distinguish.

The following lemma introduces the *univariate sumcheck* protocol, a tool that will be central in the amortization of [PRZ26a]. This sumcheck protocol reduces checking $\sum_{h \in \mathbb{H}} f(h) = 0$ to checking the existence of univariate polynomials $g(X)$ and $q(X)$ satisfying the following equation:

Lemma 2.35 (Univariate sumcheck [BCR⁺19]). *Let $\mathbb{H}$ be a multiplicative subgroup of $\mathbb{F}^*$ of order n and let $f \in \mathbb{F}^{<d}[X]$ with $d \geq n$. Then $\sum_{h \in \mathbb{H}} f(h) = 0$ if and only if there exist polynomials $g \in \mathbb{F}^{<n-1}[X]$ and $q \in \mathbb{F}^{<d-n}[X]$ such that*

$$f(X) = X \cdot g(X) + (X^n - 1) \cdot q(X).$$

Lemma 2.35 is extremely useful when combined with a polynomial commitment scheme, since a verifier $\mathcal{V}$ can query the polynomials $f(X)$, $g(X)$ and $q(X)$ at a random point to check the validity of the claim $\sum_{h \in \mathbb{H}} f(h) = 0$ that a prover $\mathcal{P}$ sends. The summation expression might seem of little utility, but many algebraic relations over matrices and vectors can be expressed as such. It is common to find the multivariate version of the sumcheck protocol [LFKN92] at the core of SNARKs whose arithmetization is multilinear rather than univariate, such as Spartan [Set20] and HyperNova [KS24]. We only need the univariate variant, since the relations of Chapter 3 are all stated over a multiplicative subgroup $\mathbb{H}$.

2.9 Reductions of Knowledge

Reductions of knowledge are central to the work of [PRZ26a] (together with relations, see Definition 2.9) and, therefore, fundamental to this thesis as well. Reductions of knowledge can be interpreted as *reducing* the validity of a statement x (i.e. $x \in \mathcal{R}$) to the validity of another statement x' (i.e. $x' \in \mathcal{R}'$). By setting $\mathcal{R}' = \mathcal{R}$, this reduction allows *folding* many statements $x_1, \dots, x_K$ for a relation $\mathcal{R}$ to a single statement x for $\mathcal{R}$, while saving computational work for the prover. In Section 3.2, we present the reductions that compose the amortization protocol.

Reductions of knowledge are defined in the interactive case and, as usual, they have their non-interactive counterpart. The latter are known as *non-interactive reductions of knowledge* (NIRoK) and they are defined as reductions without interaction between the prover and verifier. We formalize these concepts in the following definitions:

Definition 2.36 (Reduction of knowledge, [PRZ26a, Sec. 4.1]; cf. [KP23]). Let $\mathcal{R}, \mathcal{R}'$ be ternary relations. A *reduction of knowledge* $\Pi : \mathcal{R} \rightarrow \mathcal{R}'$ is a tuple $(\text{Gen}, \mathcal{P}, \mathcal{V})$ where

- $\text{pp} \leftarrow \text{Gen}(1^\lambda, \text{pp}')$ is a PPT algorithm that on input the security parameter λ and parameters $\text{pp}' \in \{0, 1\}^{\text{poly}(\lambda)}$ outputs parameters pp ,
- $(w', x') \leftarrow \langle \mathcal{P}(\text{pp}, x, w, \text{aux}), \mathcal{V}(\text{pp}, x) \rangle$ is an interactive protocol between PPT interactive algorithms $\mathcal{P}, \mathcal{V}$, where $\mathcal{P}$ takes as input the parameters pp , a statement x , a witness w and auxiliary input aux and after the interaction outputs a fresh witness w' , and $\mathcal{V}$ takes as input the parameters pp and a statement x and after the interaction outputs a statement x' ,

satisfying the following properties.

- **Completeness.** For all $\lambda \in \mathbb{N}$, $\text{pp}' \in \{0, 1\}^{\text{poly}(\lambda)}$ and PPT algorithms $\mathcal{A}$,

$$\Pr \left[\begin{array}{c|c} (\text{pp}, x, w) \in \mathcal{R} & \text{pp} \leftarrow \text{Gen}(1^\lambda, \text{pp}') \\ \wedge & (x, w, \text{aux}) \leftarrow \mathcal{A}(\text{pp}) \\ (\text{pp}, x', w') \notin \mathcal{R}' & (w', x') \leftarrow \langle \mathcal{P}(\text{pp}, x, w, \text{aux}), \mathcal{V}(\text{pp}, x) \rangle \end{array} \right] = 0.$$

- **Knowledge soundness.** For all $\lambda \in \mathbb{N}$, $\text{pp}' \in \{0, 1\}^{\text{poly}(\lambda)}$ and all expected polynomial time adversaries $\mathcal{P}^*, \mathcal{A}$ there exists a polynomial time extractor $\mathcal{E}$ such that

$$\Pr \left[\begin{array}{c|c} (\text{pp}, x, w) \in \mathcal{R} & \begin{array}{l} \text{pp} \leftarrow \text{Gen}(1^\lambda, \text{pp}') \\ (x, \text{st}) \leftarrow \mathcal{A}(\text{pp}) \\ w \leftarrow \mathcal{E}(\text{pp}, x, \text{st}) \end{array} \end{array} \right] \geq \Pr \left[\begin{array}{c|c} (\text{pp}, x', w') \in \mathcal{R}' & \begin{array}{l} \text{pp} \leftarrow \text{Gen}(1^\lambda, \text{pp}') \\ (x, \text{st}) \leftarrow \mathcal{A}(\text{pp}) \\ (w', x') \leftarrow \langle \mathcal{P}^*(\text{pp}, x, \text{st}), \mathcal{V}(\text{pp}, x) \rangle \end{array} \end{array} \right] - \text{negl}(\lambda).$$

- **Public reducibility.** For all $\lambda \in \mathbb{N}$, $\text{pp}' \in \{0, 1\}^{\text{poly}(\lambda)}$ there exists a polynomially computable function ϕ such that for all polynomial time adversaries $\mathcal{A}, \mathcal{P}^*$,

$$\Pr \left[\begin{array}{c|c} \phi(\text{pp}, x, \text{tr}) \neq x' & \begin{array}{l} \text{pp} \leftarrow \text{Gen}(1^\lambda, \text{pp}') \\ (x, \text{st}) \leftarrow \mathcal{A}(\text{pp}) \\ (w', x') \leftarrow \langle \mathcal{P}^*(\text{pp}, x, \text{st}), \mathcal{V}(\text{pp}, x) \rangle \\ \text{tr} \leftarrow \text{tr}(\langle \mathcal{P}^*(\text{pp}, x, \text{st}), \mathcal{V}(\text{pp}, x) \rangle) \end{array} \end{array} \right] = 0,$$

where $\text{tr}(\cdot)$ denotes the transcript of the interaction of $\mathcal{P}^*, \mathcal{V}$ with their respective inputs.

- **Public coin.** If furthermore the protocol $\langle \mathcal{P}, \mathcal{V} \rangle$ is public coin, we say that the reduction of knowledge is also public coin.

Definition 2.37 (Non-interactive reduction of knowledge in the RO model, [PRZ26a, Sec. 4.2]). Let $\mathcal{R}, \mathcal{R}'$ be ternary relations. A *non-interactive reduction of knowledge* $\text{NIRoK} : \mathcal{R} \rightarrow \mathcal{R}'$ for the pair $\mathcal{R}, \mathcal{R}'$ is a tuple of PPT algorithms $(\text{Gen}, \text{Reduce}, \text{Verify})$ where

- $\text{pp} \leftarrow \text{Gen}(1^\lambda, \text{pp}')$: on input the security parameter λ and parameters $\text{pp}' \in \{0, 1\}^{\text{poly}(\lambda)}$ outputs parameters pp ,
- $\pi, \mathbf{x}', \mathbf{w}' \leftarrow \text{Reduce}^{\rho^{(\cdot)}}(\text{pp}, \mathbf{x}, \mathbf{w}, \text{aux})$: on input the parameters pp , a pair $(\mathbf{x}, \mathbf{w})$, auxiliary input aux and access to random oracles $\rho_1, \dots, \rho_\ell$, outputs a proof π and a new pair $(\mathbf{x}', \mathbf{w}')$,
- $0/1 \leftarrow \text{Verify}^{\rho^{(\cdot)}}(\text{pp}, \mathbf{x}, \mathbf{x}', \pi)$: on input the parameters pp , statements $\mathbf{x}, \mathbf{x}'$, a proof π and access to random oracles $\rho_1, \dots, \rho_\ell$ outputs a bit,

satisfying completeness and knowledge soundness defined analogously to Definition 2.36, where all probabilities are additionally taken over $\rho_1, \dots, \rho_\ell \leftarrow \mathcal{U}(\lambda)$.

It is possible to use a variant of the Fiat–Shamir transform FS to compile a $\text{NIRoK} = \text{FS}(\Pi)$ from an interactive reduction $\Pi : \mathcal{R} \rightarrow \mathcal{R}'$. The details of such transformation can be found in [PRZ26a, Def. 4.3.1]. For the sake of exposition, we will consider the simplified FS transformation introduced in Definition 2.31.

The key role of reductions in this thesis stems, also, from their composability, both sequential and in parallel. In other words, composing two reductions of knowledge yields another reduction. We present the definition of how to compose reductions and the theorem that certifies that the resulting composed protocol is indeed a reduction of knowledge:

Definition 2.38 (Composition of reductions of knowledge, [PRZ26a, Def. 4.4.2]). Let $\mathcal{R}_1, \mathcal{R}_2, \mathcal{R}_3, \mathcal{R}_4$ be ternary relations.

1. Let $\Pi_1 = (\text{Gen}, \mathcal{P}_1, \mathcal{V}_1) : \mathcal{R}_1 \rightarrow \mathcal{R}_2$ and $\Pi_2 = (\text{Gen}, \mathcal{P}_2, \mathcal{V}_2) : \mathcal{R}_2 \rightarrow \mathcal{R}_3$ be reductions of knowledge. The *sequential composition* of Π_1, Π_2 , denoted $\Pi_s = \Pi_2 \circ \Pi_1$, is the tuple $(\text{Gen}, \mathcal{P}, \mathcal{V})$ where

$$\langle \mathcal{P}(\text{pp}, \mathbf{x}_1, \mathbf{w}_1, \text{aux} = (\text{aux}_1, \text{aux}_2)), \mathcal{V}(\text{pp}, \mathbf{x}_1) \rangle$$

is the protocol in which $\mathcal{P}, \mathcal{V}$ interactively compute

$$\begin{aligned} (\mathbf{w}_2, \mathbf{x}_2) &\leftarrow \langle \mathcal{P}_1(\text{pp}, \mathbf{x}_1, \mathbf{w}_1, \text{aux}_1), \mathcal{V}_1(\text{pp}, \mathbf{x}_1) \rangle, \text{ and then} \\ (\mathbf{w}_3, \mathbf{x}_3) &\leftarrow \langle \mathcal{P}_2(\text{pp}, \mathbf{x}_2, \mathbf{w}_2, \text{aux}_2), \mathcal{V}_2(\text{pp}, \mathbf{x}_2) \rangle; \end{aligned}$$

$\mathcal{P}$ outputs $\mathbf{w}_3$ and $\mathcal{V}$ outputs $\mathbf{x}_3$.

2. Let $\Pi_1 = (\text{Gen}, \mathcal{P}_1, \mathcal{V}_1) : \mathcal{R}_1 \rightarrow \mathcal{R}_2$ and $\Pi_2 = (\text{Gen}, \mathcal{P}_2, \mathcal{V}_2) : \mathcal{R}_3 \rightarrow \mathcal{R}_4$ be reductions of knowledge. The *parallel composition* of Π_1, Π_2 , denoted $\Pi_p = \Pi_1 \times \Pi_2$, is the tuple $(\text{Gen}, \mathcal{P}, \mathcal{V})$ where

$$\langle \mathcal{P}(\text{pp}, (\mathbf{x}_1, \mathbf{x}_3), (\mathbf{w}_1, \mathbf{w}_3), \text{aux} = (\text{aux}_1, \text{aux}_3)), \mathcal{V}(\text{pp}, (\mathbf{x}_1, \mathbf{x}_3)) \rangle$$

is the protocol in which $\mathcal{P}, \mathcal{V}$ interactively compute

$$\begin{aligned} (w_2, x_2) &\leftarrow \langle \mathcal{P}_1(pp, x_1, w_1, aux_1), \mathcal{V}_1(pp, x_1) \rangle, \\ (w_4, x_4) &\leftarrow \langle \mathcal{P}_2(pp, x_3, w_3, aux_3), \mathcal{V}_2(pp, x_3) \rangle; \end{aligned}$$

$\mathcal{P}$ outputs (w_2, w_4) and $\mathcal{V}$ outputs (x_2, x_4) .

Theorem 2.39 ([PRZ26a, Thm. 4.4.1]; proven for the interactive case in [KP23]). *Let $\mathcal{R}_1, \mathcal{R}_2, \mathcal{R}_3, \mathcal{R}_4$ be ternary relations.*

1. *Let $\Pi_1 : \mathcal{R}_1 \rightarrow \mathcal{R}_2$ (resp. $\text{NIRoK}_1 : \mathcal{R}_1 \rightarrow \mathcal{R}_2$) and $\Pi_2 : \mathcal{R}_2 \rightarrow \mathcal{R}_3$ (resp. $\text{NIRoK}_2 : \mathcal{R}_2 \rightarrow \mathcal{R}_3$) be reductions of knowledge (resp. non-interactive reductions of knowledge). Then, the sequential composition $\Pi = \Pi_2 \circ \Pi_1$ (resp. $\text{NIRoK} = \text{NIRoK}_2 \circ \text{NIRoK}_1$) is a reduction of knowledge from $\mathcal{R}_1$ to $\mathcal{R}_3$.*
2. *Let $\Pi_1 : \mathcal{R}_1 \rightarrow \mathcal{R}_2$ (resp. $\text{NIRoK}_1 : \mathcal{R}_1 \rightarrow \mathcal{R}_2$) and $\Pi_2 : \mathcal{R}_3 \rightarrow \mathcal{R}_4$ (resp. $\text{NIRoK}_2 : \mathcal{R}_3 \rightarrow \mathcal{R}_4$) be reductions of knowledge (resp. non-interactive reductions of knowledge). Then, the parallel composition $\Pi = \Pi_1 \times \Pi_2$ (resp. $\text{NIRoK} = \text{NIRoK}_1 \times \text{NIRoK}_2$) is a reduction of knowledge from $\mathcal{R}_1 \times \mathcal{R}_3$ to $\mathcal{R}_2 \times \mathcal{R}_4$.*

2.10 Local Folding

The final section of this chapter is dedicated to *local folding* schemes derived from reductions of knowledge. The terminology *folding* refers to iteratively reducing the validity of many instances $x_1, \dots, x_K$ for a relation $\mathcal{R}$ to a single statement x for the same relation $\mathcal{R}$, where at each step the number of statements is halved. At every step, the protocol that allows the folding of two instances is a non-interactive reduction of knowledge (or NIRoK , see Section 2.9).

First of all, we will introduce a “partial” local folding scheme named *promise local folding scheme*. This type of scheme can be considered “incomplete” in the sense that the folding process postpones (or promises) proving some of the statements that arise during the protocol. These promise local folding schemes will work as a bridge between individual NIRoK s and a (plain) folding scheme. The latter can be viewed as promise local folding schemes with *trivial promises*, i.e. the relation that is promised is the truth relation, so no further work needs to be performed.

Definition 2.40 (Promise local folding scheme, [PRZ26a, Def. 6.1]). Let $K = 2^\kappa$ be a constant and $\mathcal{R}', \mathcal{R}$ be ternary relations. An $\mathcal{R}'$ -*Promise K -Local Folding Scheme* Λ for $\mathcal{R}$ is a tuple of PPT algorithms ($\text{Gen}, \text{LProve}, \text{LVerify}$) with access to random oracles $\rho_1, \dots, \rho_\ell$ such that

- $pp \leftarrow \text{Gen}(1^\lambda, pp')$: on input the security parameter λ and $pp' \in \{0, 1\}^{\text{poly}(\lambda)}$ outputs parameters pp ,
- $(\{\pi_b, x'_b, w'_b\}_{b \in \{0, 1\}^\kappa}, x, w) \leftarrow \Lambda.\text{LProve}^{\rho^{(\cdot)}}(pp, \{x_b, w_b\}_{b \in \{0, 1\}^\kappa})$: takes as input parameters pp and $K = 2^\kappa$ statement/witness pairs x_b, w_b (that supposedly satisfy $(pp, x_b, w_b) \in \mathcal{R}$) and outputs a fresh statement/witness pair (x, w) (that supposedly satisfies $(pp, x, w) \in \mathcal{R}$) and K statement/witness pairs $\{x'_b, w'_b\}_{b \in \{0, 1\}^\kappa}$ (each supposedly satisfying $(pp, x'_b, w'_b) \in \mathcal{R}'$) and corresponding proofs $\{\pi_b\}_{b \in \{0, 1\}^\kappa}$,

- $0/1 \leftarrow \Lambda.\text{LVerify}^{\rho^{(\cdot)}}(\text{pp}, \mathbf{b}, \mathbf{x}_{\mathbf{b}}, \mathbf{x}, \mathbf{x}'_{\mathbf{b}}, \pi_{\mathbf{b}})$: takes as input parameters pp , statements $\mathbf{x}_{\mathbf{b}}, \mathbf{x}$ and $\mathbf{x}'_{\mathbf{b}}$ and a proof $\pi_{\mathbf{b}}$, and outputs 0 or 1.

If the promise relation $\mathcal{R}'$ is the trivial relation that is always true, we refer to the scheme as a (*plain*) K -local folding scheme. The scheme satisfies the following properties.

- **Completeness.** For all $\mathbf{b} \in \{0, 1\}^{\kappa}$, $\lambda \in \mathbb{N}$, $\text{pp}' \in \{0, 1\}^{\text{poly}(\lambda)}$ and PPT algorithms $\mathcal{A}$,

$$\Pr \left[\begin{array}{c} \{q_{\mathbf{b}}\}_{\mathbf{b} \in \{0, 1\}^{\kappa}} \subseteq \mathcal{R} \\ \Downarrow \\ b = 1 \wedge (\text{pp}, \mathbf{x}, \mathbf{w}) \in \mathcal{R} \\ \wedge (\text{pp}, \mathbf{x}'_{\mathbf{b}}, \mathbf{w}'_{\mathbf{b}}) \in \mathcal{R}' \end{array} \left| \begin{array}{l} \rho_1, \dots, \rho_{\ell} \leftarrow \mathcal{U}(\lambda) \\ \text{pp} \leftarrow \text{Gen}(1^{\lambda}, \text{pp}') \\ \{\mathbf{x}_{\mathbf{b}}, \mathbf{w}_{\mathbf{b}}\}_{\mathbf{b}} \leftarrow \mathcal{A}^{\rho}(\text{pp}) \\ \{\pi_{\mathbf{b}}, \mathbf{x}'_{\mathbf{b}}, \mathbf{w}'_{\mathbf{b}}\}_{\mathbf{b}}, \mathbf{x}, \mathbf{w} \leftarrow \\ \quad \Lambda.\text{LProve}^{\rho}(\text{pp}, \{\mathbf{x}_{\mathbf{b}}, \mathbf{w}_{\mathbf{b}}\}_{\mathbf{b}}) \\ q_{\mathbf{b}} = (\text{pp}, \mathbf{x}_{\mathbf{b}}, \mathbf{w}_{\mathbf{b}})_{\mathbf{b}} \\ b \leftarrow \Lambda.\text{LVerify}^{\rho}(\text{pp}, \mathbf{b}, \mathbf{x}_{\mathbf{b}}, \mathbf{x}, \mathbf{x}'_{\mathbf{b}}, \pi_{\mathbf{b}}) \end{array} \right. \right] \geq 1 - \text{negl}(\lambda).$$

- **Knowledge soundness.** For all $\mathbf{b} \in \{0, 1\}^{\kappa}$, $\lambda \in \mathbb{N}$, $\text{pp}' \in \{0, 1\}^{\text{poly}(\lambda)}$ and expected polynomial time adversaries $\mathcal{A}_1, \mathcal{A}_2$, there exists an expected polynomial time extractor $\mathcal{E}$ such that

$$\Pr \left[(\text{pp}, \mathbf{x}_{\mathbf{b}}, \mathbf{w}_{\mathbf{b}}) \in \mathcal{R} \left| \begin{array}{l} \rho_1, \dots, \rho_{\ell} \leftarrow \mathcal{U}(\lambda) \\ \text{pp} \leftarrow \text{Gen}(1^{\lambda}, \text{pp}') \\ \mathbf{x}_{\mathbf{b}}, \text{st} \leftarrow \mathcal{A}_1^{\rho}(\text{pp}) \\ \mathbf{w}_{\mathbf{b}} \leftarrow \mathcal{E}^{\rho}(\text{pp}, \mathbf{x}_{\mathbf{b}}, \text{st}) \end{array} \right. \right] \geq$$

$$\Pr \left[\begin{array}{c} (\text{pp}, \mathbf{x}, \mathbf{w}) \in \mathcal{R} \wedge \\ (\text{pp}, \mathbf{x}'_{\mathbf{b}}, \mathbf{w}'_{\mathbf{b}}) \in \mathcal{R}' \wedge \\ \Lambda.\text{LVerify}^{\rho}(\text{pp}, \mathbf{b}, \mathbf{x}_{\mathbf{b}}, \mathbf{x}, \mathbf{x}'_{\mathbf{b}}, \pi_{\mathbf{b}}) = 1 \end{array} \left| \begin{array}{l} \rho_1, \dots, \rho_{\ell} \leftarrow \mathcal{U}(\lambda) \\ \text{pp} \leftarrow \text{Gen}(1^{\lambda}, \text{pp}') \\ \mathbf{x}_{\mathbf{b}}, \text{st} \leftarrow \mathcal{A}_1^{\rho}(\text{pp}) \\ \pi_{\mathbf{b}}, \mathbf{x}, \mathbf{w}, \mathbf{x}'_{\mathbf{b}}, \mathbf{w}'_{\mathbf{b}} \leftarrow \mathcal{A}_2^{\rho}(\text{pp}, \text{st}) \end{array} \right. \right] - \text{negl}(\lambda).$$

- **Efficiency.** For all $\kappa \in \mathbb{N}$, $\mathbf{b} \in \{0, 1\}^{\kappa}$, $\lambda \in \mathbb{N}$ it should hold that $|\pi_{\mathbf{b}}| + |\mathbf{x}'_{\mathbf{b}}| = o(2^{\kappa} \cdot |\mathbf{x}|)$, namely, the inputs of the LVerify algorithm should be asymptotically smaller than the total size of the aggregated statements.

For a detailed instantiation, see Section 3.3 where we will compile the promise and plain local folding schemes that this thesis implements.

Chapter 3

Amortization Protocol

In this chapter, we will present the main protocol as instantiated in the implementation. In Section 3.1 and Section 3.2, we will go through the relations and reductions of knowledge that are the building blocks of the protocol; in Section 3.3 we will present the amortization in terms of a *local folding scheme* and the theorems that assure its completeness and soundness; and in Section 3.4 we will describe the protocol step by step and argue about its computational cost.

In a nutshell, the protocol can be viewed as a set of reductions between relations where, in each step, we try to reduce (or *fold*) the number of original statements. This process of many-to-one outputs residual statements whose validity needs to be checked, so the protocol *postpones* proving the correctness of the folding process by considering other relations and reductions. These postponed statements are called *promises* and the process of reducing the original statements to a final folded statement, while promising part of the proof, is called a *promise folding scheme*.

Naturally, these promises (which can be modeled as statements for other relations) are proven in a similar fashion. The protocol uses other reductions to reduce proving many promised statements to proving a single polynomial opening. This last opening can be proven using a homomorphic polynomial commitment scheme such as KZG (see Definition 2.24 and Definition 2.26). The protocol uses different techniques to amortize all these processes such that the resulting scheme provides computational gains to the Prover (as opposed to proving all the original statements separately).

3.1 Relations

In this section, we will introduce all the relations that are involved in the protocol and that will appear in the following sections as the endpoints of reductions of knowledge.

As discussed in Chapter 1, many SNARKs need to provide a proof for bivariate polynomial evaluation, which is independent of the witness. Statements for this relation will be considered as inputs to the whole amortization protocol:

Definition 3.1 (Bivariate polynomial evaluation relation, [PRZ26a, Sec. 2.5]). For a fixed sparse bivariate polynomial P determined by the circuit description,

$$\mathcal{R}_P = \{(\alpha, \beta, y) \mid P(\alpha, \beta) = y\}.$$

Formally, $\mathcal{R}_P$ is a ternary relation whose elements are $(\perp, (\alpha, \beta, y), \perp)$: the polynomial P is fixed by the circuit, so no public parameters are needed, and the claim is checked directly

from the statement, so there is no witness. Throughout we use (α, β, y) as shorthand for that triple.

As noted in [PRZ26a], there is a natural bijection between matrices and bivariate polynomials, so the previous relation is equivalent to a vector-matrix-vector multiplication $\lambda(\alpha)^\top \mathbf{M} \lambda(\beta) = y$, where α and β are random challenges and $\mathbf{M}$ and y are public parameters. More generally, we can work with vectors $\mathbf{u}, \mathbf{v}$ and identify $\mathbf{u} = \lambda(\alpha)$ and $\mathbf{v} = \lambda(\beta)$.

To take advantage of the succinctness and efficiency given by polynomial commitment schemes, we will express $\mathbf{u}, \mathbf{v}$ in the Lagrange basis as $\mathbf{u}^\top \lambda(X)$ and $\mathbf{v}^\top \lambda(X)$. Together with their commitments, we can consider the following relation:

Definition 3.2 (Relaxed matrix evaluation relation, [PRZ26a, Sec. 2.5]).

$$\widetilde{\mathcal{R}}_{\mathbf{M}} = \left\{ \begin{array}{c|c} \text{pp} = (\text{ck}, \text{vk}) & \text{c} = \text{PC.Commit}(\text{ck}, \mathbf{u}^\top \lambda(X)) \wedge \\ (\text{c}, \text{d}, y) & \text{d} = \text{PC.Commit}(\text{ck}, \mathbf{v}^\top \lambda(X)) \wedge \\ (\mathbf{u}, \mathbf{v}) & y = \mathbf{u}^\top \mathbf{M} \mathbf{v} \end{array} \right\}.$$

A key observation in [PRZ26a] is that, using Proposition 2.4, the polynomials $\mathbf{u}^\top \lambda(X)$ and $\mathbf{v}^\top \lambda(X)$ can be expressed as the quotient of two sparse polynomials:

$$\begin{aligned} \mathbf{u}^\top \lambda(X) &= \frac{N_u(X)}{nT_u(X)}, \\ \mathbf{v}^\top \lambda(X) &= \frac{N_v(X)}{nT_v(X)}, \end{aligned}$$

where N_u, N_v have 3 coefficients and degree n and T_u, T_v have 2 coefficient and degree 1.

Considering this observation, we can define an equivalent relation to Definition 3.2 with implicit commitments to $\mathbf{u}$ and $\mathbf{v}$:

Definition 3.3 (Matrix evaluation relation with sparse implicit commitments, [PRZ26a, Sec. 8]). For a fixed matrix $\mathbf{M} \in \mathbb{F}^{n \times n}$,

$$\mathcal{R}_{\mathbf{M}} = \left\{ \begin{array}{c|c} (\text{ck}, \text{vk}) & \mathbf{d} = (d_{N,u}, d_{T,u}, d_{N,v}, d_{T,v}) \wedge \\ (\text{c}_N, \text{c}_T, \text{d}_N, \text{d}_T, \mathbf{d}, y) & \text{c}_N = \text{PC.Commit}(\text{ck}, N_u(X)) \wedge N_u \in \mathbb{F}^{<d_{N,u}}[X] \wedge \\ \begin{pmatrix} N_u(X), T_u(X), \mathbf{u} \\ N_v(X), T_v(X), \mathbf{v} \end{pmatrix} & \text{d}_N = \text{PC.Commit}(\text{ck}, N_v(X)) \wedge N_v \in \mathbb{F}^{<d_{N,v}}[X] \wedge \\ & \text{c}_T = \text{PC.Commit}(\text{ck}, T_u(X)) \wedge T_u \in \mathbb{F}^{<d_{T,u}}[X] \wedge \\ & \text{d}_T = \text{PC.Commit}(\text{ck}, T_v(X)) \wedge T_v \in \mathbb{F}^{<d_{T,v}}[X] \wedge \\ & N_u(X)/nT_u(X) = \mathbf{u}^\top \lambda(X) \wedge \\ & N_v(X)/nT_v(X) = \mathbf{v}^\top \lambda(X) \wedge \\ & y = \mathbf{u}^\top \mathbf{M} \mathbf{v} \end{array} \right\}.$$

This last relation will be the central relation to fold in the amortization protocol. In the next section, we will present a promise folding scheme that will output a final statement for $\mathcal{R}_{\mathbf{M}}$. As the word *promise* suggests, there will be statements for other relations that need to be proven after the main folding. These statements are *polynomial constraints* over the committed polynomials in $\mathcal{R}_{\mathbf{M}}$:

Definition 3.4 (Polynomial commitment constraint relation, [PRZ26a, Sec. 7]). Let PC be a polynomial commitment scheme and assume the public parameters include the commitment and verification keys ck, vk for a degree bound D . The *polynomial commitment constraint relation* is

$$\mathcal{R}_{\text{PCC}} = \left\{ \begin{array}{c} \text{pp} \\ (\mathbf{c}, \mathbf{d}, \mathbf{Q}) \\ \mathbf{p}(X) \end{array} \left| \begin{array}{l} \{\mathbf{c}_i = \text{PC.Commit}(\text{ck}, p_i(X))\}_{i=1}^n \\ \{p_i(X) \in \mathbb{F}^{< d_i}[X], d_i \leq D\}_{i=1}^n \\ \{Q_j \in \mathbb{F}[X, Y_1, \dots, Y_n] \text{ and } Q_j(X, \mathbf{p}(X)) = \mathbf{0}\}_{j=1}^\ell \end{array} \right. \right\}.$$

We denote with $\mathcal{R}_{\text{PCC-D}}$ the same relation but with no degree constraints, namely

$$\mathcal{R}_{\text{PCC-D}} = \left\{ \begin{array}{c} \text{pp} \\ (\mathbf{c}, \mathbf{Q}) \\ \mathbf{p}(X) \end{array} \left| \begin{array}{l} \{\mathbf{c}_i = \text{PC.Commit}(\text{ck}, p_i(X))\}_{i=1}^n \\ \{p_i(X) \in \mathbb{F}^{< D}[X]\}_{i=1}^n \\ \{Q_j \in \mathbb{F}[X, Y_1, \dots, Y_n] \text{ and } Q_j(X, \mathbf{p}(X)) = \mathbf{0}\}_{j=1}^\ell \end{array} \right. \right\}.$$

The differentiation between $\mathcal{R}_{\text{PCC}}$ and $\mathcal{R}_{\text{PCC-D}}$ is needed since polynomial commitment schemes such as plain KZG can only prove openings of committed polynomials of degree less than a prescribed degree D . In the next section, we will present a reduction to obtain a statement for $\mathcal{R}_{\text{PCC-D}}$ from one for $\mathcal{R}_{\text{PCC}}$, where each degree bound $d_i \leq D$ is traded for an extra committed polynomial and an extra polynomial constraint tying it to the original.

Finally, the last relation that appears in the protocol encodes openings for committed polynomials at a point x . These statements can be batched together when combined with homomorphic polynomial commitment schemes, as discussed in the next section.

Definition 3.5 (Polynomial commitment opening relation, [PRZ26a, Sec. 7]). The relation capturing the statement “ $\mathbf{c}$ is a commitment to $p(X)$ of degree less than D and $p(x) = y$ ” is

$$\mathcal{R}_{\text{PCO}} = \left\{ \text{pp}, (\mathbf{c}, x, y), p(X) \left| \begin{array}{l} \mathbf{c} = \text{PC.Commit}(\text{ck}, p(X)) \\ p(X) \in \mathbb{F}^{< D}[X] \\ p(x) = y \end{array} \right. \right\},$$

and $\mathcal{R}_{\text{PCO}(x)} = \left\{ \text{pp}, (\mathbf{c}, y), p(X) \mid (\text{pp}, (\mathbf{c}, x, y), p(X)) \in \mathcal{R}_{\text{PCO}} \right\}$ is the same relation for a hard-coded point x .

3.2 Reductions

All the reductions below are public coin, and all of them share the generator $\text{Gen}(1^\lambda, D)$ that outputs $\text{ck}, \text{vk} \leftarrow \text{PC.Setup}(1^\lambda, D)$. In every case the prover takes as input $(\text{pp}, \mathbf{x}, \mathbf{w})$ with auxiliary input $\text{aux} = \perp$ and the verifier takes as input $(\text{pp}, \mathbf{x})$.

The entry point of the protocol is not a reduction: a client holding an $\mathcal{R}_{\text{P}}$ claim (α, β, y) obtains its $\mathcal{R}_{\text{M}}$ leaf by setting $\mathbf{u} := \boldsymbol{\lambda}(\alpha)$ and $\mathbf{v} := \boldsymbol{\lambda}(\beta)$ and taking the sparse representations of Proposition 2.4,

$$N_u(X) = (X^n - 1)\alpha - (\alpha^n - 1)X, \quad T_u(X) = X - \alpha,$$

and analogously N_v, T_v from β . Since these have at most three non-zero coefficients and depend only on the public challenges, the verifier recomputes and checks the leaf commitments alone with $\mathcal{O}(1)$ group operations; that the encoding is the correct one is

certified by an additional $\mathcal{R}_{\text{PCC}}$ promise, discharged together with the promises produced by folding.

The following reduction defines the folding step of the protocol: it takes two $\mathcal{R}_{\text{M}}$ statements and returns a single folded one, together with the polynomial constraints that certify that the fold was performed correctly. These constraints are the *promises* that will be discharged and proven by following reductions in this section.

Definition 3.6 ($\Pi_{\text{M}} : \mathcal{R}_{\text{M}}^2 \rightarrow \mathcal{R}_{\text{M}} \times \mathcal{R}_{\text{PCC}}$, [PRZ26a, Fig. 13]). Both parties parse $\mathbf{x}_b := (\mathbf{c}_{N,b}, \mathbf{c}_{T,b}, \mathbf{d}_{N,b}, \mathbf{d}_{T,b}, \mathbf{d}_b, y_b)$ for $b \in \{0, 1\}$, and $\mathcal{P}$ parses $\mathbf{w}_b := (N_{u,b}(X), T_{u,b}(X), \mathbf{u}_b, N_{v,b}(X), T_{v,b}(X), \mathbf{v}_b)$.

1. $\mathcal{P}$ computes the cross term $z := \mathbf{u}_0^\top \mathbf{M} \mathbf{v}_1 + \mathbf{u}_1^\top \mathbf{M} \mathbf{v}_0$ and sends z .
2. $\mathcal{V}$ samples $\gamma \leftarrow \mathbb{F}$ and sends it.
3. $\mathcal{P}$ computes $\mathbf{u}^* := \mathbf{u}_0 + \gamma \mathbf{u}_1$, $\mathbf{v}^* := \mathbf{v}_0 + \gamma \mathbf{v}_1$ and

$$\begin{aligned} N_u^*(X) &= N_{u,0}(X)T_{u,1}(X) + \gamma N_{u,1}(X)T_{u,0}(X), & T_u^*(X) &= T_{u,0}(X)T_{u,1}(X), \\ N_v^*(X) &= N_{v,0}(X)T_{v,1}(X) + \gamma N_{v,1}(X)T_{v,0}(X), & T_v^*(X) &= T_{v,0}(X)T_{v,1}(X), \end{aligned}$$

together with the corresponding commitments $\mathbf{c}_N^*, \mathbf{c}_T^*, \mathbf{d}_N^*, \mathbf{d}_T^*$, which it sends.

4. Both parties define the constraints $\mathbf{Q}$ over the formal variables $Y_1, \dots, Y_{12}$, which stand for the twelve committed polynomials of $\mathbf{w}_{\text{PCC}}$ in the order fixed in step (6), namely

$$\begin{aligned} Y_1 &= N_u^*, & Y_2 &= T_u^*, & Y_3 &= N_v^*, & Y_4 &= T_v^*, \\ Y_5 &= N_{u,0}, & Y_6 &= T_{u,0}, & Y_7 &= N_{v,0}, & Y_8 &= T_{v,0}, \\ Y_9 &= N_{u,1}, & Y_{10} &= T_{u,1}, & Y_{11} &= N_{v,1}, & Y_{12} &= T_{v,1}, \end{aligned}$$

so that

$$\begin{aligned} Q_1 &= Y_1 - Y_5 Y_{10} - \gamma Y_9 Y_6, & Q_3 &= Y_2 - Y_6 Y_{10}, \\ Q_2 &= Y_3 - Y_7 Y_{12} - \gamma Y_{11} Y_8, & Q_4 &= Y_4 - Y_8 Y_{12}, \end{aligned}$$

and the degrees $\mathbf{d}^* = (d_{N^*,u}, d_{T^*,u}, d_{N^*,v}, d_{T^*,v})$ given by $d_{N^*,u} = \max_b \{d_{N,u,b} + d_{T,u,1-b}\}$, $d_{N^*,v} = \max_b \{d_{N,v,b} + d_{T,v,1-b}\}$, $d_{T^*,u} = d_{T,u,0} + d_{T,u,1}$ and $d_{T^*,v} = d_{T,v,0} + d_{T,v,1}$.

5. Both parties compute $y^* = y_0 + \gamma z + \gamma^2 y_1$ and set $\mathbf{x}^* := (\mathbf{c}_N^*, \mathbf{c}_T^*, \mathbf{d}_N^*, \mathbf{d}_T^*, \mathbf{d}^*, y^*)$ and

$$\mathbf{x}_{\text{PCC}} := (\mathbf{c}_N^*, \mathbf{c}_T^*, \mathbf{d}_N^*, \mathbf{d}_T^*, \mathbf{c}_{N,0}, \mathbf{c}_{T,0}, \mathbf{d}_{N,0}, \mathbf{d}_{T,0}, \mathbf{c}_{N,1}, \mathbf{c}_{T,1}, \mathbf{d}_{N,1}, \mathbf{d}_{T,1}, (\mathbf{d}^*, \mathbf{d}_0, \mathbf{d}_1), \mathbf{Q}).$$

6. $\mathcal{P}$ outputs $\mathbf{w}^* := (N_u^*(X), T_u^*(X), N_v^*(X), T_v^*(X), \mathbf{u}^*, \mathbf{v}^*)$ and

$$\mathbf{w}_{\text{PCC}} := \begin{pmatrix} N_u^*(X), T_u^*(X), N_v^*(X), T_v^*(X), \\ N_{u,0}(X), T_{u,0}(X), N_{v,0}(X), T_{v,0}(X), \\ N_{u,1}(X), T_{u,1}(X), N_{v,1}(X), T_{v,1}(X) \end{pmatrix},$$

while $\mathcal{V}$ outputs $\mathbf{x}^*$ and $\mathbf{x}_{\text{PCC}}$.

The promises collected during the folding are $\mathcal{R}_{\text{PCC}}$ statements, which carry degree bounds. The next reduction removes them by having the prover commit to shifted copies of each polynomial, so that a single global degree bound D suffices.

Definition 3.7 ($\Pi_D : \mathcal{R}_{\text{PCC}} \rightarrow \mathcal{R}_{\text{PCC-D}}$, [PRZ26a, Fig. 8]).

1. $\mathcal{P}$ parses $\mathbf{w} := \mathbf{p}(X)$, computes the scaled polynomials $\{p'_i(X) := X^{D-d_i}p_i(X)\}_{i=1}^n$ together with the corresponding commitments $\mathbf{c}'$, and sends $\mathbf{c}'$.
2. $\mathcal{P}$ outputs $\mathbf{w}' := \tilde{\mathbf{p}}(X) := (\mathbf{p}(X), \mathbf{p}'(X))$.
3. $\mathcal{V}$ computes $\tilde{\mathbf{c}} := (\mathbf{c}, \mathbf{c}')$ and $\tilde{\mathbf{Q}} := (\mathbf{Q}, \mathbf{Q}')$, where $Q'_i(X, \tilde{\mathbf{Y}}) := Y'_i - X^{D-d_i}Y_i$, and outputs $\mathbf{x}' := (\tilde{\mathbf{c}}, \tilde{\mathbf{Q}})$.

Once the degree constraints are uniform, the polynomial constraints are checked at a single random point and batched into one committed polynomial, turning an $\mathcal{R}_{\text{PCC-D}}$ statement into a single opening claim.

Definition 3.8 ($\Pi_{\text{PC}} : \mathcal{R}_{\text{PCC-D}} \rightarrow \mathcal{R}_{\text{PCO}(x)}$, [PRZ26a, Fig. 9]). Both parties parse $\mathbf{x} := (\mathbf{c}, \mathbf{Q})$.

1. $\mathcal{V}$ samples $x \leftarrow \mathbb{F}$ and sends it.
2. $\mathcal{P}$ computes $\mathbf{y} := \mathbf{p}(x)$ and sends $\mathbf{y}$.
3. $\mathcal{V}$ samples $r \leftarrow \mathbb{F}$ and sends it.
4. $\mathcal{P}$ computes $p(X) := \sum_{i=1}^n r^{i-1}p_i(X)$ and outputs $\mathbf{w}' := p(X)$.
5. $\mathcal{V}$ computes $\mathbf{c} := \sum_{i=1}^n r^{i-1}\mathbf{c}_i$ and $y := \sum_{i=1}^n r^{i-1}y_i$, asserts $\mathbf{Q}(x, \mathbf{y}) = 0$, and outputs $\mathbf{x}' := (\mathbf{c}, y)$.

Π_{PC} is defined so that a *single* verifier samples an evaluation point x before computing the batched polynomial evaluation $y = p(x)$. This is a problem in the setting where verifiers are independent and each must choose a random evaluation point x_i . In this independent verifier setup, the prover would need to work with $\mathcal{R}_{\text{PCO}}$ statements over K different evaluation points x_i and do work linear in K . Thus, batching the K opening claims requires them to share one evaluation point.

The FS_{MT} compiler removes the obstacle: it is a variant of the Fiat–Shamir transform of Definition 2.31 in which the first verifier message is not derived from a single statement, but from a Merkle commitment to all K statements at once.

Definition 3.9 (FS_{MT} compiler, [PRZ26a, Sec. 2.2]). Let $K = 2^\kappa$, let $\Pi : \mathcal{R} \rightarrow \mathcal{R}'$ be a public coin reduction of knowledge and let $\rho_0, \dots, \rho_{\kappa+1}$ be random oracles. The transformation $\text{FS}_{\text{MT}}(\Pi, \kappa)$ returns the following pair of algorithms.

- The non-interactive prover $\{\pi_i, \mathbf{x}'_i, \mathbf{w}'_i\}_{i=1}^K \leftarrow \text{Prove}(\{\mathbf{x}_i, \mathbf{w}_i\}_{i=1}^K)$, on input the K statement/witness pairs, first computes a Merkle tree of all the statements; let $\mathbf{h}$ be its root. It then performs K executions of $\text{FS}(\Pi)$, in all of which it uses $\rho_{\kappa+1}(\mathbf{h})$ as the first message. In the i -th proof it also adds the sibling nodes $\gamma_1, \dots, \gamma_\kappa$ of the i -th Merkle path leading to $\mathbf{h}$.
- The non-interactive verifier $0/1 \leftarrow \text{Verify}(i, \mathbf{x}_i, \pi_i)$, on a proof $\pi_i = (\gamma_1, \dots, \gamma_\kappa, \pi_\Pi)$, recomputes $\mathbf{h}$ from $\gamma'_0 = \rho_0(\mathbf{x}_i)$ and $\gamma'_j = \rho_j(\gamma'_{j-1}, \gamma_j)$ or $\gamma'_j = \rho_j(\gamma_j, \gamma'_{j-1})$, according to whether the j -th sibling is a left or a right one. It then verifies π_Π as usual, except that it uses $\rho_{\kappa+1}(\mathbf{h})$ as the message of the first round of Π .

For $i \in \{1, \dots, K\}$ we write Prove_i for the algorithm that runs Prove and returns only its i -th proof π_i , and $\text{Verify}_i(\mathbf{x}_i, \pi_i) := \text{Verify}(i, \mathbf{x}_i, \pi_i)$ for the verifier of that proof. The pair $(\text{Prove}_i, \text{Verify}_i)$ is what the i -th verifier and the prover run between them.

Because $\mathbf{h}$ binds all K statements, every verifier derives the same first message: the shared evaluation point x . It does so by recomputing it from its own statement with $\log K$ oracle queries, given the sibling nodes shipped in its proof. Soundness is preserved because, from the point of view of verifier i , that point is still defined only after $\mathbf{x}_i$, which lies in the hash chain leading to $\mathbf{h}$. Analysing this transform directly would require reasoning about several transcripts sharing random-oracle queries. Instead, [PRZ26a] observes that reconstructing the authentication path can itself be read as $\kappa + 1$ public-coin rounds preceding Π , on which no verification check is imposed and whose only role is to encode the path into the structure of the transcript. Under that reading $\text{FS}_{\text{MT}}(\Pi, \kappa)$ is an ordinary Fiat–Shamir transform of a reduction of knowledge, so the standard analysis of [PRZ26a, Thm. 4.3.1] applies.

Theorem 3.10 ([PRZ26a, Thm. 2.2.1], informal). *Let $K = 2^\kappa$ be a constant, $i \in \{1, \dots, K\}$ and $\Pi : \mathcal{R} \rightarrow \mathcal{R}'$ be a reduction of knowledge. Consider the algorithms $\text{Prove}_i, \text{Verify}_i$ derived by applying the transformation $\text{FS}_{\text{MT}}(\Pi, \kappa)$ and using the i -th proof. Then $(\text{Prove}_i, \text{Verify}_i)$ is a non-interactive reduction of knowledge $\text{NIRoK}_i : \mathcal{R} \rightarrow \mathcal{R}'$ in the random oracle model.*

Knowledge soundness then reduces to the standard Fiat–Shamir argument. For every i and every auxiliary input, the verification of $\text{FS}_{\text{MT}}(\Pi)$ and that of $\text{FS}(\Pi)$ differ only in that the two inputs of the round oracles are swapped at the levels where the i -th leaf is a right child.

After applying FS_{MT} to Π_{PC} , all opening claims at the same point are homomorphically combined into a single one. This is the reduction that makes the amortization pay off: it is only applicable because the multi-verifier Fiat–Shamir transform forces all executions to share the evaluation point x .

Definition 3.11 ($\Pi_{\text{PCO}} : \mathcal{R}_{\text{PCO}(x)}^\ell \rightarrow \mathcal{R}_{\text{PCO}(x)}$, [PRZ26a, Fig. 10]). Both parties parse $\mathbf{x} := (\mathbf{c}, \mathbf{y})$ and $\mathcal{P}$ parses $\mathbf{w} := \mathbf{p}(X)$.

1. $\mathcal{V}$ samples $r \leftarrow \mathbb{F}$ and sends it.
2. $\mathcal{P}$ computes $p'(X) := \sum_{i=1}^\ell r^{i-1} p_i(X)$ and outputs $\mathbf{w}' := p'(X)$.
3. $\mathcal{V}$ computes $\mathbf{c}' := \sum_{i=1}^\ell r^{i-1} \mathbf{c}_i$ and $\mathbf{y}' := \sum_{i=1}^\ell r^{i-1} \mathbf{y}_i$, and outputs $\mathbf{x}' := (\mathbf{c}', \mathbf{y}')$.

This reduction closes the chain of promised statements that started with Π_{M} : a single $\mathcal{R}_{\text{PCO}}$ statement is all that remains of them, and it is proven directly by opening the committed polynomial with the instantiated homomorphic polynomial commitment scheme, KZG in our case (Definition 2.26).

The final reduction returns to the folded $\mathcal{R}_{\text{M}}$ statement from Π_{M} . Once all the leaves have been folded into a single $\mathcal{R}_{\text{M}}$ statement, its vectors $\mathbf{u}, \mathbf{v}$ are no longer Lagrange evaluation vectors, so the claim $y = \mathbf{u}^\top \mathbf{M} \mathbf{v}$ must be reduced back to a bivariate polynomial evaluation. This is done with a univariate sumcheck over $\mathbb{H}$ (Lemma 2.35), which introduces the auxiliary vector $\mathbf{w} := \mathbf{M} \mathbf{v}$ and splits the claim into the two halves $y = \mathbf{u}^\top \mathbf{w}$ and $\mathbf{w} = \mathbf{M} \mathbf{v}$.

Definition 3.12 ($\Pi_P : \mathcal{R}_M \rightarrow \mathcal{R}_P \times \mathcal{R}_{\text{PCO}}$, [PRZ26a, Sec. 2.5]). Write $u(X), v(X), w(X)$ for the interpolants over $\mathbb{H}$ of $\mathbf{u}, \mathbf{v}$ and $\mathbf{w} := \mathbf{M}\mathbf{v}$, and $\Lambda(\alpha, X) := \boldsymbol{\lambda}(\alpha)^\top \boldsymbol{\lambda}(X)$. Let $P(X, Y)$ be the bivariate polynomial of Proposition 2.5 interpolating $\mathbf{M}$ over $\mathbb{H} \times \mathbb{H}$, so that $P(\alpha, \beta) = \boldsymbol{\lambda}(\alpha)^\top \mathbf{M} \boldsymbol{\lambda}(\beta)$, and write $P(\alpha, X)$ for its partial evaluation at α in the first variable.

1. $\mathcal{P}$ computes $\mathbf{w} := \mathbf{M}\mathbf{v}$ and sends commitments to $w(X)$ and to the degree-shifted copies of N_u, T_u, N_v, T_v and w .
2. $\mathcal{V}$ samples $\alpha, \eta \leftarrow \mathbb{F}$ and sends them.
3. Both parties define

$$g(X) := u(X) \cdot w(X), \quad g'(X) := \Lambda(\alpha, X) \cdot w(X) - P(\alpha, X) \cdot v(X),$$

and $G(X) := g(X) + \eta \cdot g'(X)$, so that $\sum_{h \in \mathbb{H}} g(h) = y$ and $\sum_{h \in \mathbb{H}} g'(h) = 0$.

4. $\mathcal{P}$ applies Lemma 2.35 to write $G(X) - y/n = X \cdot q_0(X) + (X^n - 1) \cdot q_1(X)$ and sends commitments to q_0, q_1 and to their degree-shifted copies.
5. $\mathcal{V}$ samples $\beta \leftarrow \mathbb{F}$ and sends it.
6. $\mathcal{P}$ sends the evaluations at β of $N_u, T_u, N_v, T_v, w, q_0, q_1$ together with the claimed value $y^* := P(\alpha, \beta)$.
7. $\mathcal{V}$ recovers $u(\beta) = N_u(\beta)/(nT_u(\beta))$ and $v(\beta) = N_v(\beta)/(nT_v(\beta))$, evaluates $\Lambda(\alpha, \beta)$ with Proposition 2.4, and asserts the sumcheck identity

$$(u(\beta) + \eta \Lambda(\alpha, \beta)) \cdot w(\beta) - \eta v(\beta) y^* = \frac{y}{n} + \beta q_0(\beta) + (\beta^n - 1) q_1(\beta).$$

8. $\mathcal{V}$ samples $\eta' \leftarrow \mathbb{F}$; both parties batch all committed polynomials, shifted and unshifted, with the powers of η' into a single commitment $\mathbf{c}^*$ and value y^{**} .
9. $\mathcal{V}$ outputs the $\mathcal{R}_P$ statement (α, β, y^*) together with the $\mathcal{R}_{\text{PCO}}$ statement $(\mathbf{c}^*, \beta, y^{**})$, and $\mathcal{P}$ outputs the corresponding batched polynomial as the $\mathcal{R}_{\text{PCO}}$ witness.

This last reduction is important so that the protocol's output statement is a statement for the same relation $\mathcal{R}_P$ that the verifiers submitted as input.

3.3 Local Folding Scheme Compiler

In this section, we will present the protocol in its final form: a (plain) local folding scheme $\Lambda_{\mathbf{M}, \kappa}$ as in Definition 2.40. It will be shaped from the previous reductions of knowledge when transformed into non-interactive reductions of knowledge (NIRoK, see Definition 2.37), which will then be compiled into an $\mathcal{R}_{\text{PCC}}^\kappa$ -promise local folding scheme Λ'_κ and, finally, composed together into a (plain) local folding scheme Λ_κ . This process will be applied to the reduction $\Pi_{\mathbf{M}}$ from Definition 3.6, resulting in the final $\Lambda_{\mathbf{M}, \kappa}$. This step-by-step construction follows the structure of [PRZ26a].

The following definition works for a $\text{NIRoK} : \mathcal{R}^2 \rightarrow \mathcal{R} \times \mathcal{R}_{\text{PCC}}$ that discharges polynomial commitment constraint statements for $\mathcal{R}_{\text{PCC}}$ from a generic $\mathcal{R}$. It defines a promise local folding scheme $\Lambda'_\kappa = \text{LPCC}(\text{NIRoK}, \kappa)$ and describes the algorithms (LProve, LVerify) and the shape of the resulting proof (a κ -step path from $\mathcal{V}$'s leaf statement) and promises (κ statements for $\mathcal{R}_{\text{PCC}}$).

Definition 3.13 (L_{PCC} compiler, [PRZ26a, Fig. 11]). Let $K = 2^\kappa$ and let $\text{NIRoK} : \mathcal{R}^2 \rightarrow \mathcal{R} \times \mathcal{R}_{PCC}$ be a non-interactive reduction of knowledge. The compiler $L_{PCC}(\text{NIRoK}, \kappa)$ produces the scheme Λ'_κ , defined inductively on κ and sharing the generator of NIRoK .

- For $\kappa = 1$, $\Lambda'_1.L\text{Prove}$ runs

$$\pi, (x, x_{PCC}), (w, w_{PCC}) \leftarrow \text{NIRoK.Reduce}(\text{pp}, (x_0, w_0), (x_1, w_1))$$

and gives both leaves the same proof (x_0, x_1, π) and the same promise (x_{PCC}, w_{PCC}) . The verifier of leaf b accepts iff the b -th statement recorded in the proof is its own and NIRoK.Verify accepts the reduction of (x_0, x_1) to (x, x_{PCC}) .

- For $\kappa > 1$, $\Lambda'_\kappa.L\text{Prove}$ runs $\Lambda'_{\kappa-1}.L\text{Prove}$ on each half of the K pairs, reduces the two folded statements so obtained with one further call to NIRoK.Reduce , and appends the resulting node proof and promise to those already held by every leaf of the corresponding half. The verifier of leaf $\mathbf{b} = (b, \mathbf{b}')$ accepts iff $\Lambda'_{\kappa-1}.L\text{Verify}$ accepts its sub-path and NIRoK.Verify accepts the reduction performed at the root node.

A leaf therefore ends up with a κ -step path proof and with the κ promises collected along its path to the root.

The following theorem states that the L_{PCC} compiler from the previous definition does indeed output an $\mathcal{R}_{PCC}^\kappa$ -promise local folding scheme for $\mathcal{R}$. A reader interested in the proof of this theorem can check [PRZ26a, Sec. 7.1].

Theorem 3.14 ([PRZ26a, Thm. 7.1.1]). *Let $K = 2^\kappa$ be a constant and $\text{NIRoK} : \mathcal{R}^2 \rightarrow \mathcal{R} \times \mathcal{R}_{PCC}$ be a non-interactive reduction of knowledge. Then the construction $L_{PCC}(\text{NIRoK}, \kappa)$ is an $\mathcal{R}_{PCC}^\kappa$ -promise Local Folding scheme for $\mathcal{R}$.*

In a similar fashion, we present the definition of a compiler L that outputs a (plain) local folding scheme Λ_κ and the theorem that certifies that Λ_κ is indeed a (plain) local folding scheme for a generic relation $\mathcal{R}$.

Definition 3.15 (L compiler, [PRZ26a, Sec. 7.2]). Let $K = 2^\kappa$ and $\text{NIRoK} : \mathcal{R}^2 \rightarrow \mathcal{R} \times \mathcal{R}_{PCC}$ be as above. The compiler $L(\text{NIRoK}, \kappa)$ discharges the promises that $L_{PCC}(\text{NIRoK}, \kappa)$ leaves open. Writing Π_D , Π_{PC} and Π_{PCO} for the reductions of Section 3.2 and FS_{MT} for the compiler of Definition 3.9, the prover

- (1) runs $L_{PCC}(\text{NIRoK}, \kappa)$, reducing the K statements to a single folded x^* and leaving each leaf i with a promise $x_{PCC,i}$ and a path proof $\pi_{1,i}$;
- (2) runs $\text{FS}_{MT}(\Pi_{PC} \circ \Pi_D, \kappa)$ to reduce each promise $x_{PCC,i}$ to an opening claim $x_{PCO(x),i}$ at a point x common to all i , with proof $\pi_{2,i}$;
- (3) runs $L_{PCC}(\text{FS}(\Pi_{PCO}), \kappa)$ to reduce all the $x_{PCO(x),i}$ to a single $x_{PCO(x)}$, with proofs $\pi_{3,i}$;
- (4) proves $x_{PCO(x)}$ with the polynomial commitment scheme, with proof π_{PC} ,

and sets the i -th proof to $\pi_i := ((x_{PCC,i}, \pi_{1,i}), (x_{PCO(x),i}, \pi_{2,i}), (x_{PCO(x)}, \pi_{3,i}), \pi_{PC})$. The verifier of leaf i checks the four steps in the same order, against its own statement.

Theorem 3.16 ([PRZ26a, Thm. 7.2.1]). *Let $K = 2^\kappa$ be a constant and $\text{NIRoK} : \mathcal{R}^2 \rightarrow \mathcal{R} \times \mathcal{R}_{PCC}$ be a non-interactive reduction of knowledge. Then the construction $\Lambda_\kappa = L(\text{NIRoK}, \kappa)$ is a (plain) Local Folding scheme for $\mathcal{R}$.*

When the compilers L_{PCC} and L are applied to the **NIRoK** obtained from $\text{FS}(\Pi_{\mathbf{M}})$, we get: (i) $\Lambda'_{\mathbf{M},\kappa} = L_{\text{PCC}}(\text{FS}(\Pi_{\mathbf{M}}), \kappa)$ which is an $\mathcal{R}_{\text{PCC}}^\kappa$ -promise Local Folding scheme for $\mathcal{R}_{\mathbf{M}}$; and (ii) $\Lambda_{\mathbf{M},\kappa} = L(\text{FS}(\Pi_{\mathbf{M}}), \kappa)$, a (plain) local folding scheme for $\mathcal{R}_{\mathbf{M}}$. The step-by-step description of the amortization protocol $\Lambda_{\mathbf{M},\kappa}$ can be found in Section 3.4.

3.4 Protocol

In this section, we will present the protocol step by step from the previous sections' relations and reductions. The protocol considers the following setup:

- K independent verifiers, each holding an $\mathcal{R}_P$ statement that it wants to delegate proving.
- A single prover, which can be viewed as a *Server* that wants to amortize proving the K statements.

Broadly speaking, the protocol's prover picks up many independent statements that need proving and reduces them to a single statement of the same type (which also needs proving). Of course, this reduction (or *folding*) process, together with the last statement's proving time, is *cheaper* than naively proving all the original statements. This is what we refer to as *amortization*: saving computational time for the prover, when compared to working with each verifier separately.

This overview of the protocol is missing a critical part: to fold the input statements, we *postpone* some checks at every step. By postponing, we mean that the prover will provide a proof at a later stage of the protocol. Without these postponed proofs (or *promises*), nothing holds and the verifier should never accept. Of course, the cost of this postponed proving is still cheaper than the naïve baseline.

At the end of the protocol, the verifier must check the proof of the folding procedure, the final folded statement's proof *and* the postponed proof of the promises. Figure 3.1 summarizes this shape.

As mentioned above, amortization in this context refers to saving computational work for the prover, when comparing the amortization protocol to just proving the K statements individually. If $|\mathcal{P}|$ denotes the group operations of a SNARK's prover and $|\mathcal{V}|$ those of the same SNARK's verifier, the cryptographic cost for the prover in the trivial case is linear in the number of statements and $|\mathcal{P}|$ (i.e. $\mathcal{O}(K \cdot |\mathcal{P}|)$). The cost of each of the K verifiers stays $|\mathcal{V}|$. In the amortized protocol, the cryptographic cost of the prover is reduced to $\mathcal{O}(K \log K + |\mathcal{P}|)$, while each verifier pays $\mathcal{O}(\log K + |\mathcal{V}|)$.

We now compose the reductions of Section 3.2 into the full amortization protocol. Let $K = 2^\kappa$ be the number of delegated statements, one per verifier.

Figure 3.2 traces the protocol as a directed acyclic graph whose vertices record how many statements of each relation are still open, and whose edges are the reductions that consume them. Two branches leave the folding step: the left one carries the single folded $\mathcal{R}_{\mathbf{M}}$ root to its final $\mathcal{R}_P$ claim, while the right one collects the promises accumulated during folding and collapses them to a single $\mathcal{R}_{\text{PCO}}$ statement.

More precisely, the protocol starts with $K \cdot \mathcal{R}_P$ sent to $\mathcal{P}$ by the K independent verifiers and proceeds as:

- (1) $\mathcal{P}$ and $\mathcal{V}$ encode the $K \cdot \mathcal{R}_P$ leaves as $K \cdot \mathcal{R}_{\mathbf{M}}$ statements, promising $K \cdot \mathcal{R}_{\text{PCC}}$ statements that will be added to the rest of the $\mathcal{R}_{\text{PCC}}$ statements in step (4).

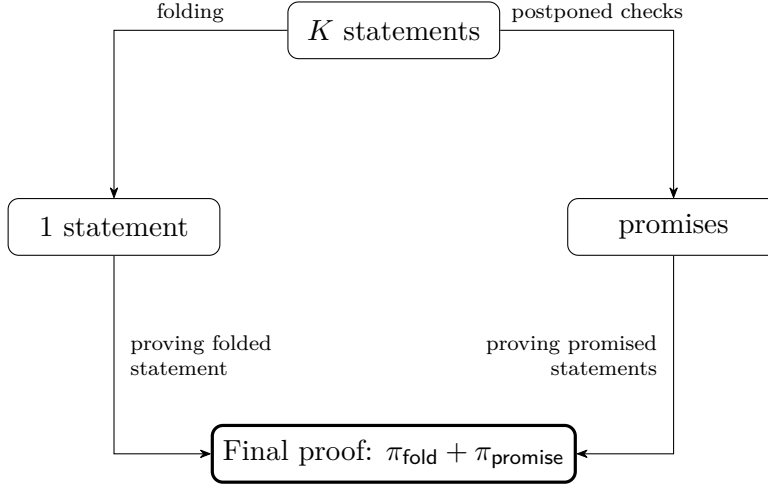


Figure 3.1: Schematic view of the amortization protocol. The K input statements are folded into a single statement of the same type, while the checks postponed at every fold step accumulate as promises. Proving the folded statement and proving the promises are two separate tasks, and the final proof is the pair. Figure 3.2 is this same picture with the relations and the statement counts filled in.

- (2) $\mathcal{P}$ folds $K \cdot \mathcal{R}_{\mathbf{M}}$ into $1 \cdot \mathcal{R}_{\mathbf{M}}$ and $K\kappa \cdot \mathcal{R}_{\text{PCC}}$ promises.
- (3) The folded $\mathcal{R}_{\mathbf{M}}$ is reduced to an $\mathcal{R}_P$ statement and the resulting $\mathcal{R}_{\text{PCO}}$ is opened inside the proof.
- (4) The $K \cdot \mathcal{R}_{\text{PCC}}$ statements promised in step (1) are added to the $K\kappa \cdot \mathcal{R}_{\text{PCC}}$ promises from step (2).
- (5) All $K(\kappa + 1) \cdot \mathcal{R}_{\text{PCC}}$ are reduced to $K(\kappa + 1) \cdot \mathcal{R}_{\text{PCC-D}}$ by removing the degree check.
- (6) Every $(\kappa + 1) \cdot \mathcal{R}_{\text{PCC-D}}$ from each $\mathcal{V}$ is batched into an $\mathcal{R}_{\text{PCO}(x)}$ statement, leaving $K \cdot \mathcal{R}_{\text{PCO}(x)}$ statements.
- (7) The $K \cdot \mathcal{R}_{\text{PCO}(x)}$ statements are batched into $1 \cdot \mathcal{R}_{\text{PCO}}$.

The whole batch therefore ends in a single $\mathcal{R}_P$ claim plus a single polynomial commitment opening, both of which are independent of K . A local verifier checks only its own slice: its κ -step fold path, its reconstructed promise bundle, the shared discharge, and the Π_P reduction, so its work grows only as $\log K$.

To finish this chapter, we will present the computational cost of the amortization protocol for the prover $\mathcal{P}$ and for the verifier $\mathcal{V}$. The reference point is the following bound, which instantiates the local folding scheme of Theorem 3.16 with the KZG scheme of Definition 2.26.

Corollary 3.17 ([PRZ26a, Cor. 8.2.1]). *Let $\mathbf{M} \in \mathbb{F}^{n \times n}$ be a matrix with sparsity $\|\mathbf{M}\| = s_{\mathbf{M}} = \omega(K)$ and let $\Lambda_{\mathbf{M}, \kappa} = \mathbf{L}(\text{FS}(\Pi_{\mathbf{M}}), \kappa)$ using the KZG commitment scheme. Let $(\mathbf{pp}, \mathbf{x}_{\mathbf{b}}, \mathbf{w}_{\mathbf{b}})_{\mathbf{b} \in \{0,1\}^\kappa}$ be statements derived from statements (α, β, y) of $\mathcal{R}_P$ using the KZG commitment scheme, and consider an execution of $\Lambda_{\mathbf{M}, \kappa}.\text{LProve}$ and $\Lambda_{\mathbf{M}, \kappa}.\text{LVerify}$ with such inputs. Then*

- the complexity of LProve is dominated by

$$\left(K \cdot s_{\mathbf{M}} + K \cdot \log^2 K + (n + K) \cdot \log(n + K) \right) \mathbb{F} + (n + K \log K) \mathbb{G},$$

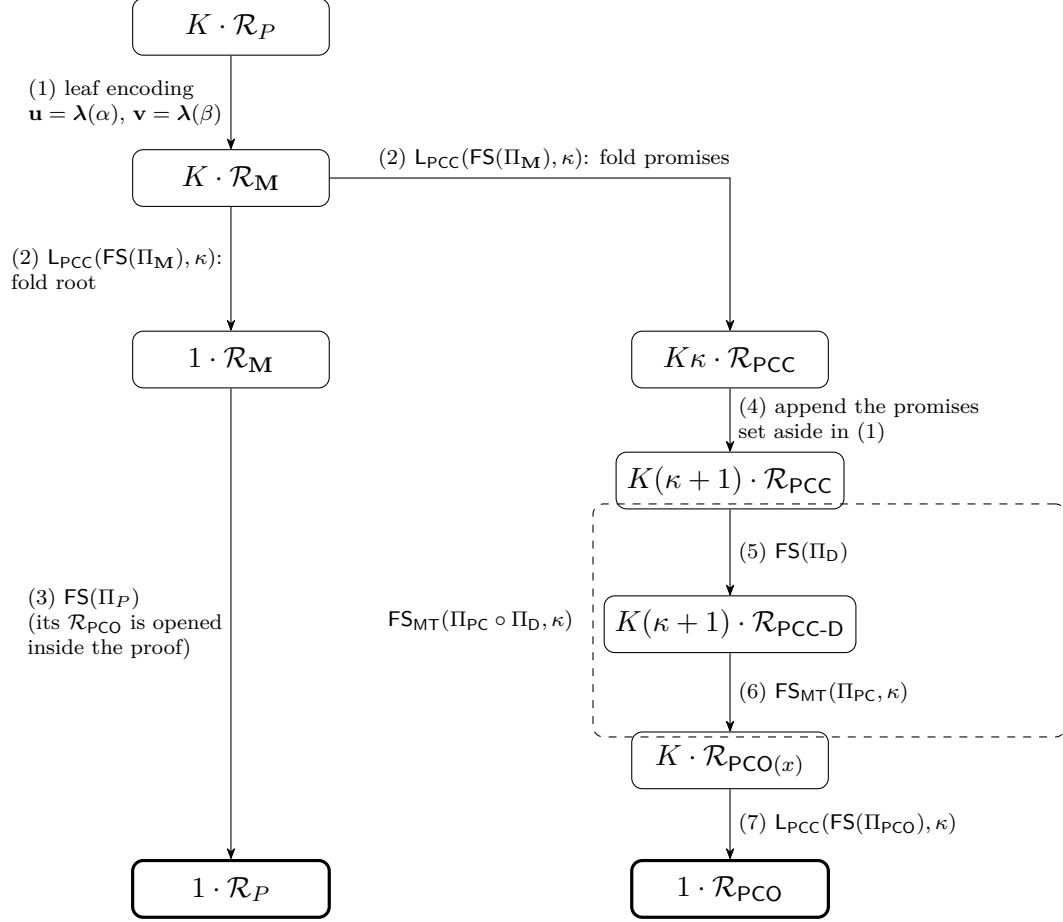


Figure 3.2: The amortization protocol as a DAG. Vertices give the number of open statements of each relation and edges the reductions consuming them, numbered as in the enumeration above. With $K = 2^\kappa$ leaves: (1) each leaf's $\mathcal{R}_P$ claim is encoded as an $\mathcal{R}_M$ statement, setting aside one $\mathcal{R}_{\text{PCC}}$ promise per leaf; (2) the promise folding scheme $\text{L}_{\text{PCC}}(\text{FS}(\Pi_M), \kappa)$ folds the K leaves through a binary tree, emitting one $\mathcal{R}_{\text{PCC}}$ promise per fold node; (3) $\text{FS}(\Pi_P)$ reduces the folded root to the final $\mathcal{R}_P$ claim; (4) the K promises set aside in (1) are appended; (5) $\text{FS}(\Pi_D)$ removes the degree constraints and (6) $\text{FS}_{\text{MT}}(\Pi_{\text{PC}}, \kappa)$ collapses each verifier's bundle of $\kappa + 1$ statements to one opening claim at a point x made common to all K executions, the dashed box marking these two as the inner reductions that $\text{FS}_{\text{MT}}(\Pi_{\text{PC}} \circ \Pi_D, \kappa)$ composes in step (2) of Definition 3.15; and (7) $\text{L}_{\text{PCC}}(\text{FS}(\Pi_{\text{PCO}}), \kappa)$ batches the K claims into one. The two thick vertices are the outputs of the protocol.

- the complexity of LVerify is dominated by $1 \cdot \mathsf{P} + \log K \cdot (\mathbb{F} + \mathbb{G} + \rho)$,
- the proof size is $\lambda \cdot \log K$,

where $\mathbb{G}, \mathbb{F}$ denote group and field operations, P pairing operations, and ρ random oracle queries.

Table 3.1 breaks that cost down along the steps of Figure 3.2. The verifier column is the work of one local verifier, not of all K of them together, and the hypothesis $s_{\mathbf{M}} = \omega(K)$ is what lets the two K^2 terms of steps (6) and (7) be absorbed into $K \cdot s_{\mathbf{M}}$.

Step	$\mathcal{P}$	$\mathcal{V}$	Proof size
(1) leaf encoding	$\mathcal{O}(K) \mathbb{G}$	$\mathcal{O}(1) \mathbb{G}$	—
(2) $\text{LPCC}(\text{FS}(\Pi_{\mathbf{M}}), \kappa)$	$(K s_{\mathbf{M}} + K \log^2 K) \mathbb{F} + K \log K \mathbb{G}$	$\log K (\mathbb{F} + \rho)$	$\lambda \log K$
(3) $\text{FS}(\Pi_P)$	$(s_{\mathbf{M}} + (n + K) \log(n + K)) \mathbb{F} + (n + K) \mathbb{G}$	$1 \mathsf{P} + \log n \mathbb{F}$	λ
(4) append the promises	—	—	—
(5) $\text{FS}(\Pi_D)$	$K \log K \mathbb{F} + K \log K \mathbb{G}$	$\log K \mathbb{G}$	$\lambda \log K$
(6) $\text{FS}_{\text{MT}}(\Pi_{\text{PC}}, \kappa)$	$K^2 \mathbb{F} + K \mathbb{G}$	$\log K (\mathbb{F} + \mathbb{G} + \rho)$	$\lambda \log K$
(7) $\text{LPCC}(\text{FS}(\Pi_{\text{PCO}}), \kappa)$	$K^2 \mathbb{F} + K \mathbb{G}$	$\log K (\mathbb{F} + \mathbb{G} + \rho)$	$\lambda \log K$
PC opening of the final $\mathcal{R}_{\text{PCO}}$	$(n + K) \log(n + K) \mathbb{F} + (n + K) \mathbb{G}$	$1 \mathsf{P}$	λ
Total	$(K s_{\mathbf{M}} + K \log^2 K + (n + K) \log(n + K)) \mathbb{F} + (n + K \log K) \mathbb{G}$	$2 \mathsf{P} + \log n \mathbb{F} + \log K (\mathbb{F} + \mathbb{G} + \rho)$	$\lambda \log K$

Table 3.1: Computational cost of the amortization protocol under the KZG instantiation, broken down along the steps of Figure 3.2, with $K = 2^\kappa$, $s_{\mathbf{M}} = \|\mathbf{M}\|$ and $s_{\mathbf{M}} = \omega(K)$. Here $\mathbb{F}$ and $\mathbb{G}$ denote field and group operations, P pairing operations and ρ random oracle queries. The total row sums every step; dropping step (3) from it gives exactly Corollary 3.17.

Every row but the third comes from Corollary 3.17, whose statement analyzes the local folding scheme $\Lambda_{\mathbf{M}, \kappa} = \text{L}(\text{FS}(\Pi_{\mathbf{M}}), \kappa)$ together with the opening of the single $\mathcal{R}_{\text{PCO}}$ statement it produces. Step (3) falls outside the original analysis, since the corollary takes the folded $\mathcal{R}_{\mathbf{M}}$ root as its endpoint whereas Π_P carries that root on to the final $\mathcal{R}_P$ claim; deleting its row from the table recovers the corollary’s bound.

With step (3), the prover’s cost and the proof size stay untouched. On the prover side, the two sparse passes over $\mathbf{M}$ needed to form $\mathbf{w} = \mathbf{M}\mathbf{v}$ and $M(\alpha, \cdot)$ contribute $s_{\mathbf{M}} \mathbb{F}$, absorbed by $K \cdot s_{\mathbf{M}}$; the constant number of interpolations over $\mathbb{H}$ and FFT-based products of Definition 3.12 contribute $n \log n \mathbb{F}$; and the commitments and the batched opening are taken over polynomials whose degree-shifted copies reach the bound $D = \mathcal{O}(n + K)$, so they contribute the same $(n + K) \log(n + K) \mathbb{F} + (n + K) \mathbb{G}$ as the last row. The proof carries a constant number of commitments and evaluations, absorbed by $\lambda \log K$. Where step (3) does show is the verifier: Π_P ships its own batched opening and the verification of Definition 3.12 checks it in-proof, so each local verifier performs a *second* pairing check, and testing the sumcheck identity requires α^n and β^n , which costs $\log n$ field operations. Neither term appears in the corollary, precisely because Π_P is not part of what it measures.

In Chapter 5, we will test the protocol and its costs when running it over the SNARK *Marlin* [CHM⁺20]. As a reference, we expect the cost comparison to follow Table 3.2.

Table 3.2 states the amortized prover cost in $s_{\mathbf{M}}$ alone, whereas Table 3.1 keeps n and $s_{\mathbf{M}}$ apart. [PRZ26a] notes that asymptotically $s_{\mathbf{M}} = \mathcal{O}(n)$, while sparse circuits in practice satisfy $s_{\mathbf{M}} \approx 1.5n$ to $3n$. Under that hypothesis we get $s_{\mathbf{M}} = \Theta(n)$ and, together with $s_{\mathbf{M}} = \omega(K)$ from Corollary 3.17, the prover cost of Table 3.1 collapses term by term onto the one above:

	With amortization	Without amortization
Prover time	$(K \cdot s_{\mathbf{M}} + s_{\mathbf{M}} \cdot \log s_{\mathbf{M}}) \mathbb{F} + (K \cdot \log K + s_{\mathbf{M}}) \mathbb{G}$	$K(s_{\mathbf{M}} \cdot \log s_{\mathbf{M}} \mathbb{F} + s_{\mathbf{M}} \mathbb{G})$
Verifier time	$2 \cdot \mathbf{P} + \log n \mathbb{F} + \log K \cdot (\mathbb{F} + \mathbb{G} + \rho)$	$1 \cdot (\mathbb{F} + \mathbb{G} + \rho + \mathbf{P})$
Proof size	$\lambda \cdot \log K$	λ

Table 3.2: Asymptotic comparison of the witness-independent polynomial-evaluation component when serving K independent clients, measured in group ($\mathbb{G}$), field ($\mathbb{F}$), random oracle queries (ρ) and pairing ($\mathbf{P}$) operations. $s_{\mathbf{M}}$ denotes the sparsity of the underlying constraint system and λ the security parameter. The non-amortized costs correspond to standard R1CS/CCS-based SNARK constructions such as Marlin [CHM⁺20]. Adapted from [PRZ26a, Tab. 1].

- $K \cdot s_{\mathbf{M}}$ is common to both;
- $K \log^2 K = o(K^2) = o(K \cdot s_{\mathbf{M}})$, because $s_{\mathbf{M}} = \omega(K)$, so the folding overhead is absorbed by the first term;
- $(n + K) \log(n + K) = \Theta(s_{\mathbf{M}} \log s_{\mathbf{M}})$, because $n = \Theta(s_{\mathbf{M}})$ and $K = o(s_{\mathbf{M}})$;
- in the group column, $K \log K$ is common and $n = \Theta(s_{\mathbf{M}})$.

Together these give

$$\begin{aligned}
& \left(K s_{\mathbf{M}} + K \log^2 K + (n + K) \log(n + K) \right) \mathbb{F} + (n + K \log K) \mathbb{G} \\
&= \Theta \left(\left(K s_{\mathbf{M}} + s_{\mathbf{M}} \log s_{\mathbf{M}} \right) \mathbb{F} + \left(K \log K + s_{\mathbf{M}} \right) \mathbb{G} \right),
\end{aligned}$$

so the two prover expressions are the same bound written in different variables. The absorption does need $n = \mathcal{O}(s_{\mathbf{M}})$: for a matrix with $o(n)$ non-zero entries the term $(n + K) \log(n + K)$ would genuinely exceed $s_{\mathbf{M}} \log s_{\mathbf{M}}$.

Chapter 4

SNARK Instantiation: Marlin

In this chapter we present the Marlin SNARK [CHM⁺20] which will be used to produce the initial $K \cdot \mathcal{R}_P$ statements required as inputs to the amortization protocol.

The Marlin SNARK was introduced in [CHM⁺20] and is based on the idea of *Algebraic Holographic Proofs* (AHP) for R1CS relations. AHPs are a class of interactive oracle proof system (IOPs) that modify its definition in two ways:

- Holographic: the verifier does not receive its input in full, but rather as oracle access to an encoding of it. This allows for a verifier that runs faster than the time needed to read the whole input. The encoding of the input is carried by an *indexer*, which receives as input an index i to be processed and outputs one or more univariate polynomials that encode it.
- Algebraic: any prover (honest or malicious) must produce oracles that are low-degree polynomials. This property restricts completeness and relaxes soundness.

The underlying AHP of Marlin is a 3-round IOP that can be split into two phases: a witness-dependent phase and a witness-independent phase. As introduced in Chapter 1, our amortization protocol works over witness-independent statements for the relation $\mathcal{R}_P$. This is indeed the case for the Marlin SNARK and it corresponds to the last round of the AHP. The last round can be seen as a solution to the *lincheck problem*. In Section 4.1 we introduce the witness-dependent part of the AHP, which outputs the raw statement for $\mathcal{R}_P$. In Section 4.2 we will focus on describing the lincheck problem and the solution proposed by Marlin. We will use such solution as a benchmark to compare the performance of our amortization protocol.

4.1 Witness Dependence

This section records the beginning of the AHP: the rounds that precede the lincheck, the claim they leave behind, and how that claim is brought into the shape the amortization protocol consumes. We follow the optimized five-message AHP of [CHM⁺20, App. E], which folds Marlin’s intermediate sumcheck into a single committed polynomial $t(X)$.

The input is an R1CS instance $(\mathbf{A}, \mathbf{B}, \mathbf{C})$ over $\mathbb{H}$ together with a satisfying assignment $\mathbf{z}$ with $\mathbf{Az} \circ \mathbf{Bz} = \mathbf{Cz}$, as in Definition 2.14. The matrices index $\mathbf{z}$ directly over $\mathbb{H}$ and there is a single public input $z_1 = 1$, so the public part occupies the subgroup $\{1\}$, whose vanishing polynomial is $X - 1$. Throughout, $\mathbf{M}$ ranges over $\{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$ with bundling weights $\eta_{\mathbf{A}} = 1$, $\eta_{\mathbf{B}} = \eta$ and $\eta_{\mathbf{C}} = \eta^2$, and D is the degree bound of the SRS.

Round 1.

- (1) $\mathcal{P}$ computes $\mathbf{z}_\mathbf{A} := \mathbf{A}\mathbf{z}$ and $\mathbf{z}_\mathbf{B} := \mathbf{B}\mathbf{z}$, and interpolates $\hat{z}, \hat{z}_\mathbf{A}, \hat{z}_\mathbf{B} \in \mathbb{F}^{<n}[X]$ over $\mathbb{H}$.
- (2) $\mathcal{P}$ forms the shifted witness encoding $\hat{w}(X) := (\hat{z}(X) - \hat{x}(X))/(X - 1)$, where $\hat{x}$ interpolates the public input; the division is exact because $\hat{z}(1) = z_1$.
- (3) If zero knowledge is required, $\mathcal{P}$ blinds $\hat{w}, \hat{z}_\mathbf{A}, \hat{z}_\mathbf{B}$ with random multiples of $v_\mathbb{H}$, which leaves their values on $\mathbb{H}$ — and hence the R1CS relation — untouched, and samples a mask $s \in \mathbb{F}^{<3n}[X]$ with $\sum_{h \in \mathbb{H}} s(h) = 0$.
- (4) $\mathcal{P}$ sends commitments to $\hat{w}, \hat{z}_\mathbf{A}, \hat{z}_\mathbf{B}$ and, when present, to s .

Round 2.

- (5) $\mathcal{V}$ samples $\alpha \leftarrow \mathbb{F} \setminus \mathbb{H}$ and $\eta \leftarrow \mathbb{F}$ and sends them.
- (6) Both parties fix the row kernel $r_\alpha(X) := u_\mathbb{H}(\alpha, X)$, where $u_\mathbb{H}(X, Y) := (X^n - Y^n)/(X - Y)$ is the *unnormalized bivariate Lagrange kernel*, so that $r_\alpha(h_i) = v_\mathbb{H}(\alpha)/(\alpha - h_i)$.
- (7) $\mathcal{P}$ computes $t \in \mathbb{F}^{<n}[X]$, the polynomial defined in Equation (4.1) below, and forms
$$q_1(X) := s(X) + r_\alpha(X) \left(\hat{z}_\mathbf{A}(X) + \eta \hat{z}_\mathbf{B}(X) + \eta^2 \hat{z}_\mathbf{A}(X) \hat{z}_\mathbf{B}(X) \right) - t(X) \hat{z}(X).$$
- (8) $\mathcal{P}$ applies Lemma 2.35 to write $q_1(X) = h_1(X)v_\mathbb{H}(X) + Xg_1(X)$ with $\deg g_1 \leq n-2$, and sends commitments to t, g_1, h_1 and to the degree-shifted copy $X^{D-(n-1)}g_1(X)$ that binds that degree.
- (9) $\mathcal{V}$ samples $\beta \leftarrow \mathbb{F} \setminus \mathbb{H}$ and sends it.
- (10) $\mathcal{P}$ sends the evaluations at β of $\hat{w}, \hat{z}_\mathbf{A}, \hat{z}_\mathbf{B}, s, t, g_1, h_1$, together with a single η' -batched KZG opening of all committed polynomials at β .
- (11) $\mathcal{V}$ recovers $\hat{z}(\beta) = \hat{w}(\beta)(\beta - 1) + 1$, evaluates $r_\alpha(\beta)$ and $v_\mathbb{H}(\beta)$ in $\mathcal{O}(\log n)$ field operations, and accepts iff the batched opening verifies and

$$s(\beta) + r_\alpha(\beta) \left(\hat{z}_\mathbf{A}(\beta) + \eta \hat{z}_\mathbf{B}(\beta) + \eta^2 \hat{z}_\mathbf{A}(\beta) \hat{z}_\mathbf{B}(\beta) \right) - t(\beta) \hat{z}(\beta) = h_1(\beta)v_\mathbb{H}(\beta) + \beta g_1(\beta).$$

The polynomial t of step (7) is obtained by transposing the row weights $r_\alpha(h_i)$ through each matrix and bundling the three results, which written out gives

$$t(X) = \sum_{\mathbf{M} \in \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}} \eta_\mathbf{M} \sum_{i,j \in [n]} \mathbf{M}_{i,j} u_\mathbb{H}(\alpha, h_i) \lambda_j(X) \in \mathbb{F}^{<n}[X], \quad (4.1)$$

that is, the interpolant over $\mathbb{H}$ of $t(h_j) = \sum_{\mathbf{M}} \eta_\mathbf{M} \sum_i \mathbf{M}_{i,j} u_\mathbb{H}(\alpha, h_i)$. This is what makes the check of step (11) sound: by construction $\sum_{h \in \mathbb{H}} q_1(h) = \sum_{\mathbf{M}} \eta_\mathbf{M} \sum_{i \in [n]} r_\alpha(h_i) \left((\mathbf{M}\mathbf{z})_i - (\mathbf{z}_\mathbf{M})_i \right)$, with $\mathbf{z}_\mathbf{C} := \mathbf{z}_\mathbf{A} \circ \mathbf{z}_\mathbf{B}$, which vanishes whenever $\mathbf{z}$ satisfies the R1CS instance and, for random α and η , only then except with probability $\mathcal{O}(n)/|\mathbb{F}|$ by Corollary 2.7.

Departures from the published protocol

Four deliberate departures from [CHM⁺20], none of which changes what the rounds prove.

- (i) The oracle $\hat{z}_C$ and the quotient of the entry-wise product check are dropped, and $\hat{z}_A \hat{z}_B$ is substituted for $\hat{z}_C$ inside the bundle. The C summand then reads $Cz = z_A \circ z_B$, so the Hadamard constraint is absorbed into the lincheck bundle and needs no separate check. This saves one commitment at the price of raising $\deg q_1$ to $3n - 3$.
- (ii) The three bundling challenges are replaced by the powers $1, \eta, \eta^2$ of a single one, matching the three-matrix convention of Section 5.1.
- (iii) The mask is drawn with zero sum over $\mathbb{H}$, so the claimed sum stays 0 and need not be sent.
- (iv) These rounds carry their own batched opening at β instead of joining the lincheck's, which bills them one extra degree- D multi-scalar multiplication.

From $t(\beta)$ to an $\mathcal{R}_P$ statement

Nothing in the two rounds ties the committed t to the matrices: step (11) treats $t(\beta)$ as an *asserted* value. Proving that assertion against (A, B, C) is the job of Marlin's holographic lincheck (see Definition 4.4), and it is where delegation becomes possible, since evaluating Equation (4.1) at β gives

$$t(\beta) = \sum_{\mathbf{M}} \eta_{\mathbf{M}} \sum_{i,j \in [n]} \mathbf{M}_{i,j} u_{\mathbb{H}}(\alpha, h_i) \lambda_j(\beta),$$

which depends only on the public matrices and on (α, β, η) (the witness has dropped out). This is what makes K independent Marlin proofs leave K statements of the same shape, all about the same constraint system.

That value is not yet an $\mathcal{R}_P$ statement, because $\mathcal{R}_P$ weights rows by $\lambda_i(\alpha)$ and not by $u_{\mathbb{H}}(\alpha, h_i)$. The following steps close the gap.

- (1) **Locate the discrepancy.** From the closed forms $u_{\mathbb{H}}(\alpha, h_i) = v_{\mathbb{H}}(\alpha)/(\alpha - h_i)$ and $\lambda_i(\alpha) = \frac{h_i}{n} \cdot \frac{v_{\mathbb{H}}(\alpha)}{\alpha - h_i}$,

$$\lambda_i(\alpha) = \frac{h_i}{n} u_{\mathbb{H}}(\alpha, h_i).$$

The two weightings differ by the factor h_i/n , which depends on the row, so no rescaling of the single field element $t(\beta)$ can convert one into the other.

- (2) **Replace the kernel.** Take $r_{\alpha}(X) := \Lambda(\alpha, X)$ with $\Lambda(\alpha, X) := \mathbf{\lambda}(\alpha)^{\top} \mathbf{\lambda}(X)$, the unique polynomial of degree less than n with $\Lambda(\alpha, h_i) = \lambda_i(\alpha)$. By Proposition 2.4 it has the closed form

$$\Lambda(\alpha, X) = \frac{(X^n - 1)\alpha - (\alpha^n - 1)X}{n(X - \alpha)},$$

so $\mathcal{V}$ evaluates $\Lambda(\alpha, \beta)$ in $\mathcal{O}(\log n)$ field operations, exactly as it does $u_{\mathbb{H}}(\alpha, \beta)$ in step (11). This is the same kernel and the same evaluation that Π_P uses in Definition 3.12.

(3) **Re-run round 2 unchanged.** The sumcheck identity of step (7) does not depend on which row weights are used: for any $w_1, \dots, w_n$, defining t by $t(h_j) = \sum_{\mathbf{M}} \eta_{\mathbf{M}} \sum_i \mathbf{M}_{i,j} w_i$ gives $\sum_i w_i (\mathbf{M}\mathbf{z})_i = \sum_j t(h_j) z_j$, which is all q_1 requires. Equivalently, this is the original round 2 run against the row-rescaled matrices $\mathbf{D}\mathbf{M}$ with $\mathbf{D} = \text{diag}(h_1/n, \dots, h_n/n)$.

(4) **Read off the statement.** The polynomial that now comes out of round 2 is $t(X) = \sum_{\mathbf{M}} \eta_{\mathbf{M}} \sum_{i,j} \mathbf{M}_{i,j} \lambda_i(\alpha) \lambda_j(X)$, so

$$t(\beta) = \sum_{\mathbf{M}} \eta_{\mathbf{M}} \sum_{i,j \in [n]} \mathbf{M}_{i,j} \lambda_i(\alpha) \lambda_j(\beta) = P_{\mathbf{A}}(\alpha, \beta) + \eta P_{\mathbf{B}}(\alpha, \beta) + \eta^2 P_{\mathbf{C}}(\alpha, \beta) =: y,$$

and (α, β, y) is exactly the three-matrix $\mathcal{R}_P$ statement of Section 5.1, in the shape Definition 4.4 expects.

The implementation does not perform this substitution: it recomputes y from the matrices at the boundary and hands that to the lincheck, rather than passing $t(\beta)$ through. The two rounds therefore serve only to measure the witness-dependent share of a Marlin proof, for which both give the same degrees, the same m and the same operation counts, and differ only in the field values carried.

4.2 The Lincheck Problem

The lincheck problem asks whether two vectors, presented as low-degree encodings over $\mathbb{H}$, are related by a prescribed matrix. It is the algebraic core of every R1CS argument built on univariate polynomials: the three constraints $\mathbf{A}\mathbf{z} = \mathbf{z}_{\mathbf{A}}$, $\mathbf{B}\mathbf{z} = \mathbf{z}_{\mathbf{B}}$ and $\mathbf{C}\mathbf{z} = \mathbf{z}_{\mathbf{C}}$ of Definition 2.14 are three instances of it.

Definition 4.1 (Lincheck problem, [CHM⁺20, Sec. 5.2]). Let $\mathbb{H} = \{h_1, \dots, h_n\}$ be a multiplicative subgroup of $\mathbb{F}^*$, let $\mathbf{M} \in \mathbb{F}^{n \times n}$ and let $d \geq n$. Given oracle access to two polynomials $f_1, f_2 \in \mathbb{F}^{<d}[X]$, the *lincheck problem* for $\mathbf{M}$ asks to decide whether

$$f_1(h_i) = \sum_{j \in [n]} \mathbf{M}_{i,j} f_2(h_j) \quad \text{for every } i \in [n],$$

that is, whether the vectors that f_1 and f_2 encode over $\mathbb{H}$ are related by $\mathbf{M}$.

Marlin solves Definition 4.1 *holographically*: an indexer preprocesses $\mathbf{M}$ once into a sparse encoding over a second subgroup, and the verifier is given oracle access to that encoding rather than to $\mathbf{M}$ itself. Throughout this section $\mathbb{K} = \{k_1, \dots, k_m\}$ denotes that second multiplicative subgroup of $\mathbb{F}^*$, whose order $m \geq s_{\mathbf{M}}$ is the domain size of the encoding. It plays no part in the amortization and should not be confused with the number K of amortized statements of Chapter 3. We write $v_{\mathbb{H}}(X) = X^n - 1$ and $v_{\mathbb{K}}(X) = X^m - 1$ for the vanishing polynomials of the two subgroups.

Definition 4.2 (Sparse matrix encoding, [CHM⁺20, Sec. 5.2.1]). Fix a canonical ordering $(i_1, j_1), \dots, (i_{s_{\mathbf{M}}}, j_{s_{\mathbf{M}}})$ of the non-zero entries of $\mathbf{M}$. The *indexer* outputs the three polynomials $\text{row}, \text{col}, \text{val}_{\mathbf{M}} \in \mathbb{F}^{<m}[X]$ interpolating over $\mathbb{K}$ the values

$$\text{row}(k_t) = h_{i_t}, \quad \text{col}(k_t) = h_{j_t}, \quad \text{val}_{\mathbf{M}}(k_t) = \mathbf{M}_{i_t, j_t} \cdot \frac{h_{i_t} h_{j_t}}{n^2}$$

for $t \leq s_{\mathbf{M}}$, and $\text{row}(k_t) = \text{col}(k_t) = h_1$, $\text{val}_{\mathbf{M}}(k_t) = 0$ for the remaining $m - s_{\mathbf{M}}$ points.

The point of Definition 4.2 is that it turns an evaluation of the bivariate polynomial attached to $\mathbf{M}$ (an $\mathcal{R}_P$ statement) into a sum of m terms over $\mathbb{K}$, each of which the verifier can evaluate from a constant number of oracle queries.

Proposition 4.3 (Sparse evaluation of P , [CHM⁺20]). *Let P be the bivariate polynomial of Proposition 2.5 interpolating $\mathbf{M}$ over $\mathbb{H} \times \mathbb{H}$. For every $\alpha, \beta \in \mathbb{F} \setminus \mathbb{H}$,*

$$P(\alpha, \beta) = v_{\mathbb{H}}(\alpha) v_{\mathbb{H}}(\beta) \sum_{k \in \mathbb{K}} \frac{\text{val}_{\mathbf{M}}(k)}{(\alpha - \text{row}(k))(\beta - \text{col}(k))}.$$

Marlin's solution

Proposition 4.3 expresses the claimed value as a sum over $\mathbb{K}$, so it can be discharged by a univariate sumcheck (Lemma 2.35) over $\mathbb{K}$ instead of $\mathbb{H}$. This is Marlin's *inner* sumcheck, and it is the only part of the AHP that is witness-independent. We state it directly for the three-matrix claim of Section 5.1, which is what the implementation proves; the single-matrix case is recovered by setting $\eta = 0$.

Definition 4.4 (Marlin's holographic lincheck, [CHM⁺20, App. E]). Let (α, β, y) be an $\mathcal{R}_P$ statement with $\alpha, \beta \notin \mathbb{H}$ and $y = P_{\mathbf{A}}(\alpha, \beta) + \eta P_{\mathbf{B}}(\alpha, \beta) + \eta^2 P_{\mathbf{C}}(\alpha, \beta)$. The indexer publishes commitments to $\text{row}, \text{col}, \text{val}_{\mathbf{A}}, \text{val}_{\mathbf{B}}, \text{val}_{\mathbf{C}}$ and to the interpolant rowcol of $\text{row} \cdot \text{col}$ over $\mathbb{K}$.

1. Both parties define

$$\begin{aligned} a(X) &:= v_{\mathbb{H}}(\alpha) v_{\mathbb{H}}(\beta) (\text{val}_{\mathbf{A}} + \eta \text{val}_{\mathbf{B}} + \eta^2 \text{val}_{\mathbf{C}})(X), \\ \hat{b}(X) &:= \alpha\beta - \alpha \text{col}(X) - \beta \text{row}(X) + \text{rowcol}(X). \end{aligned}$$

2. $\mathcal{P}$ interpolates $f \in \mathbb{F}^{<m}[X]$ with $f(k) = a(k)/\hat{b}(k)$ for $k \in \mathbb{K}$, writes $f(X) = y/m + X \cdot g_2(X)$ by Lemma 2.35 and sets $h_2(X) := (a(X) - \hat{b}(X)f(X))/v_{\mathbb{K}}(X)$. It sends commitments to g_2, h_2 and to the degree-shifted copy $X^{D-(m-1)}g_2(X)$.
3. $\mathcal{V}$ samples $\gamma \leftarrow \mathbb{F}$ and sends it.
4. $\mathcal{P}$ sends the evaluations at γ of $g_2, h_2, \text{row}, \text{col}, \text{val}_{\mathbf{A}}, \text{val}_{\mathbf{B}}, \text{val}_{\mathbf{C}}$ and rowcol .
5. $\mathcal{V}$ recomputes $a(\gamma)$ and $\hat{b}(\gamma)$ from those evaluations and asserts

$$a(\gamma) - \hat{b}(\gamma) \left(\gamma g_2(\gamma) + \frac{y}{m} \right) = h_2(\gamma) v_{\mathbb{K}}(\gamma).$$

6. $\mathcal{V}$ samples $\eta' \leftarrow \mathbb{F}$; both parties batch the nine commitments (the six of the indexer, g_2, h_2 and the shift) with the powers of η' into a single $\mathcal{R}_{\text{PCO}}$ statement at γ , in which the shifted slot is assigned the value $\gamma^{D-(m-1)}g_2(\gamma)$.

Three points of Definition 4.4 deserve a comment. First, the prover commits only to g_2 and h_2 , and not to f itself: the decomposition $f = y/m + Xg_2$ already encodes the claim $\sum_{k \in \mathbb{K}} f(k) = y$, so substituting it into $a - \hat{b}f = h_2 v_{\mathbb{K}}$ collapses both conditions into the single identity of step (5).

Second, the degree bound $\deg g_2 \leq m - 2$ supplied by Lemma 2.35 is not free: it must be enforced, or a prover could pass for an arbitrary claimed value $y' = y + \Delta$ by replacing

g_2 with $g_2 + cX^{m-1}$ and h_2 with $h_2 - c\hat{b}$, where $c = -\Delta/m$, since the perturbed f then differs from the honest one by $c \cdot v_{\mathbb{K}}$ and so agrees with it on $\mathbb{K}$. The shifted commitment of step (2) rules this out: under a degree- D SRS, $X^{D-(m-1)}g_2(X)$ is committable only when the bound holds, and the batch of step (6) ties the two commitments together at γ . This is the same degree-binding technique that Π_D (Definition 3.7) applies to the promises of the amortization protocol.

Third, the genuine denominator $(\alpha - \text{row}(X))(\beta - \text{col}(X))$ has degree $2(m-1)$, whereas $\hat{b}$ of step (1) has degree at most $m-1$ and agrees with it on $\mathbb{K}$, which is all the argument above needs. Interpolating rowcol once at indexing time therefore halves the degree of the products the prover has to form.

Cost analysis

Proposition 4.5 (Cost of Definition 4.4). *Instantiate Definition 4.4 with the KZG scheme of Definition 2.26 over a degree- D SRS with $D \geq m$, and take the indexer's six commitments as given. Then the prover performs $\mathcal{O}(m \log m) \mathbb{F}$ and $\mathcal{O}(D) \mathbb{G}$; the verifier performs $\mathcal{O}(\log n + \log m) \mathbb{F}$, a nine-term multi-scalar multiplication and two pairing operations; and the proof consists of four group elements and eight field elements.*

The prover's field cost is dominated by the interpolations of f and a over $\mathbb{K}$ and by the product $\hat{b} \cdot f$, all of which are $\mathcal{O}(m \log m)$ once the m inverses $\hat{b}(k)^{-1}$ have been batched into a single field inversion; its group cost is dominated by the commitment to the shifted g_2 and by the batched opening, both of which are multi-scalar multiplications of size D rather than of size m . The shifted commitment $X^{D-(m-1)}g_2(X)$ has at most $m-1$ non-zero coefficients, so a multi-scalar multiplication that pulls out just the bases in the range $[D-(m-1), D-1]$ would cost $\mathcal{O}(m)$, and this is the same saving already exploited for the sparse encodings of Section 5.1. The opening is genuinely $\Theta(D)$: dividing by $(X - \gamma)$ propagates from the leading coefficient downwards, so the quotient is dense over the whole range $[0, D-2]$ even though the batched polynomial itself is not. Any protocol that binds a degree through a commitment at the top of the SRS pays this. The verifier evaluates $v_{\mathbb{H}}(\alpha), v_{\mathbb{H}}(\beta), v_{\mathbb{K}}(\gamma)$ and $\gamma^{D-(m-1)}$ by square-and-multiply and then works with a constant number of field elements.

Two consequences matter for what follows. The indexer runs once and its output is shared by every statement about the same constraint system, which is precisely the setting the amortization protocol targets: the K clients of Figure 3.2 submit K different (α, β, y) but query the same $\text{row}, \text{col}, \text{val}$. And, since the implementation takes m to be the least power of two above the joint sparsity of $(\mathbf{A}, \mathbf{B}, \mathbf{C})$, we have $m = \Theta(s_{\mathbf{M}})$ and the field cost of a single execution of Definition 4.4 is $\mathcal{O}(s_{\mathbf{M}} \log s_{\mathbf{M}}) \mathbb{F}$.

Rewriting the group cost takes one further step, since Proposition 4.5 states it in D and not in m . The degree bound is a free parameter of the SRS, constrained only from below by the largest degree that is ever committed: the lincheck on its own needs $D \geq m$, and the amortization protocol of Chapter 3 needs $D = \mathcal{O}(n + K)$, so the implementation shares one SRS sized for the larger of the two. Under the two hypotheses already carried by Table 3.1 ($s_{\mathbf{M}} = \Theta(n)$ and $s_{\mathbf{M}} = \omega(K)$) that gives $D = \Theta(n + K) = \Theta(s_{\mathbf{M}})$, and hence $\mathcal{O}(D) \mathbb{G} = \mathcal{O}(s_{\mathbf{M}}) \mathbb{G}$. It is worth being explicit that this is the reason the two match: nothing in Definition 4.4 forces the prover's group cost to scale with the sparsity rather than with the SRS, and for a matrix sparse enough that $s_{\mathbf{M}} = o(n)$ the lincheck would cost $\Theta(n) \mathbb{G}$ while the table would predict $\Theta(s_{\mathbf{M}}) \mathbb{G}$. With that in hand, a single execution costs $\mathcal{O}(s_{\mathbf{M}} \log s_{\mathbf{M}}) \mathbb{F} + \mathcal{O}(s_{\mathbf{M}}) \mathbb{G}$, so running it K times independently is exactly

the “without amortization” column of Table 3.2, and this is the baseline against which Section 5.2 measures.

Proposition 4.5 leaves the indexer aside, since it runs before any statement is known. Its cost is that of interpolating $\text{row}, \text{col}, \text{val}_{\mathbf{A}}, \text{val}_{\mathbf{B}}, \text{val}_{\mathbf{C}}$ and rowcol over $\mathbb{K}$ and committing to them: six inverse FFTs, for $\mathcal{O}(m \log m) \mathbb{F}$, and six multi-scalar multiplications of size m , for $\mathcal{O}(m) \mathbb{G}$, with an output of six group elements that the verifier keeps as its preprocessed key. Taken together with Proposition 4.5, the four quantities of interest are therefore an indexer quasi-linear in m and paid once per constraint system, a prover costing $\mathcal{O}(m \log m) \mathbb{F} + \mathcal{O}(D) \mathbb{G}$ per statement, a verifier costing $\mathcal{O}(\log n + \log m) \mathbb{F}$ and two pairings, and a proof of four group and eight field elements. The last two are constant in n and m , which is what makes the lincheck a sensible endpoint for the amortization: whatever K is, the batch ends in one constant-size proof checked in logarithmic time. We note that the implementation resolves each matrix entry by a linear scan when building the encoding of Definition 4.2, so its indexer is quadratic rather than quasi-linear in $s_{\mathbf{M}}$; being one-time preprocessing, it is excluded from every measurement reported in Section 5.2.

Chapter 5

Results

In this chapter we present the implementation and the results of running the amortization protocol presented in Chapter 3. In order to instantiate the folding scheme, we used the Marlin SNARK [CHM⁺20] to produce the initial $K \cdot \mathcal{R}_P$ statements required as inputs and compared the performance of the amortization protocol against running Marlin K times independently.

In Section 5.1 we introduce several important implementation decisions and optimizations. In Section 5.2 we present the benchmarks of our implementation, as well as other relevant results that justify the usefulness of the amortization protocol in practice.

5.1 Implementation Details

This section records the points where the implementation departs from the presentation of Chapter 3, which follows the single-matrix relations of [PRZ26a].

The Three-Matrix Relation

The implementation targets R1CS, whose arithmetization produces the three constraint matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ of Definition 2.14. Rather than running the protocol three times, it instantiates $\mathcal{R}_M$ once with all three matrices at the same pair of vectors. We write $\mathcal{R}_{\mathbf{A}, \mathbf{B}, \mathbf{C}}$ for the resulting relation: its statements carry three values $y_{\mathbf{A}}, y_{\mathbf{B}}, y_{\mathbf{C}}$ asserting

$$y_{\mathbf{X}} = \mathbf{u}^\top \mathbf{X} \mathbf{v}, \quad \mathbf{X} \in \{\mathbf{A}, \mathbf{B}, \mathbf{C}\},$$

against a single pair of committed rational encodings (N_u, T_u) and (N_v, T_v) of $\mathbf{u}$ and $\mathbf{v}$. Because the encodings are shared, folding three bilinear forms costs the same commitments as folding one, and only the three field elements $y_{\mathbf{A}}, y_{\mathbf{B}}, y_{\mathbf{C}}$ are carried along.

The same specialization applies to Π_P (Definition 3.12). The implementation introduces one auxiliary vector $\mathbf{w}_{\mathbf{X}} := \mathbf{X} \mathbf{v}$ per matrix and proves all six halves with a single η -weighted univariate sumcheck, whose combined value is $y_{\mathbf{A}} + \eta y_{\mathbf{B}} + \eta^2 y_{\mathbf{C}}$. The verifier's reconstruction depends only on the linear combination $\mu_{\mathbf{A}} + \eta \mu_{\mathbf{B}} + \eta^2 \mu_{\mathbf{C}}$ of the three bivariate evaluations, so the prover ships a single field element instead of three, and the output $\mathcal{R}_P$ statement asserts

$$y = P_{\mathbf{A}}(\alpha, \beta) + \eta P_{\mathbf{B}}(\alpha, \beta) + \eta^2 P_{\mathbf{C}}(\alpha, \beta).$$

Figure 5.1 is Figure 3.2 with $\mathcal{R}_M$ replaced by $\mathcal{R}_{\mathbf{A}, \mathbf{B}, \mathbf{C}}$; the statement counts and the reductions are unchanged.

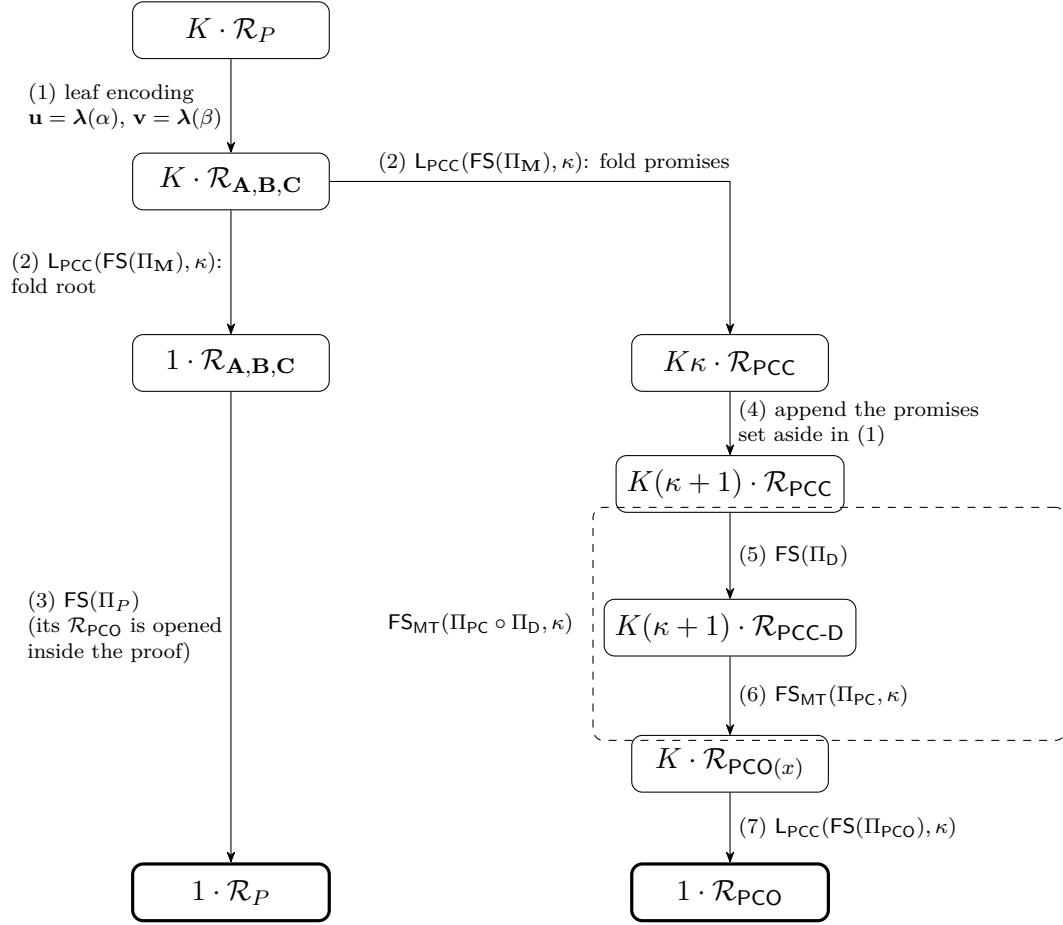


Figure 5.1: The amortization protocol as implemented, with the three-matrix relation $\mathcal{R}_{\mathbf{A},\mathbf{B},\mathbf{C}}$ in place of $\mathcal{R}_{\mathbf{M}}$. Vertices give the number of open statements of each relation and edges the reductions consuming them; both are identical to Figure 3.2.

Correspondence with the Source Tree

The implementation names the reductions after their Rust modules, which do not always match the names used in Chapter 3. Table 5.1 gives the correspondence. Throughout the thesis we use the names of [PRZ26a]; in particular we write Π_D for the degree-test reduction of Definition 3.7, which the source tree calls `RokDt`.

Marlin itself sits outside that table. The two rounds of Section 4.1, which produce the statement that Definition 4.4 discharges, live in `marlin_ahp::outer_sumcheck`, ported from the reference `arkworks` implementation and re-typed onto the KZG and Blake3 machinery described below; the lincheck itself is `marlin_ahp::inner_lincheck`. Keeping both under one roof is what allows a full Marlin proving time to be measured in one place and split into its witness-dependent and witness-independent halves, which is the measurement reported in Section 5.2.

Thesis	Source module	Aggregate map
Π_M	<code>reductions::rok_abc</code>	$\mathcal{R}_{A,B,C}^2 \rightarrow \mathcal{R}_{A,B,C} \times \mathcal{R}_{PCC}$
Π_D	<code>reductions::rok_dt</code>	$\mathcal{R}_{PCC} \rightarrow \mathcal{R}_{PCC-D}$
Π_{PC}	<code>reductions::rok_pcc</code>	$\mathcal{R}_{PCC-D} \rightarrow \mathcal{R}_{PCO(x)}$
Π_{PCO}	<code>reductions::rok_pco</code>	$\mathcal{R}_{PCO(x)}^\ell \rightarrow \mathcal{R}_{PCO(x)}$
Π_P	<code>reductions::rok_p</code>	$\mathcal{R}_{A,B,C} \rightarrow \mathcal{R}_P \times \mathcal{R}_{PCO}$
$L_{PCC}(FS(\Pi_M), \kappa)$	<code>reductions::abc_fold</code>	$K \cdot \mathcal{R}_{A,B,C} \rightarrow \mathcal{R}_{A,B,C} + K\kappa \cdot \mathcal{R}_{PCC}$
$FS_{MT}(\Pi_{PC} \circ \Pi_D, \kappa)$	<code>reductions::fsmt_pcc</code>	$(\kappa + 1) \cdot \mathcal{R}_{PCC} \rightarrow \mathcal{R}_{PCO(x)}$ per leaf
$L_{PCC}(FS(\Pi_{PCO}), \kappa)$	<code>reductions::pco_fold</code>	$K \cdot \mathcal{R}_{PCO(x)} \rightarrow \mathcal{R}_{PCO}$
—	<code>reductions::discharge</code>	$K(\kappa + 1) \cdot \mathcal{R}_{PCC} \rightarrow \mathcal{R}_{PCO}$

Table 5.1: Correspondence between the names used in this thesis and the modules of the implementation. The last block lists the compiled schemes and the composite discharge, which has no separate name in [PRZ26a].

Instantiation

The implementation is written in Rust on top of `arkworks`, over the BLS12-381 curve with scalar field $\mathbb{F}_r$. The polynomial commitment scheme is KZG (Definition 2.26), whose additive homomorphism is what makes the random linear combinations of Π_{PCO} and the batching step of Π_{PC} possible. The Fiat–Shamir transform (Definition 2.31) is instantiated with a Blake3 transcript with domain-separated forks, and the Merkle commitments binding the K leaves before the fold challenges are squeezed use the same hash.

Three engineering points are worth recording, since they affect the measured prover cost of Section 5.2 without changing the protocol. The first concerns how the single root witness is assembled.

Read literally, the last two steps of Figure 3.2 build it in two stages. First, $FS_{MT}(\Pi_{PC} \circ \Pi_D, \kappa)$ leaves each leaf i with the $\mathcal{R}_{PCO(x)}$ witness

$$p_i(X) = \sum_{k=0}^{m_i-1} r_i^k p_{i,k}(X),$$

the r_i -weighted batch of that leaf’s promise polynomials (step 3 of Definition 3.8). Then

$L_{\text{PCC}}(\text{FS}(\Pi_{\text{PCO}}), \kappa)$ folds the K witnesses up a binary tree, each node combining a pair as $p' = p_0 + r p_1$ (Definition 3.11), until a single P_{root} remains after κ levels.

That path is expensive for a reason that has nothing to do with the protocol. The promise polynomials $p_{i,k}$ are sparse, but their batch p_i is dense of degree $\mathcal{O}(n + K)$, so the first stage densifies K polynomials of that size and the second builds $K - 1$ more at the internal nodes: $\mathcal{O}(K(n + K))$ field operations in total, with the K leaf witnesses and the κ levels of intermediate nodes all materialized.

None of that structure is needed. Both stages are linear in the witnesses, so unrolling the tree expresses the root as one fixed linear combination of the original promise polynomials,

$$P_{\text{root}}(X) = \sum_{i=0}^{K-1} \text{coeff}_i \cdot \left(\sum_{k=0}^{m_i-1} r_i^k p_{i,k}(X) \right),$$

where

- $i \in \{0, \dots, K - 1\}$ indexes the leaves, one per verifier;
- $p_{i,0}(X), \dots, p_{i,m_i-1}(X)$ is the flat polynomial basis of leaf i , namely the witness polynomials of its $\kappa + 1$ $\mathcal{R}_{\text{PCC}}$ promises after Π_{D} (each promise's committed polynomials together with their degree-shifted copies) concatenated in bundle order, so that $m_i = \sum_j 2n_{i,j}$ with $n_{i,j}$ the number of committed polynomials of the j -th promise of leaf i ;
- $r_i \in \mathbb{F}$ is the batching challenge that Π_{PC} squeezes for leaf i (step 3 of Definition 3.8), so that the inner sum $\sum_k r_i^k p_{i,k}(X)$ is precisely leaf i 's $\mathcal{R}_{\text{PCO}(x)}$ witness;
- $\text{coeff}_i \in \mathbb{F}$ is the weight with which leaf i enters the folded root. Since Π_{PCO} folds a pair as $c' = c_0 + r c_1$ (Definition 3.11), it is the product of the challenges of the fold nodes along leaf i 's path to the root at which that leaf lies in the right child, and $\text{coeff}_i = 1$ when the leaf is a left child at every level;
- $P_{\text{root}}(X)$ is the resulting single $\mathcal{R}_{\text{PCO}}$ witness, opened at the point x shared by all K executions.

The implementation accumulates this double sum in one pass over the sparse $p_{i,k}$, so no leaf witness and no internal node is ever built: the $\mathcal{O}(K(n + K))$ densification disappears, leaving a single $\mathcal{O}(n + K)$ opening plus an $\mathcal{O}(\kappa K^2)$ sparse accumulation that is independent of n . The saving is confined to the prover's bookkeeping. Both stages still run in full on statements and commitments, which is where the transcript is written, so every challenge, statement and proof is identical to what the literal two-stage computation would produce.

Second, the polynomials produced by interpolation are dense, while the (N, T) encodings are genuinely sparse, so the implementation keeps the two representations apart and converts only where they meet. The distinction matters in Π_P , whose pairwise products $u \cdot w$, $\Lambda(\alpha, \cdot) \cdot w$ and $M(\alpha, \cdot) \cdot v$ cost $\mathcal{O}(n^2)$ when carried out on a sparse representation but $\mathcal{O}(n \log n)$ under the FFT-based multiplication of the dense one; the (N, T) pairs, which have at most three non-zero coefficients at a leaf, stay sparse and are converted only at the batched accumulator.

Third, commitments to sparse polynomials use a multi-scalar multiplication whose size is the number of non-zero terms rather than the degree bound, by pulling only the corresponding bases out of the structured reference string. This is what makes the cost of committing scale with the sparsity of a polynomial rather than with the degree bound D ,

and it is the reason the protocol’s use of the three-coefficient encodings of Proposition 2.4 translates into a concrete saving: at a leaf, committing to N_u, T_u, N_v, T_v costs a constant number of group operations instead of $\mathcal{O}(n)$. It is also what the group-operation column of Table 3.1 assumes throughout.

5.2 Benchmarks

In this section we present the results (and main contributions) of our implementation of the amortization protocol presented in Chapter 3, when instantiated with the Marlin SNARK. We compare the performance of the amortization protocol against running Marlin K times independently, for different circuit sizes and different number of independent statements. We also present experimental results that aim to answer the following questions:

1. Is verifying the protocol’s proof faster than running the local SNARK?
2. What percentage of the total cost of Marlin is witness-dependent and witness-independent?

All the measurements below were taken on a single laptop: an AMD Ryzen 9 270 (8 cores, 16 threads, 4.0 GHz boost) with 64 GB of DDR5-5600 memory, running Windows 11 and Rust 1.94 with the release profile. Since both sides of every comparison run on the same machine, what matters for the conclusions is the ratio between the two curves rather than the absolute timings, which would shift on different hardware.

The following metrics are based on running the Marlin SNARK for different circuit sizes $n \in \{2^{10}, 2^{12}, \dots, 2^{20}\}$ and different number of independent statements (or verifiers) $K \in \{2^0, 2^1, \dots, 2^7\}$. The statements are generated by fixing the sparsity of the R1CS matrices to $s_M = d \cdot n$ and generating random R1CS instances with different values of d .

Amortization protocol

The first two results of Figure 5.2 and Figure 5.3 show the comparison between:

- The cost of running the amortization protocol,
- The baseline cost of running K independent linchecks.

The figures only consider the inner sumcheck of Marlin’s lincheck for the cost comparison. This is because, in both cases, we would run the witness-dependent part of Marlin for the K statements and, therefore, its cost would be the same. For this set of experiments, the density is fixed to the case $d = 2$, so the two families of experiments are run on matrices of the same shape. We also note that the figures show results for $n \in \{2^{10}, 2^{12}, \dots, 2^{20}\}$.

Table 5.2 repeats Table 3.2, which is what the measurements below should be read against: a prover cost that is linear in K on the baseline side (without amortization) and, on the amortized side, linear in K only in the $K \cdot s_M$ term while everything else grows as $K \log K$ or slower; and a verifier that pays an extra $\log K$.

Figure 5.2 shows the cost comparison for a prover running the amortization protocol vs. the baseline cost of running K independent linchecks. The horizontal axis represents the number of independent statements K and the vertical axis represents the time in

	With amortization	Without amortization
Prover time	$(K \cdot s_{\mathbf{M}} + s_{\mathbf{M}} \cdot \log s_{\mathbf{M}}) \mathbb{F} + (K \cdot \log K + s_{\mathbf{M}}) \mathbb{G}$	$K(s_{\mathbf{M}} \cdot \log s_{\mathbf{M}} \mathbb{F} + s_{\mathbf{M}} \mathbb{G})$
Verifier time	$2 \cdot \mathbf{P} + \log n \mathbb{F} + \log K \cdot (\mathbb{F} + \mathbb{G} + \rho)$	$1 \cdot (\mathbb{F} + \mathbb{G} + \rho + \mathbf{P})$

Table 5.2: Expected asymptotic behavior of the two sides of the comparison, reproduced from Table 3.2. Costs are measured in group ($\mathbb{G}$), field ($\mathbb{F}$), random oracle queries (ρ) and pairing ($\mathbf{P}$) operations, with $s_{\mathbf{M}}$ the sparsity of the constraint system. The proof-size row of Table 3.2 is omitted, since the measurements below report time only. The non-amortized column is K executions of Definition 4.4, by Proposition 4.5.

milliseconds. The blue line is the baseline cost of running K independent linchecks, which shows a clear linear dependency on the number of independent statements K as expected from $K(s_{\mathbf{M}} \cdot \log s_{\mathbf{M}} \mathbb{F} + s_{\mathbf{M}} \mathbb{G})$. The orange line is the cost of running the amortization protocol and presents a superlinear shape that tends to flatten out as the size of the circuit grows. Since the group operations dominate the prover time, the superlinear shape can be attributed to the $(K \cdot \log K + s_{\mathbf{M}}) \mathbb{G}$ term, which will stay under the baseline $K \cdot s_{\mathbf{M}} \mathbb{G}$ as long as $s_{\mathbf{M}} = \omega(K)$ (which is a reasonable assumption for real-world applications). In other words, the prover time is dominated by the sparsity term $s_{\mathbf{M}} \mathbb{G}$, while the superlinear term $K \cdot \log K \mathbb{G}$ becomes negligible. The results show that the two curves intersect at $K = 4$ in all experiments with circuit sizes $n \geq 2^{12}$. Real-world applications have circuit sizes that are much larger than $n = 2^{12}$: the evaluation of Marlin itself already spans 2^{10} to 2^{20} constraints [CHM⁺20, Fig. 2], and the circuits that motivate distributed proving for zkRollups and zkEVMs are larger still, with [LXZ⁺24] reporting measurements on circuits of 2^{21} to 2^{25} gates. The crossover therefore falls well inside the regime of interest, so we can conclude that the amortization protocol can provide significant gains. The size of the gain past the crossover grows with the circuit, which is what the $s_{\mathbf{M}}$ -dominated shape predicts: at $K = 128$ the amortized prover is 1.3 times faster than the baseline at $n = 2^{12}$, 9.5 times at $n = 2^{16}$ and 21 times at $n = 2^{20}$, where the measured times are $1.7 \cdot 10^6$ ms without amortization against $8.1 \cdot 10^4$ ms with it.

Figure 5.3 shows the parallel cost comparison for a verifier in the same setting (i.e. amortization protocol vs. baseline of K independent linchecks). As expected from the cost analysis in Table 5.2, the verifier’s cost in the amortization protocol includes an added logarithmic dependency on K due to the proof of inclusion in the folding process. We can also observe that, for $K = 1$, the cost of checking the amortization includes the extra pairing operation from the reduction of knowledge $\mathbf{FS}(\Pi_P)$. The baseline is constant in K , as predicted, and also constant in n : it stays at ~ 4 ms across the whole sweep, since the lincheck verifier is succinct. The amortized verifier starts at ~ 8 ms for $K = 1$ and reaches ~ 35 ms at $K = 128$, an increase of roughly $\times 9$ over the baseline; it too is essentially independent of n , so the price of amortization is paid in $\log K$ alone.

As mentioned in Section 1.1, the amortization protocol can be framed in the setup of a delegation service, where a local prover (the protocol’s verifier) wants to avoid running the witness-independent part and decides to outsource it to a server (the protocol’s prover). In this section’s previous results, we have shown that the server does amortize the computational cost of proving the K statements, while the verifier pays a logarithmic price. Figure 5.4 aims to show that this extra cost for the verifier constitutes less work than proving the witness-independent part by itself (i.e. running the SNARK as a local prover and avoiding any delegation). This is the first question posed at the beginning of this section.

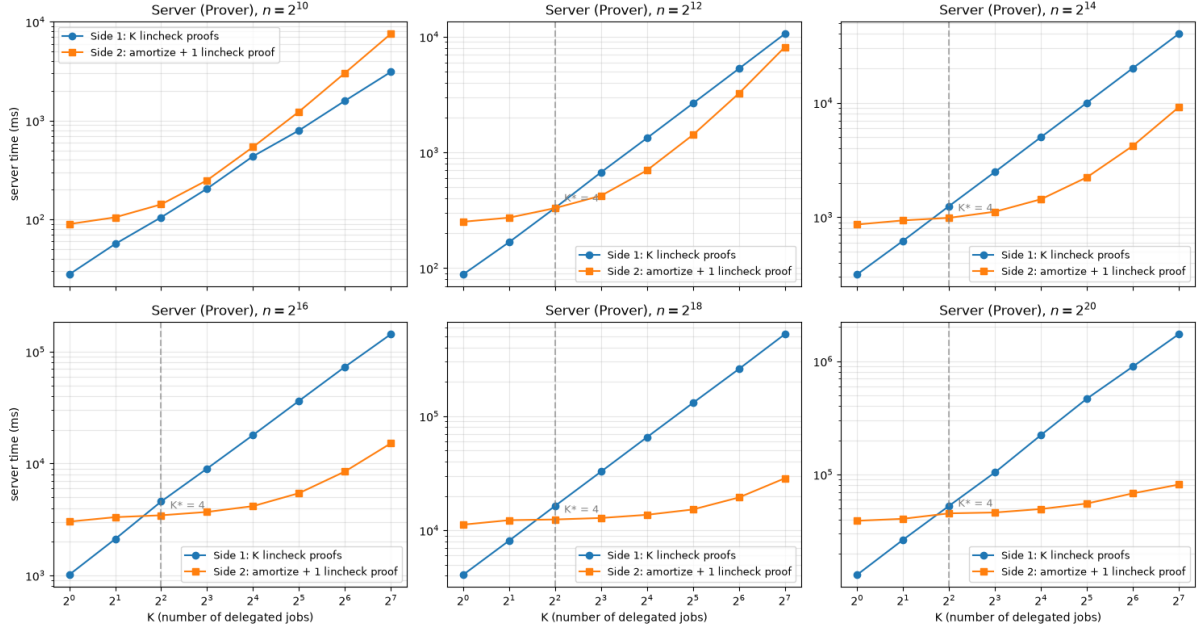


Figure 5.2: Prover’s cost of running K linchecks vs. running the whole amortization.

The dashed line is the cost of a local prover proving its own $\mathcal{R}_P$ statement, that is, running Marlin’s inner sumcheck itself; the solid line is the cost of delegating it and verifying the resulting amortized proof. Both are local-party costs, so the gap between them is the gain from delegating.

The dashed line is flat in K but grows with the circuit, since a single execution costs $\mathcal{O}(s_M \log s_M) \mathbb{F} + \mathcal{O}(s_M) \mathbb{G}$ by Proposition 4.5. The solid line is the verifier cost of Figure 5.3, which is independent of n and grows only as $\log K$. The gain therefore widens with n : at $K = 128$ delegating is $\times 2.4$ cheaper than proving locally at $n = 2^{12}$, $\times 28$ at $n = 2^{16}$ and $\times 325$ at $n = 2^{20}$.

The exception is the smallest circuit. At $n = 2^{10}$ the local proof costs ~ 28 ms, which the amortized verification overtakes between $K = 32$ (27.4 ms, still cheaper) and $K = 64$ (29.8 ms, no longer cheaper), ending at $\times 0.8$ the local cost for $K = 128$. For circuits that small the client is better off proving on its own, which is the expected behavior: the $\log K$ price of the folding path can only be amortized when the work being delegated is large enough.

Witness-independent percentage

Figure 5.5 and Figure 5.6 aim to answer the second question posed at the beginning of this section:

What percentage of the total cost of Marlin is witness-dependent and witness-independent?

Figure 5.5 compares both parts when varying the circuit size n over $\{2^{10}, 2^{12}, \dots, 2^{18}\}$ and when toggling the zero-knowledge property on and off. These two experiments measure a full Marlin proof rather than the amortization, so they stop one step short of the $n = 2^{20}$ reached above. The results are split into the three rounds composing Marlin’s AHP: round 1 and 2 for the witness-dependent part, and round 3 for the witness-independent part. There is an intermediate step between rounds 2 and 3 that bridges

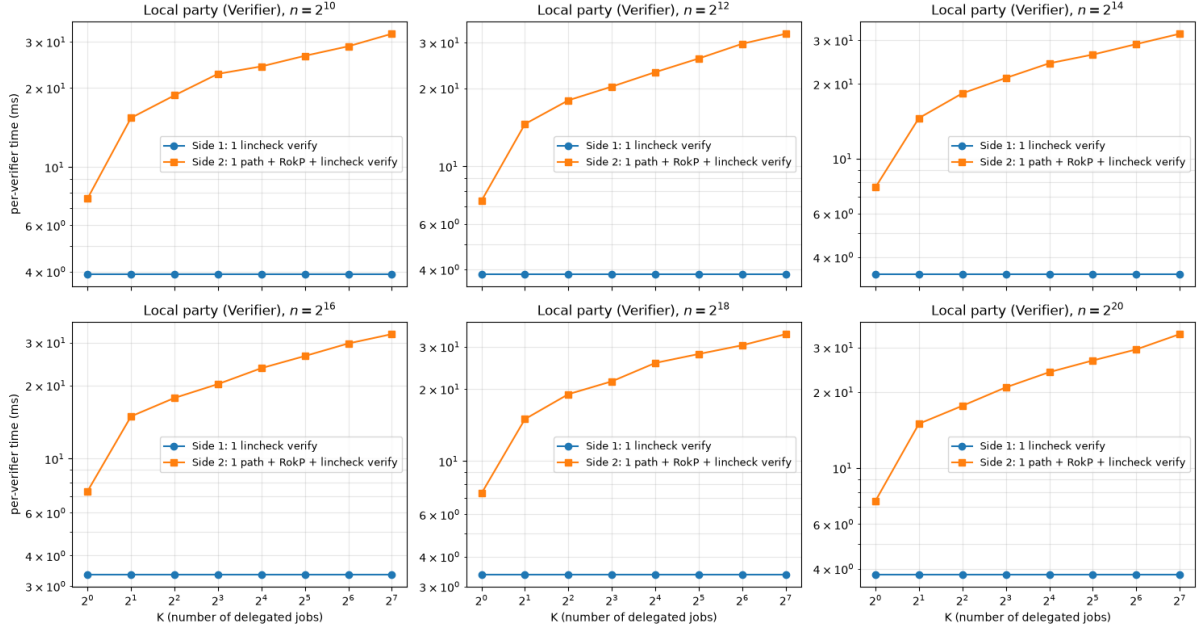


Figure 5.3: Verifier’s cost of checking 1 lincheck vs. checking the whole amortization.

the output of rounds 1 and 2 into the input of round 3 ($t(X)$ (*witness-independent*) in Figure 5.5), but its cost is negligible compared to the other two parts. An extended explanation about this intermediate step can be found in Section 4.1. The results show that $\sim 40\%$ of the total cost of Marlin is witness-independent, which is a large enough percentage to justify the gains given by the amortization protocol. The experiments also seem to indicate that the size of the circuit is unrelated to the percentages. This reflects the cost analysis from Proposition 4.5, where the prover’s time is dependent on the sparsity of the matrices, not the total size of the circuit.

Figure 5.6 aims to show the dependency on the sparsity of the matrices. The experiments follow the same setting as the previous ones, but with variable sparsity instead of variable circuit size. The results show an increasing witness-independent percentage as the sparsity of the matrices increases, which provides a stronger argument for the amortization protocol, with up to $\sim 60\%$ of the total cost of Marlin being witness-independent for the denser matrices.

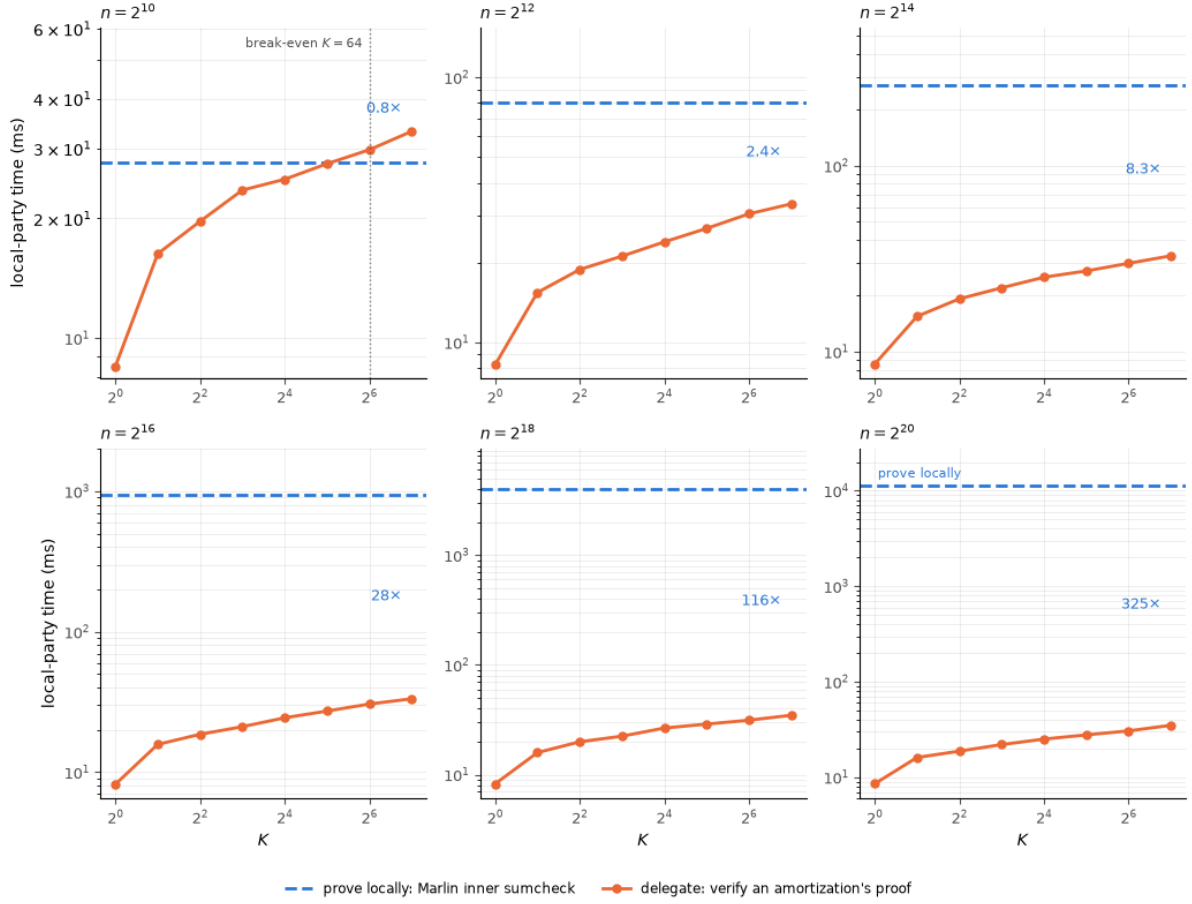


Figure 5.4: Cost for a local party of proving its own $\mathcal{R}_P$ statement with Marlin’s inner sumcheck (dashed) against delegating it and verifying the amortized proof (solid), for each circuit size. The annotation gives the ratio between the two at $K = 128$.

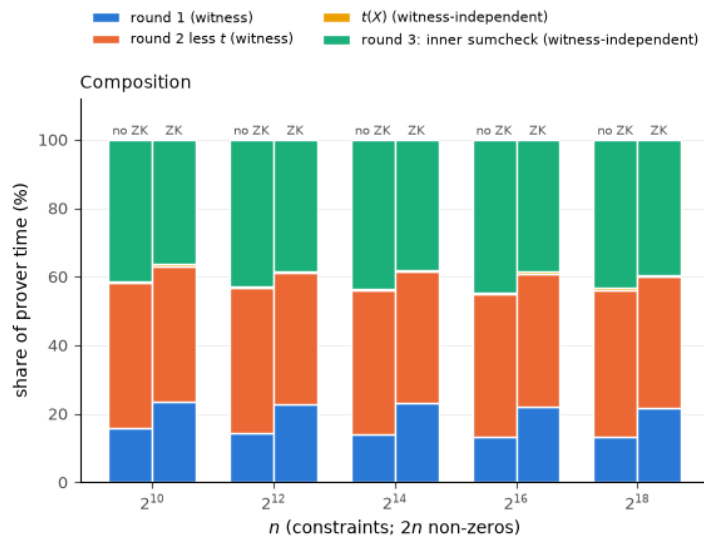


Figure 5.5: Percentage of witness-dependent and witness-independent parts of Marlin in the whole proof, with and without zk. Varying n .

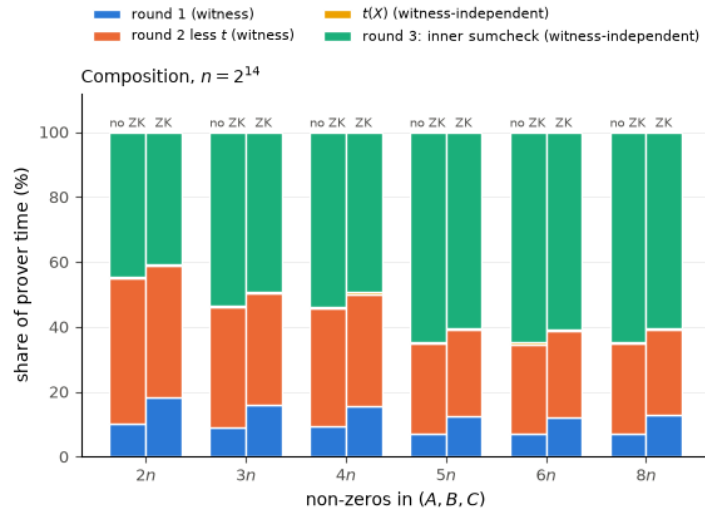


Figure 5.6: Percentage of witness-dependent and witness-independent parts of Marlin in the whole proof, with and without zk. Varying matrix density.

Chapter 6

Conclusion and Future Work

In this chapter, we summarize the contributions of this thesis and discuss potential directions for future research.

6.1 Conclusion

Reductions of knowledge are a powerful tool in the design of zero-knowledge proof systems, especially in the construction of recursive proofs. The work of [PRZ26a] explores their potential in the context of a single prover interacting with several independent verifiers. These verifiers want a proof for a public statement over some common circuit. These public statements are modeled as bivariate polynomial evaluations, which are common in many SNARK families and can be seen as witness-independent statements.

The work of this thesis analyzes the amortization protocol proposed in [PRZ26a] and provides a concrete implementation of it. The results show that the protocol delivers the expected computational gains when compared against running K independent Marlin [CHM⁺20] proofs. The prover can reduce its cryptographic cost from $K \cdot s_{\mathbf{M}} \mathbb{G}$ to $(K \cdot \log K + s_{\mathbf{M}}) \mathbb{G}$, as long as the sparsity of the circuit satisfies $s_{\mathbf{M}} = \omega(K)$. The experiments show a prover time drop from $\sim 1.7 \cdot 10^6$ ms to $\sim 8 \cdot 10^4$ ms for $K = 128$ and $n = 2^{20}$, which is a $\sim 95\%$ reduction in a real-world setting. The two sides cross at $K = 4$ for every circuit size at or above $n = 2^{12}$, and the gain past that point widens with the circuit, from $\times 1.3$ at $n = 2^{12}$ to $\times 21$ at $n = 2^{20}$.

In contrast, the verifier’s cost increases from constant work to logarithmic work in K , due to the proof of inclusion in the folding process. The experiments show a verifier time increase from ~ 4 ms to $\sim 3.5 \cdot 10^1$ ms in the same setting, which constitutes an increase of roughly $\times 9$. Further results show that this overhead is minimal compared to the cost of running the witness-independent part locally, with up to a $\times 325$ speedup.

The experiments also show that the computational cost for the witness-independent part of Marlin ranges from $\sim 40\%$ to $\sim 60\%$, which is enough to justify the possible interest in a delegation server.

6.2 Future Work

The thesis opens several avenues for future research:

- The amortization protocol can be applied to other proof systems that reduce the witness-independent part to a bivariate polynomial evaluation, such as

Lunar [CFF⁺21] or Basilisk [RZ21]. It would be interesting to analyze the computational gains in those settings.

- Extend the amortization study to different SNARK families on two fronts: (i) multivariate commitments (e.g. Spartan [Set20], Samaritan [GPS25]); and (ii) non-homomorphic commitments based on hashing (e.g. Fractal [COS20]).
- Implement a full end-to-end protocol, combining collaborative SNARKs [OB22] to process the witness-dependent part with the amortized witness-independent part.

Bibliography

- [Bab85] László Babai. Trading group theory for randomness. In *17th ACM STOC*, pages 421–429. ACM Press, May 1985.
- [BB04] Dan Boneh and Xavier Boyen. Short signatures without random oracles. In Christian Cachin and Jan Camenisch, editors, *EUROCRYPT 2004*, volume 3027 of *LNCS*, pages 56–73. Springer, Berlin, Heidelberg, May 2004.
- [BBB⁺18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE Symposium on Security and Privacy*, pages 315–334. IEEE Computer Society Press, May 2018.
- [BBHR19] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable zero knowledge with no trusted setup. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 701–732. Springer, Cham, August 2019.
- [BCC88] Gilles Brassard, David Chaum, and Claude Crépeau. Minimum disclosure proofs of knowledge. *Journal of Computer and System Sciences*, 37(2):156–189, 1988.
- [BCL⁺21] Benedikt Bünz, Alessandro Chiesa, William Lin, Pratyush Mishra, and Nicholas Spooner. Proof-carrying data without succinct arguments. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 681–710, Virtual Event, August 2021. Springer, Cham.
- [BCMS20] Benedikt Bünz, Alessandro Chiesa, Pratyush Mishra, and Nicholas Spooner. Recursive proof composition from accumulation schemes. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part II*, volume 12551 of *LNCS*, pages 1–18. Springer, Cham, November 2020.
- [BCPR14] Nir Bitansky, Ran Canetti, Omer Paneth, and Alon Rosen. On the existence of extractable one-way functions. In David B. Shmoys, editor, *46th ACM STOC*, pages 505–514. ACM Press, May / June 2014.
- [BCR⁺19] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. Aurora: Transparent succinct arguments for R1CS. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part I*, volume 11476 of *LNCS*, pages 103–128. Springer, Cham, May 2019.

- [BFM88] Manuel Blum, Paul Feldman, and Silvio Micali. Non-interactive zero-knowledge and its applications (extended abstract). In *20th ACM STOC*, pages 103–112. ACM Press, May 1988.
- [BFS20] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent SNARKs from DARK compilers. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 677–706. Springer, Cham, May 2020.
- [BG93] Mihir Bellare and Oded Goldreich. On defining proofs of knowledge. In Ernest F. Brickell, editor, *CRYPTO’92*, volume 740 of *LNCS*, pages 390–420. Springer, Berlin, Heidelberg, August 1993.
- [BGH19] Sean Bowe, Jack Grigg, and Daira Hopwood. Halo: Recursive proof composition without a trusted setup. Cryptology ePrint Archive, Report 2019/1021, 2019.
- [BMNW25] Benedikt Bünz, Pratyush Mishra, Wilson Nguyen, and William Wang. Arc: Accumulation for reed-solomon codes. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VII*, volume 16006 of *LNCS*, pages 128–160. Springer, Cham, August 2025.
- [CFF⁺21] Matteo Campanelli, Antonio Faonio, Dario Fiore, Anaïs Querol, and Hadrián Rodríguez. Lunar: A toolbox for more efficient universal and updatable zk-SNARKs and commit-and-prove extensions. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part III*, volume 13092 of *LNCS*, pages 3–33. Springer, Cham, December 2021.
- [CHM⁺20] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas P. Ward. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 738–768. Springer, Cham, May 2020.
- [CLMZ23] Alessandro Chiesa, Ryan Lehmkuhl, Pratyush Mishra, and Yinuo Zhang. Eos: Efficient private delegation of zkSNARK provers. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023*, pages 6453–6469. USENIX Association, August 2023.
- [CNR⁺22] Matteo Campanelli, Anca Nitulescu, Carla Ràfols, Alexandros Zacharakis, and Arantxa Zapico. Linear-map vector commitments and their practical applications. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part IV*, volume 13794 of *LNCS*, pages 189–219. Springer, Cham, December 2022.
- [COS20] Alessandro Chiesa, Dev Ojha, and Nicholas Spooner. Fractal: Post-quantum and transparent recursive proofs from holography. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 769–793. Springer, Cham, May 2020.
- [DH76] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976.

- [FKL18] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 33–62. Springer, Cham, August 2018.
- [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *CRYPTO’86*, volume 263 of *LNCS*, pages 186–194. Springer, Berlin, Heidelberg, August 1987.
- [Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In Michael Mitzenmacher, editor, *41st ACM STOC*, pages 169–178. ACM Press, May / June 2009.
- [GGJ⁺25] Sanjam Garg, Aarushi Goel, Abhishek Jain, Bhaskar Roberts, and Sruthi Sekar. Malicious security in collaborative zk-SNARKs: More than meets the eye. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VII*, volume 16006 of *LNCS*, pages 396–428. Springer, Cham, August 2025.
- [GGPR13] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. Quadratic span programs and succinct NIZKs without PCPs. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 626–645. Springer, Berlin, Heidelberg, May 2013.
- [GMR89] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.
- [GPS25] Chaya Ganesh, Sikhar Patranabis, and Nitin Singh. Samaritan: Linear-time prover SNARK from new multilinear polynomial commitments. In Goichiro Hanaoka and Bo-Yin Yang, editors, *ASIACRYPT 2025, Part V*, volume 16249 of *LNCS*, pages 456–489. Springer, Singapore, December 2025.
- [Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 305–326. Springer, Berlin, Heidelberg, May 2016.
- [GW11] Craig Gentry and Daniel Wichs. Separating succinct non-interactive arguments from all falsifiable assumptions. In Lance Fortnow and Salil P. Vadhan, editors, *43rd ACM STOC*, pages 99–108. ACM Press, June 2011.
- [GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Report 2019/953, 2019.
- [Kil92] Joe Kilian. A note on efficient zero-knowledge proofs and arguments (extended abstract). In *24th ACM STOC*, pages 723–732. ACM Press, May 1992.

- [KP23] Abhiram Kothapalli and Bryan Parno. Algebraic reductions of knowledge. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 669–701. Springer, Cham, August 2023.
- [KS24] Abhiram Kothapalli and Srinath T. V. Setty. HyperNova: Recursive arguments for customizable constraint systems. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part X*, volume 14929 of *LNCS*, pages 345–379. Springer, Cham, August 2024.
- [KST22] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 359–388. Springer, Cham, August 2022.
- [KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 177–194. Springer, Berlin, Heidelberg, December 2010.
- [LFKN92] Carsten Lund, Lance Fortnow, Howard J. Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. *Journal of the ACM*, 39(4):859–868, 1992.
- [LXZ⁺24] Tianyi Liu, Tiancheng Xie, Jiaheng Zhang, Dawn Song, and Yupeng Zhang. Pianist: Scalable zkRollups via fully distributed zero-knowledge proofs. In *2024 IEEE Symposium on Security and Privacy*, pages 1777–1793. IEEE Computer Society Press, May 2024.
- [LZW⁺25] Xuanming Liu, Zhelei Zhou, Yinghao Wang, Yanxin Pang, Jinye He, Bingsheng Zhang, Xiaohu Yang, and Jiaheng Zhang. Scalable collaborative zk-SNARK and its application to fully distributed proof delegation. In Lujo Bauer and Giancarlo Pellegrino, editors, *USENIX Security 2025*, pages 2025–2044. USENIX Association, August 2025.
- [Mic94] Silvio Micali. CS proofs (extended abstracts). In *35th FOCS*, pages 436–453. IEEE Computer Society Press, November 1994.
- [OB22] Alex Ozdemir and Dan Boneh. Experimenting with collaborative zk-SNARKs: Zero-knowledge proofs for distributed secrets. In Kevin R. B. Butler and Kurt Thomas, editors, *USENIX Security 2022*, pages 4291–4308. USENIX Association, August 2022.
- [PRZ26a] Nikitas Paslis, Carla Ràfols, and Alexandros Zacharakis. Amortized multi-verifier proofs from reductions of knowledge. Cryptology ePrint Archive, Report 2026/1553, 2026. <https://eprint.iacr.org/2026/1553>.
- [PRZ26b] Nikitas Paslis, Carla Ràfols, and Alexandros Zacharakis. Proof-carrying data via holography accumulation. Cryptology ePrint Archive, Report 2026/538, 2026. <https://eprint.iacr.org/2026/538>.
- [RAD⁺78] Ronald L Rivest, Len Adleman, Michael L Dertouzos, et al. On data banks and privacy homomorphisms. *Foundations of secure computation*, 4(11):169–180, 1978.

- [RZ21] Carla Ràfols and Arantxa Zapico. An algebraic framework for universal and updatable SNARKs. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 774–804, Virtual Event, August 2021. Springer, Cham.
- [RZ23] Carla Ràfols and Alexandros Zacharakis. Folding schemes with selective verification. In Abdelrahman Aly and Mehdi Tibouchi, editors, *LATIN-CRYPT 2023*, volume 14168 of *LNCS*, pages 229–248. Springer, Cham, October 2023.
- [Sch80] Jacob T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *Journal of the ACM*, 27(4):701–717, 1980.
- [SCP⁺22] Shravan Srinivasan, Alexander Chepurnoy, Charalampos Papamanthou, Alin Tomescu, and Yupeng Zhang. Hyperproofs: Aggregating and maintaining proofs in vector commitments. In Kevin R. B. Butler and Kurt Thomas, editors, *USENIX Security 2022*, pages 3001–3018. USENIX Association, August 2022.
- [Set20] Srinath Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part III*, volume 12172 of *LNCS*, pages 704–737. Springer, Cham, August 2020.
- [SVdV16] Berry Schoenmakers, Meilof Veeningen, and Niels de Vreede. Trinocchio: Privacy-preserving outsourcing by distributed verifiable computation. In Mark Manulis, Ahmad-Reza Sadeghi, and Steve Schneider, editors, *ACNS 2016*, volume 9696 of *LNCS*, pages 346–366. Springer, Cham, June 2016.
- [TCZ⁺20] Alin Tomescu, Robert Chen, Yiming Zheng, Ittai Abraham, Benny Pinkas, Guy Golan-Gueta, and Srinivas Devadas. Towards scalable threshold cryptosystems. In *2020 IEEE Symposium on Security and Privacy*, pages 877–893. IEEE Computer Society Press, May 2020.
- [Tha22] Justin Thaler. *Proofs, Arguments, and Zero-Knowledge*, volume 4(2–4) of *Foundations and Trends in Privacy and Security*. now Publishers, 2022.
- [YLF⁺22] Thomas Yurek, Licheng Luo, Jaiden Fairoze, Aniket Kate, and Andrew Miller. hbACSS: How to robustly share many secrets. In *NDSS 2022*. The Internet Society, April 2022.
- [Zip79] Richard Zippel. Probabilistic algorithms for sparse polynomials. In *Symbolic and Algebraic Computation (EUROSAM '79)*, volume 72 of *LNCS*, pages 216–226. Springer, 1979.
- [ZXH⁺22] Jiaheng Zhang, Tiancheng Xie, Thang Hoang, Elaine Shi, and Yupeng Zhang. Polynomial commitment with a one-to-many prover and applications. In Kevin R. B. Butler and Kurt Thomas, editors, *USENIX Security 2022*, pages 2965–2982. USENIX Association, August 2022.